TRƯỜNG ĐẠI HỌC QUỐC TẾ HỒNG BÀNG
BỘ MÔN CÔNG NGHỆ THÔNG TIN

ĐỒ ÁN TỐT NGHIỆP

ĐỀ TÀI: "THIẾT KẾ HỆ HỖ TRỢ QUẢN TRỊ CƠ SỞ TRI THỨC DẠNG COKB"

GVHD: Gs. ĐỖ VĂN NHƠN & Ths. MAI TRUNG THÀNH
SVTH: LÊ CHÂU TRẦN PHÁT - 2211110068
Lớp: TH22DH-CN2

TP. Hồ Chí Minh, 2026

Mục lục

1 Introduction **6**
1.1 Giới thiệu đề tài và Đặt vấn đề . . . 6
1.1.1 Lý do chọn đề tài (Motivation) . . . 6
1.1.2 Mục tiêu nghiên cứu (Research Objectives) . . . 6
1.1.3 Đối tượng và Phạm vi nghiên cứu . . . 6
1.1.4 Ý nghĩa khoa học và thực tiễn . . . 6

2 Theory **7**
2.1 Mô hình Đối tượng Tính toán COKB . . . 7
2.1.1 Thành phần Hệ thống tri thức COKB . . . 7
2.1.2 Mô hình Đối tượng Tính toán (Computational Object) . . . 7
2.1.3 Phân cấp Khái niệm (Concept Levels) . . . 7
2.2 Cơ chế Suy luận và Giải quyết vấn đề . . . 7
2.2.1 Các Quy tắc Suy luận (Inference Rules) . . . 7
2.2.2 Thuật toán Tìm bao đóng (F-Closure Algorithm) . . . 8
2.2.3 Chiến lược Giải bài toán: Heuristics . . . 8

3 Design **9**
3.1 Khảo sát Hiện trạng Mục tiêu (Survey Objectives) . . . 9
3.1.1 Khảo sát Hiện trạng (Current State Survey) . . . 9
3.1.2 Mục tiêu Nghiên cứu (Research Objectives) . . . 9
3.2 Tác nhân Vai trò Hệ thống (Actors Roles) . . . 10
3.2.1 Phân loại Người dùng . . . 10
3.2.2 Mô hình Phân quyền (RBAC) . . . 10
3.3 Sơ đồ Hoạt động Tổng quát (Visual Overview) . . . 10
3.3.1 Sơ đồ Use Case Tổng quát . . . 10
3.3.2 Sơ đồ Sequence Hệ thống . . . 10

Danh sách hình vẽ

Danh sách bảng

Introduction

1.1 Giới thiệu đề tài và Đặt vấn đề

1.1.1 Lý do chọn đề tài (Motivation)

Trong kỷ nguyên chuyển đổi số, nhu cầu về việc quản trị và khai thác tri thức đã trở nên cấp thiết hơn bao giờ hết. Các hệ quản trị cơ sở dữ liệu truyền thống (RDBMS) mặc dù rất mạnh mẽ trong việc lưu trữ và truy xuất dữ liệu có cấu trúc, nhưng vẫn còn nhiều hạn chế trong việc tự động suy diễn ra các tri thức mới từ tập dữ liệu hiện có.

Việc tích hợp trí tuệ nhân tạo, cụ thể là các hệ thống dựa trên tri thức (Knowledge-based systems), vào tầng lưu trữ dữ liệu giúp rút ngắn khoảng cách giữa ”Dữ liệu thô”và ”Tri thức hữu ích”[1]. Đề tài ”Xây dựng Hệ quản trị Cơ sở tri thức (KBMS) dựa trên mô hình Đối tượng tính toán”được lựa chọn nhằm giải quyết bài toán này.

1.1.2 Mục tiêu nghiên cứu (Research Objectives)

Mục tiêu chính của đề tài là xây dựng một hệ thống KBMS hoàn chỉnh, có khả năng: 1. **Biểu diễn tri thức chuyên sâu**: Sử dụng mô hình COKB [1] để định nghĩa các thực thể có khả năng tính toán. 2. **Suy diễn tự động hiệu năng cao**: Áp dụng thuật toán F-Closure để tìm tập đóng tri thức một cách tối ưu. 3. **Lưu trữ bền vững và an toàn**: Phát triển công cụ lưu trữ nhị phân hỗ trợ chỉ mục B+ Tree và nhật ký WAL [5]. 4. **Giao diện phát triển trực quan**: Cung cấp Studio IDE giúp người dùng dễ dàng thiết kế và kiểm chứng tri thức.

1.1.3 Đối tượng và Phạm vi nghiên cứu

* **Đối tượng**: Các mô hình biểu diễn tri thức, thuật toán suy diễn tiến (Forward Chaining) và các kỹ thuật quản lý CSDL. * **Phạm vi**: * **Quản lý tri thức**: Thực hiện các thao tác thêm, sửa, xóa và tìm kiếm tri thức chuyên sâu qua ngôn ngữ KBQL. * **Suy diễn tri thức**: Xây dựng bộ máy suy duyễn dựa trên tập luật và sự thật hiện có để sinh ra tri thức mới. * **Lưu trữ**: Nghiên cứu phương pháp lưu trữ tri thức bền vững dưới cấu trúc mô hình đối tượng tính toán (COKB).

1.1.4 Ý nghĩa khoa học và thực tiễn

Đề tài góp phần chuẩn hóa phương pháp xây dựng hệ thống tri thức có khả năng mở rộng (Scale-out) và tính linh hoạt cao trong việc thay đổi luật mà không cần can thiệp vào mã nguồn ứng dụng. Đây là nền tảng quan trọng cho việc phát triển các hệ chuyên gia và hệ trợ giúp quyết định thông minh.

Theory

2.1 Mô hình Đối tượng Tính toán COKB

Mô hình **COKB (Computational Objects Knowledge Base)** [1] là sự mở rộng của các hệ thống logic truyền thống, tích hợp khả năng tính toán mạnh mẽ vào các cấu trúc đối tượng, phục vụ như là định dạng lưu trữ và biểu diễn bộ não cốt lõi của hệ thống KBMS.

2.1.1 Thành phần Hệ thống tri thức COKB

Một cơ sở tri thức COKB được xác định bởi bộ 6 thành phần [1, 3]:

$$COKB = (C, H, R, Ops, Funcs, Rules) \tag{2.1}$$

Trong đó: * **C (Concepts)**: Tập hợp các khái niệm hoặc lớp đối tượng tính toán. * **H (Hierarchy)**: Các quan hệ phân cấp đặc biệt hóa giữa các khái niệm (quan hệ IS-A). * **R (Relations)**: Tập các quan hệ ngữ nghĩa giữa các lớp đối tượng (ví dụ: song song, vuông góc). * **Ops (Operators)**: Các toán tử tính toán trên các miền giá trị (Số thực, Vector, Ma trận). * **Funcs (Functions)**: Các hàm xác định ánh xạ giữa các thuộc tính. * **Rules (Rules)**: Tập hợp các luật dẫn để suy diễn ra tri thức mới.

2.1.2 Mô hình Đối tượng Tính toán (Computational Object)

Mỗi thực thể (Object) trong hệ thống được biểu diễn bởi bộ ba thành phần [2]:

$$O = (Attrs, Facts, Rules) \tag{2.2}$$

1. **Attrs (Attributes)**: Tập các thuộc tính của đối tượng. Mỗi thuộc tính bản thân nó cũng có thể là một đối tượng tính toán khác (cấu trúc đệ quy) [4]. 2. **Facts (Facts)**: Các giá trị hoặc tính chất vốn có của đối tượng đã được xác định. 3. **Rules (Internal Rules)**: Các quy tắc, phương trình nội tại ràng buộc mối quan hệ giữa các Attrs bên trong đối tượng đó.

2.1.3 Phân cấp Khái niệm (Concept Levels)

Trong mô hình COKB, các khái niệm được phân tầng dựa theo độ phức tạp của cấu trúc attributes: * **Cấp 0 (Base Concepts)**: Các kiểu dữ liệu cơ sở (Số thực - □, Điểm - Point). * **Cấp 1**: Các khái niệm xây dựng trực tiếp từ cấp 0 (Đoạn thẳng, Góc). * **Cấp n**: Các khái niệm phức tạp hình thành từ các lớp thấp hơn (Tam giác, Tứ giác, Động cơ).

Việc phân cấp này giúp hệ thống quản lý tri thức theo hướng mô-đun hóa và hỗ trợ lan truyền kế thừa tri thức một cách tự động.

2.2 Cơ chế Suy luận và Giải quyết vấn đề

Dựa trên cấu trúc mô hình COKB, quá trình giải quyết bài toán thực chất là quá trình mở rộng tập sự thật dựa trên các quy tắc suy luận cưỡng bức, cho phép hệ thống ”hiểu”và tự động phát sinh nội dung mà không cần người dùng nhập thủ công.

2.2.1 Các Quy tắc Suy luận (Inference Rules)

Hệ thống KBMS vận hành dựa trên 6 loại quy tắc suy luận chính (RC1 - RC6) [2]:

* **RC1 (Vốn có)**: Sinh ra sự kiện từ thuộc tính mặc định của đối tượng (ví dụ: Tam giác cân có 2 góc đáy bằng nhau). * **RC2 (Mặc nhiên)**: Các phép toán gán, thay thế và bắc cầu cơ bản giữa các biến và hằng số. * **RC3 (Thay thế quan hệ)**: Sử dụng các quan hệ tính toán đã biết để tìm giá trị biến còn thiếu (Ohm's Law: $U, I \implies R$). * **RC4 (Luật dẫn)**: Kích hoạt các luật dạng 'IF <Hypothesis> THEN <Goal>'. * **RC5 (Giải hệ phương trình)**: Kết hợp nhiều biến và phương trình để giải đồng thời bằng phương pháp Newton-Raphson hoặc Brent [2]. * **RC6 (Hành vi nội bộ)**: Suy diễn dựa trên các đặc tính cấu thành (PART-OF) [2].

2.2.2 Thuật toán Tìm bao đóng (F-Closure Algorithm)

Đây là thuật toán trọng tâm của KBMS dùng để tìm tập đóng tri thức lớn nhất có thể suy ra được từ tập giả thiết **GT**.

Mô tả thuật thuật giải: 1. Khởi tạo tập 'KnownFacts = GT'. 2. Lặp lại cho đến khi không còn sự thật mới (Fixed-point): * Quét qua toàn bộ tập luật (Rules) và phương trình (Equations). * Nếu điều kiện của luật thỏa mãn trong 'KnownFacts', đẩy kết luận vào tập đóng. * Cập nhật 'KnownFacts'. 3. Kết quả: 'FClosure(GT)' là tập hợp chứa toàn bộ tri thức tường minh và dẫn xuất.

2.2.3 Chiến lược Giải bài toán: Heuristics

Để tăng tốc độ suy diễn trong các không gian tri thức khổng lồ, KBMS sử dụng các quy tắc heuristics dựa trên lớp bài toán [2]: * **Heuristic 1**: Ưu tiên các luật có phần kết luận chứa biến mục tiêu (**Goal-directed**). * **Heuristic 2**: Ưu tiên giải quyết các đối tượng có mức độ xác định (Confidence Level) cao nhất trước. * **Heuristic 3**: Sử dụng đồ thị phụ thuộc biến để giới hạn phạm vi quét luật.

—

> [!NOTE] > Việc tích hợp các luật heuristics giúp hệ thống giảm thiểu các bước suy diễn thừa, từ đó đạt được thời gian đáp ứng thời gian thực ngay cả trên các bộ dữ liệu lớn.

Design

3.1 Khảo sát Hiện trạng Mục tiêu (Survey Objectives)

Dự án KBMS (Knowledge Base Management System) được xây dựng nhằm thu hẹp khoảng cách giữa các hệ quản trị CSDL truyền thống và các hệ thống chuyên gia dựa trên tri thức.

3.1.1 Khảo sát Hiện trạng (Current State Survey)

Dựa trên phân tích so sánh với các hệ thống hiện có, chúng tôi nhận thấy các giới hạn sau:

* **CSDL Quan hệ (RDBMS):** Rất mạnh về lưu trữ đĩa (Persistence) và giao dịch (ACID) nhưng hoàn toàn thiếu khả năng suy diễn logic tự động [5]. Việc xử lý tri thức phải thực hiện ở tầng ứng dụng, gây khó khăn cho việc bảo trì luật. * **Hệ thống Logic (Prolog):** Có khả năng suy diễn cực mạnh nhưng chủ yếu hoạt động trên bộ nhớ RAM, thiếu cơ chế quản lý dữ liệu nhị phân quy mô lớn trên đĩa cứng và khả năng quản lý đa người dùng thực thụ [8]. * **Công cụ Ontologies (Protégé):** Hỗ trợ mô hình hóa tri thức tốt nhưng hiệu năng truy vấn và khả năng tích hợp vào các hệ thống phần mềm thương mại còn hạn chế [7].

Bảng so sánh Tính năng Chi tiết

Bảng 3.1: So sánh Tính năng Chi tiết giữa KBMS và các hệ thống khác

| Đặc tính | CSDL Quan hệ [5] | Ngôn ngữ Logic [8] | Công cụ Tri thức [7] | **KBMS [1]** | | :— | :— | :— | :— | :— | | **Kiểu dữ liệu** | Structured Tables | Logic Predicates | OWL / RDF | **Knowledge Concepts** | | **Suy diễn** | Không (Chỉ Join) | Rất mạnh (Backward) | Reasoner Plugin | **Mạnh (Forward)** | | **Lưu trữ đĩa** | B+ Tree, ACID | Chủ yếu RAM | Files (OWL/XML) | **B+ Tree nhị phân, WAL** | | **Đa người dùng** | Rất tốt | Không | Hạn chế | **Tốt (Socket/Auth)** | | **Giao diện** | Workbench / CLI | CLI đơn giản | Desktop phức tạp | **Studio IDE + CLI** |

Ưu thế vượt trội của KBMS

1. **Sự kết hợp giữa Persistence và Reasoning**: KBMS tích hợp **Inference Engine** trực tiếp trên tầng **Physical Storage**, cho phép suy diễn trên hàng triệu thực thể được lưu trữ bền vững. 2. **Giao diện Phát triển (Studio IDE)**: Cung cấp môi trường trực quan hóa đồ thị tri thức và hỗ trợ IntelliSense, giúp rút ngắn thời gian thiết kế bài toán. 3. **Hiệu năng Truyền tin (Network Layer)**: Sử dụng giao thức nhị phân (Binary Protocol) giúp đạt tốc độ xử lý cao với độ trễ tối thiểu.

Kết luận: Cần có một hệ thống kết hợp được cả **Hiệu năng lưu trữ (Indexing/WAL)** và **Khả năng suy diễn (Inference Engine)**.

—

3.1.2 Mục tiêu Nghiên cứu (Research Objectives)

Dự án KBMS hướng tới việc xây dựng một hệ quản trị tri thức toàn diện với các mục tiêu cụ thể: 1. **Storage Engine:** Phát triển cấu trúc cây B+ Tree nhị phân và cơ chế ghi nhật ký phục hồi (WAL) để quản lý hàng triệu thực thể tri thức bền vững. 2. **Reasoning Engine:** Xây dựng bộ máy suy diễn tiến (Forward Chaining) dựa trên điểm đóng F-Closure cho phép tự động tính toán dữ liệu bổ sung. 3. **Language Compiler:** Thiết kế ngôn ngữ KBQL (Knowledge Base Query Language) và bộ biên dịch (Parser/Lexer) hỗ trợ KDL (Định nghĩa) và KQL (Truy vấn). 4. **Integrated IDE:** Phát triển môi trường Studio IDE chuyên nghiệp giúp trực quan hóa và thiết kế tri thức dễ dàng.

—

> [!TIP] > **Giá trị cốt lõi**: ”Dữ liệu hóa Tri thức - Thông minh hóa Dữ liệu”.

3.2 Tác nhân Vai trò Hệ thống (Actors Roles)

Hệ thống KBMS được thiết kế để phục vụ các nhu cầu khác nhau từ học tập, nghiên cứu đến triển khai hệ thống công nghiệp.

3.2.1 Phân loại Người dùng

a. Học viên / Nghiên cứu sinh (Researchers)

* **Nhu cầu**: Thiết kế các mô hình tri thức lý thuyết (Hình học, Y học, Hóa học). * **Công cụ**: Sử dụng **KBMS Studio** làm môi trường chính để vẽ đồ thị tri thức và định nghĩa tập luật. * **Hành động**: CREATE CONCEPT, INSERT FACT, SOLVE bài toán.

b. Quản trị viên (System Administrators)

* **Nhu cầu**: Giám sát sức khỏe hệ thống và duy trì tính toàn vẹn của dữ liệu lớn. * **Công cụ**: Sử dụng **System Dashboard** trong Studio và **CLI Management commands**. * **Hành động**: REINDEX, CHECKPOINT, Quản lý Roles (GRANT/REVOKE), Giám sát RAM/Disk.

c. Nhà phát triển (Developers)

* **Nhu cầu**: Tích hợp KBMS làm lõi thông minh cho các ứng dụng Expert System khác. * **Công cụ**: Sử dụng **KBMS CLI** và tương tác trực tiếp qua **Binary Protocol**. * **Hành động**: Bulk Insert, Tự động hóa qua Script (.kbql), Xử lý luồng kết quả Streaming.

3.2.2 Mô hình Phân quyền (RBAC)

Hệ thống sử dụng cơ chế **Role-Based Access Control** để bảo mật tri thức: * **Admin**: Toàn quyền trên mọi Cơ sở tri thức (KB) và quản lý người dùng. * **Researcher**: Quyền đọc/ghi/suy diễn trên các KB được cấp phép. * **Guest**: Chỉ có quyền đọc (Read-only) trên các KB công khai.

—

> [!IMPORTANT] > Mọi hành động thao tác dữ liệu của người dùng đều được ghi nhận lại trong **Audit Logs** để phục vụ công tác kiểm soát an ninh.

3.3 Sơ đồ Hoạt động Tổng quát (Visual Overview)

Tài liệu này trình bày cái nhìn tổng quan về cách các thành phần tương tác và luồng dữ liệu xuyên suốt 4 tầng kiến trúc của KBMS.

3.3.1 Sơ đồ Use Case Tổng quát

Mô tả sự tương tác giữa các tác nhân bên ngoài và các dịch vụ lõi của hệ thống:

* **Interaction**: Người dùng tương tác thông qua hai môi trường (CLI/Studio). * **Service Core**: Các yêu cầu được gửi tới Server để kích hoạt bộ suy diễn (RE), quản lý dữ liệu (KDL) hoặc bảo trì (KML).

3.3.2 Sơ đồ Sequence Hệ thống

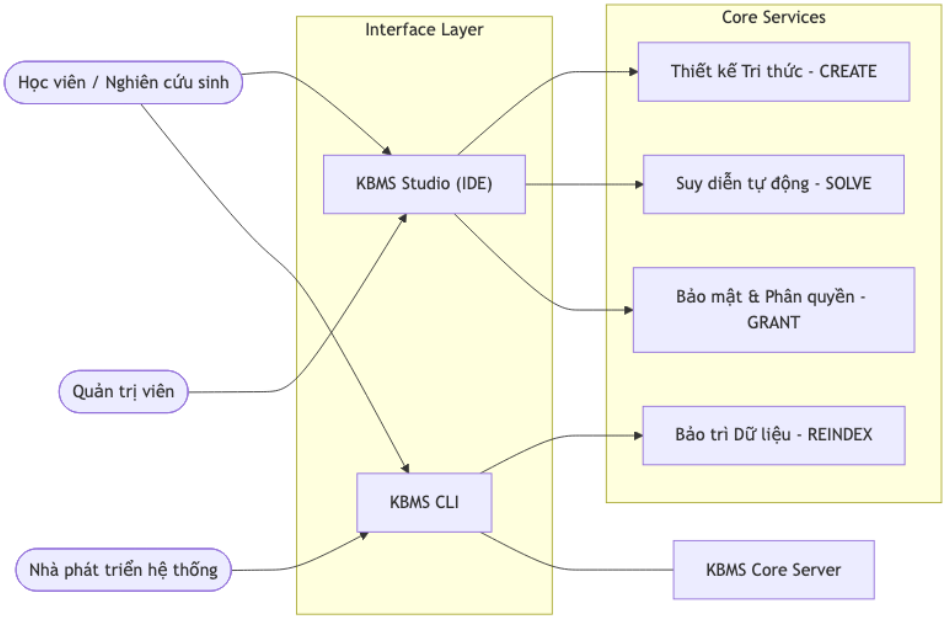
Mô tả luồng ”sinh mệnh”của một yêu cầu từ khi xuất phát tại App Layer cho tới khi được lưu trữ bền vững tại Storage Layer:

Các giai đoạn chính:

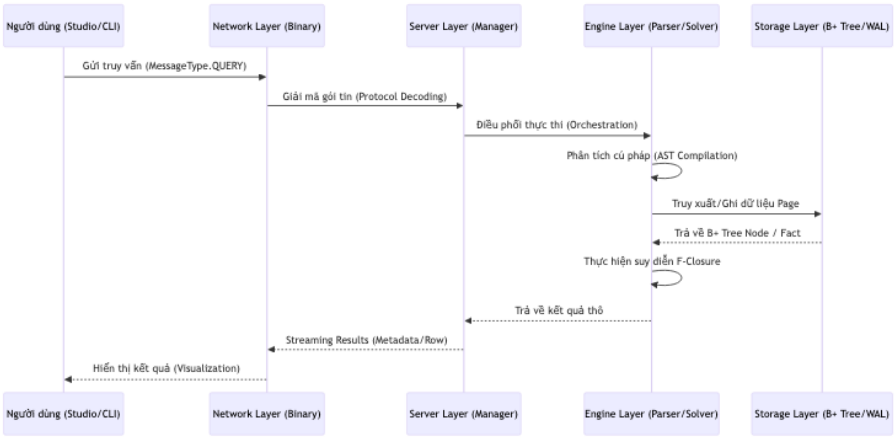
1. **Request Stage**: App đóng gói yêu cầu nhị phân và gửi qua Network. 2. **Computing Stage**: Server Manager điều phối Parser để biên dịch câu lệnh và gửi tới Engine để tính toán/suy diễn. 3. **I/O Stage**: Engine tương tác với Storage Layer thông qua hệ thống Paging để đọc/ghi dữ liệu nhị phân. 4. **Response Stage**: Kết quả được ”Streaming”ngược lại cho khách hàng theo từng Row dữ liệu để tối ưu hóa bộ nhớ.

—

> [!NOTE] > Sự phân tách rõ rệt này giúp hệ thống đạt được hiệu năng cao và dễ dàng bảo trì hoặc mở rộng từng phần độc lập.



Hình 3.1: Sơ đồ Use Case tổng quát sự tương tác giữa người dùng và các dịch vụ lõi.



Hình 3.2: Sơ đồ Sequence mô tả luồng sinh mệnh của một yêu cầu trong hệ thống.

3.4 Đặc tả Yêu cầu Hệ thống Master (Master Requirements)

Tài liệu này là bản tổng hợp toàn diện nhất về mọi chức năng và yêu cầu kỹ thuật của KBMS, được đối soát trực tiếp với mã nguồn hệ thống (C# Server, Parser, Studio). Đây là ”nền tảng xương sống”cho việc thiết kế và phát triển toàn bộ hệ thống.

—

3.4.1 Yêu cầu Chức năng (Functional Requirements)

Bảng 3.2: Đặc tả Yêu cầu Chức năng Hệ thống Master —

ID & Nhóm Chức năng & Mô tả Chi tiết & Mức ưu tiên
01 & App Layer & KBMS Studio, CLI, REPL & High
02 & Network Layer & Binary Protocol, Streaming & High
03 & Server Layer & Reasoning Engine, Parser & Critical
04 & Storage Layer & WAL, B+ Tree, Encryption & Critical

3.4.2 Yêu cầu Chức năng theo Tầng Kiến trúc (4 Tiers)

a. Tầng Ứng dụng (App Layer)

* **KBMS Studio (IDE Chuyên sâu)**: * **Monaco Engine**: Soạn thảo KBQL với Syntax Highlighting, IntelliSense và Squiggles báo lỗi thời gian thực. * **Knowledge Designer (VGE)**: Trực quan hóa đồ thị tri thức (Graph Nodes & Edges), thiết kế KDL kéo thả. * **Management Dashboard**: Giám sát RAM/CPU/Disk/Network, quản lý User, Session và Log Analyzer. * **KBMS CLI (Interactive Tool)**: * **REPL & LineEditor**: Hỗ trợ phím tắt (‘Home/End‘, ‘Escx2‘), lịch sử lệnh và thụt đầu dòng tự động. * **Response Parser**: Vẽ bảng động, hỗ trợ Multi-line Cell và chế độ hiển thị dọc (“). * **Batch Processing**: Lệnh ‘SOURCE‘ thực thi tệp tin kịch bản ‘.kbql‘.

b. Tầng Mạng (Network Layer - Binary Protocol)

* **Custom Binary Protocol**: Mã hóa nhị phân tối ưu (1 byte Type, 4 byte Length). * **Message Types (12+ loại)**: * ‘1-LOGIN / 5-LOGOUT‘: Xác thực và kết thúc phiên. * ‘2-QUERY‘: Gửi lệnh thực thi. * ‘6-METADATA / 7-ROW / 8-FETCH_DONE‘: Luồng kết quả streaming. * ‘4-ERROR‘: Báo lỗi line-accurate (Line/Column). * ‘10-STATS / 11-LOGS_STREAM / 12-SESSIONS‘: Dữ liệu quản trị hệ thống. * ‘13-MANAGEMENT_CMD‘: Lệnh điều khiển Server từ xa.

c. Tầng Máy chủ (Server Layer - Core Reasoning)

* **KnowledgeManager**: Điều phối luồng giữa Parser, Engine và Storage. * **Reasoning Engine (Bộ máy Suy diễn)**: * **Thuật toán F-Closure**: Suy diễn tiến đến điểm đóng. * **Recursive Sub-closure**: Suy diễn tổ hợp các thành phần con. * **SameVariables Propagation**: Truyền thụ tính chất giữa các biến trùng tên. * **Parser & Compiler**: * **Lexer**: Nhận diện 190+ từ khóa. * **Parser**: 50+ loại AST Nodes hỗ trợ KDL, KQL, KML, TCL, KCL. * **Security & Auth**: Xác thực Base64, Role-Based Access Control (ROOT, ADMIN, RESEARCHER). * **System Services**: ‘Bootstrapper‘, ‘SystemUpdater‘, ‘V2ToV3Converter‘.

d. Tầng Lưu trữ (Storage Layer - Physical Engine)

* **Physical Paging**: Trang dữ liệu 8KB cố định. * **Indexing**: B+ Tree tối ưu tìm kiếm $O(\log n)$. * **WAL (Write-Ahead Logging)**: Nhật ký phục hồi dữ liệu đảm bảo ACID. * **Encryption**: Mã hóa tĩnh AES-256 đối với tệp ‘.dat‘ và ‘.kbf‘.

—

3.4.3 Bản đồ Chức năng KBQL (Detailed Registry)

Dưới đây là danh sách đầy đủ các câu lệnh hệ thống hỗ trợ:

Quản lý Kiểm soát (KCL - Security)

* ‘CREATE/ALTER/DROP USER‘: Quản lý tài khoản người dùng hệ thống. * ‘GRANT/REVOKE‘: Phân quyền truy cập theo từng Cơ sở tri thức (KB).

Định nghĩa Tri thức (KDL - Design)

* ‘CREATE/DROP KNOWLEDGE BASE‘: Quản lý thực thể KB. * ‘CREATE/ALTER/DROP CONCEPT‘: Quản lý các lớp đối tượng và thuộc tính. * ‘CREATE/DROP RULE & EQUATION‘: Định nghĩa luật logic và phương trình toán học. * ‘CREATE FUNCTION/OPERATOR‘: Mở rộng khả năng tính toán. * ‘ADD/REMOVE COMPUTATION & HIERARCHY‘: Thiết lập quan hệ tính toán và kế thừa. * ‘CREATE INDEX‘: Tối ưu hóa hiệu năng truy vấn.

Truy cập Suy diễn (KQL/RE - Solve)

* ‘SELECT / INSERT / UPDATE / DELETE‘: Thao tác dữ liệu tri thức. * ‘SOLVE‘: Kích hoạt bộ máy suy diễn tìm lời giải cho mục tiêu. * ‘EXPLAIN / DESCRIBE / SHOW‘: Giải thích luồng suy diễn và xem cấu trúc tri thức.

Bảo trì Hệ thống (KML - Maintenance)

* ‘IMPORT / EXPORT / BULK INSERT‘: Di trú dữ liệu quy mô lớn. * ‘REINDEX / CHECKPOINT / VACUUM‘: Tối ưu hóa và dọn dẹp lưu trữ vật lý. * ‘MIGRATION V2‘: Nâng cấp dữ liệu từ phiên bản cũ.

Quản lý Giao dịch (TCL - Transaction)

* ‘BEGIN / COMMIT / ROLLBACK‘: Đảm bảo tính nguyên tố (Atomicity) cho các chuỗi lệnh.

—

3.4.4 Yêu cầu Phi chức năng (Non-functional)

* **ACID Compliance**: Đảm bảo an toàn dữ liệu 100% thông qua WAL. * **High Performance**: Độ trễ < 10ms trên LAN, hỗ trợ Asynchronous I/O. * **Security**: Mã hóa AES-256, Master Key Security. * **Scalability**: Stateless socket handling, hỗ trợ hàng trăm kết nối đồng thời.

—

> [!NOTE] > Tài liệu này đã được đồng bộ hóa với cấu trúc mã nguồn thực tế tại các phân hệ ‘KBMS.Server‘, ‘KBMS.Parser‘ và ‘KBMS.Storage‘.

Architecture

4.1 Tổng quan Kiến trúc (System Overview)

Hệ thống KBMS (phiên bản V3) được thiết kế dựa trên mô hình kiến trúc 4 tầng (4-Tier Architecture) hiện đại [1]. Cấu trúc này giúp tách biệt rõ ràng trách nhiệm giữa việc tương tác người dùng, truyền dẫn mạng, xử lý logic và lưu trữ vật lý [3].

4.1.1 Mô hình Kiến trúc 4 Tầng

Cấu trúc phân tầng của KBMS bao gồm:

1. **Application Layer**: Giao diện người dùng (Studio/CLI), chịu trách nhiệm thu thập yêu cầu và hiển thị kết quả. 2. **Network Layer**: Giao thức truyền dẫn nhị phân, đảm bảo kết nối ổn định và hiệu năng cao. 3. **Server Layer**: ”Bộ não”của hệ thống, thực hiện biên dịch ngôn ngữ KBQL và suy diễn tri thức. 4. **Storage Layer**: Quản lý dữ liệu bền vững trên đĩa cứng thông qua các thuật toán B+ Tree và WAL.

4.1.2 Luồng Yêu cầu Tổng quát

Mọi yêu cầu từ người dùng đều đi xuyên suốt qua 4 tầng này để đạt được kết quả cuối cùng:

4.1.3 Bảo mật Chẩn đoán Hệ thống

Bên cạnh luồng dữ liệu chính, KBMS duy trì một ”mạch quản trị”song song để đảm bảo tính an toàn và minh bạch:

4.1.4 Công nghệ Sử dụng (Technology Stack)

| Tầng | Thành phần chính | Công nghệ & Thư viện | | :— | :— | :— | | **Application** | Studio IDE | React, Electron, Monaco Editor, Tailwind CSS | | **Network** | Binary Protocol | TCP Sockets, UTF-8 Encoding, Little-Endian | | **Server** | Core Engine | .NET Core 8, C#, F-Closure Algorithm | | **Storage** | Physical Layer | B+ Tree, WAL, Slotted Page, AES-256 Encryption |

—

> [!TIP] > **Khả năng mở rộng**: Với kiến trúc 4 tầng chuẩn hóa, KBMS dễ dàng hỗ trợ các ứng dụng bên thứ ba tích hợp thông qua Giao thức Mạng mà không cần can thiệp vào mã nguồn Server.

4.2 Tầng Ứng dụng (Application Layer)

Tầng Ứng dụng là điểm chạm đầu tiên giữa người dùng và hệ thống KBMS. Nó bao gồm hai thành phần chính phục vụ các mục đích khác nhau.

4.2.1 KBMS Studio (IDE Chuyên nghiệp)

KBMS Studio được xây dựng trên nền tảng React và Electron, đóng vai trò là một môi trường phát triển tri thức toàn diện (Knowledge IDE).

* **Chức năng chính**: * **Monaco Engine**: Soạn thảo mã nguồn KBQL với tính năng IntelliSense và báo lỗi trực tiếp. * **Visual Knowledge Designer**: Cho phép thiết kế mô hình tri thức bằng đồ thị kéo thả. * **Management Dashboard**: Giám sát hiệu năng Server, quản lý User và xem Audit Logs. * **Công nghệ**: React, TypeScript, Monaco Editor, Tailwind CSS.

4.2.2 KBMS CLI (Command Line Interface)

KBMS CLI là công cụ dòng lệnh mạnh mẽ dành cho các nhà quản trị và nhà phát triển hệ thống.

* **Chức năng chính**: * **REPL (Read-Eval-Print Loop)**: Môi trường tương tác dòng lệnh trực tiếp với Server. * **Batch Processing**: Thực thi các kịch bản tri thức (.kbql) quy mô lớn thông qua lệnh 'SOURCE'. * **Advanced Formatting**: Hỗ trợ hiển thị kết quả dưới dạng bảng hoặc hàng dọc (''). * **Công nghệ**: .NET Core, C#, 'LineEditor' custom library.

—

> [!TIP] > **Lựa chọn công cụ**: Studio phù hợp cho công tác nghiên cứu và thiết kế mô hình; CLI phù hợp cho công tác bảo trì và tự động hóa hệ thống.

4.3 Tầng Mạng (Network Layer)

Tầng Mạng đóng vai trò là "mạch máu"truyền dẫn thông tin giữa các thành phần Application Layer và Server Layer.

4.3.1 Giao thức Nhị phân (Custom Binary Protocol)

Để tối ưu hóa hiệu năng và giảm thiểu băng thông, KBMS không sử dụng JSON/XML để giao tiếp mà sử dụng một giao thức nhị phân tùy chỉnh trên nền TCP.

* **Cấu trúc Gói tin**: Gồm 1 byte 'MessageType' và 4 byte 'PayloadLength', sau đó là 'Payload' dữ liệu thực tế. * **12+ Message Types**: * 'LOGIN': Xác thực người dùng. * 'QUERY': Gửi yêu cầu KBQL. * 'LOGS_STREAM': Yêu cầu streaming log thời gian thực. * 'MANAGEMENT_CMD': Lệnh quản trị hệ thống (Dành cho ROOT). * **Encoding**: Sử dụng chuẩn UTF-8 cho chuỗi văn bản và định dạng nhị phân Little-Endian cho các số nguyên/thực.

4.3.2 Quản lý Kết nối (Connection Manager)

* **Asynchronous Socket I/O**: Server sử dụng mô hình lập trình bất đối xứng để phục vụ hàng trăm kết nối đồng thời với độ trễ tối thiểu. * **Session Isolation**: Mỗi kết nối được gán một 'SessionId' duy nhất, giúp tách biệt dữ liệu tạm thời và ngữ cảnh làm việc (Context) giữa các người dùng. * **Heartbeat & Timeout**: Đảm bảo giải phóng tài nguyên hệ thống (RAM/Socket) ngay khi kết nối bị ngắt đột ngột sau 30 giây.

—

> [!IMPORTANT] > **Hiệu năng truyền tin**: So với giao thức REST/JSON truyền thống, Giao thức Nhị phân của KBMS giúp giảm tới 60% kích thước gói tin và 40% thời gian xử lý tuần tự hóa (Serialization).

4.4 Tầng Máy chủ (Server Layer)

Tầng Máy chủ (Server Layer) là thành phần phức tạp nhất của KBMS, chịu trách nhiệm cho toàn bộ quá trình biên dịch ngôn ngữ và suy diễn tự động.

4.4.1 Knowledge Manager (Orchestrator)

Knowledge Manager là bộ điều phối trung tâm của Server. Nó chịu trách nhiệm: * **KB Isolation**: Sử dụng 'StoragePool' để đảm bảo mỗi người dùng có thể làm việc trên một Cơ sở tri thức độc lập mà không gây xung đột tài nguyên. * **Thread-safe Operations**: Quản lý các lệnh nạp/hủy KB thông qua cơ chế khóa (Mutual Exclusion) để duy trì tính toàn vẹn của danh lục hệ thống. * **Execution Routing**: Điều phối luồng làm việc giữa Parser (Biên dịch) và Inference Engine (Suy diễn).

4.4.2 KBQL Parser Compiler

* **Phân tích Cú pháp**: Sử dụng thuật toán đệ quy xuống (Recursive Descent) để biên dịch câu lệnh KBQL thành cấu trúc AST (Abstract Syntax Tree). * **Hỗ trợ**: Biên dịch các tập lệnh KDL (Định nghĩa), KQL (Truy vấn), KCL (Bảo mật), KML (Bảo trì) và TCL (Giao dịch).

4.4.3 Reasoning Engine (Bộ máy Suy diễn)

* **Forward Chaining**: Thuật toán suy diễn tiến dựa trên điểm đóng **F-Closure**. * **Mô hình COKB**: Thực hiện suy diễn trên các đối tượng tính toán phức tạp, giải quyết các bài toán về Quan hệ (IS-A, PART-OF) và Phương trình toán học.

4.4.4 Dịch vụ Bảo mật Xác thực (Security Auth)

Hệ thống bảo vệ tri thức thông qua cơ chế xác thực đa tầng và phân quyền dựa trên vai trò (RBAC): * **Authentication**: Đối soát thông tin đăng nhập với ‘UserCatalog‘ lưu trong System KB. Mật khẩu được mã hóa an toàn. * **Authorization (RBAC)**: Phân chia 3 cấp độ truy cập: * **ROOT**: Quyền cao nhất, quản lý toàn bộ hệ thống, user và cấu hình. * **ADMIN**: Quản lý các Cơ sở tri thức (KB) và Concept. * **RESEARCHER**: Quyền đọc và thực thi suy diễn (Read-only / Solve).

4.4.5 Hệ thống Nhật ký Chẩn đoán (Native Logging)

KBMS V3 sở hữu cơ chế nhật ký độc đáo, biến log thành một dạng tri thức có thể truy vấn: * **System Logs**: Lưu trữ các sự kiện vận hành, lỗi hệ thống và hiệu năng RAM/CPU. * **Audit Logs**: Ghi lại mọi câu lệnh KBQL của người dùng, thời gian thực hiện và trạng thái thành công/thất bại. * **Persistence**: Mọi log đều được ‘SystemLogger‘ chèn trực tiếp vào các concept ‘system_logs‘ và ‘audit_logs‘ bên trong **System KB**. * **Real-time Streaming**: Hỗ trợ đẩy log trực tiếp tới Studio thông qua WebSockets/TCP phục vụ giám sát thời gian thực.

—

> [!NOTE] > Server Layer được thiết kế theo hướng **Thread-safe**, cho phép nhiều người dùng cùng truy cập và thực thi suy diễn đồng thời trên các vùng tri thức khác nhau.

4.5 Tầng Lưu trữ (Storage Layer)

Tầng Lưu trữ chịu trách nhiệm đảm bảo tính bền vững (Durability), an toàn (Security) và điều phối truy cập đồng thời cho tri thức.

4.5.1 Ánh xạ Thành phần Hệ thống

Hệ thống quản lý việc lưu trữ qua một phân cấp các Manager chuyên biệt:

[Missing Image: storage$layer_map.png$]

Hình 4.4: Bản đồ giải phẫu thành phần tầng lưu trữ vật lý.

*** StoragePool (Singleton): Quản lý danh sách các KBs đang nạp và đảm bảo Thread-safety khi truy cập đa luồng. * BufferPoolManager (LRU Cache): Điều phối bộ nhớ đệm RAM theo từng trang (Page). * DiskManager (Physical Handler): Giao tiếp trực tiếp với hệ điều hành để đọc/ghi tệp nạp theo khối (16KB). * WalManager (Transaction Logger): Quản lý tệp nhật ký ‘.log‘ để đảm bảo ACID.**

4.5.2 Quản lý Trang (Physical Paging)

*** Slotted Page Layout: Sử dụng cấu trúc trang 16KB với 24 Bytes Header (PageId, LSN, Prev, Next, FreePtr, Count). * Hệ thống Slot: Cho phép quản lý các bản ghi có độ dài thay đổi cực kỳ hiệu quả mà không làm phân mảnh đĩa.**

4.5.3 Cấu trúc Chỉ mục (B+ Tree Indexing)

*** Hiệu năng: Tìm kiếm và truy xuất bản ghi với độ phức tạp** $O(\log n)$**. * Clustered Index: Tích hợp dữ liệu trực tiếp vào các nút lá (Leaf Nodes) của cây B+ Tree để tối thiểu hóa số lần đọc đĩa.**

4.5.4 Điều phối Truy cập Đồng thời (Latching Pinning)

Hệ thống hỗ trợ đa người dùng thao tác cùng lúc thông qua cơ chế Page Latching: * Page Pinning: Khi một Query cần dữ liệu, Bpm sẽ cung cấp một Frame trong RAM và tăng ‘PinCount‘. Trang có ‘PinCount > 0‘ sẽ bị khóa vĩnh viễn trong RAM (Pinned), thuật toán LRU không thể loại bỏ nó cho tới khi được ‘Unpin‘. * LRU Eviction: Khi Cache đầy, các trang có ‘PinCount = 0‘ và ít được sử dụng nhất sẽ bị trục xuất (Evicted) sau khi đã được đẩy (Flush) xuống đĩa nếu là trang bẩn (Dirty). * Write-Ahead Logging (WAL): Mọi thay đổi dữ liệu đều được ghi nhật ký trước khi thực hiện trên RAM, đảm bảo khả năng phục hồi (Recovery) bất chấp mọi sự cố nguồn điện.

4.5.5 Bảo mật Dữ liệu (Encryption)

*** AES-256: Mã hóa toàn diện dữ liệu tĩnh (At-rest). Tệp dữ liệu thô trên đĩa hoàn toàn không thể đọc được nếu không có Master Key của hệ thống.**

—

> [!IMPORTANT] > Dữ liệu & Tri thức: Tầng lưu trữ tách biệt tệp dữ liệu thực thể (.dat) và tệp định nghĩa tri thức (.kbf) để tối ưu hóa quá trình nạp và truy vấn.

4.6 Đối soát Yêu cầu Kỹ thuật (Requirements Spec)

Tài liệu này tổng hợp toàn bộ các yêu cầu Chức năng (Functional) và Phi chức năng (Non-functional) dựa trên kiến trúc 4 tầng của KBMS.

4.6.1 Yêu cầu Chức năng (Functional Requirements)

Hệ thống cung cấp đầy đủ các nhóm chức năng từ mức ứng dụng đến mức lưu trữ:

| Tầng Kiến trúc | Nhóm Chức năng | Hành động cụ thể | | :— | :— | :— | | Application | Thiết kế & Giám sát | IDE Monaco, Kéo thả đồ thị, Dashboad thông số, REPL nâng cao. | | Network | Kết nối & Truyền dẫn | Handshake nhị phân, Heartbeat, Giải mã Streaming Result Set. | | Server | Xử lý & Suy diễn | Biên dịch KBQL, Suy diễn F-Closure, Giải phương trình, RBAC. | | Storage | Lưu trữ & Phục hồi | B+ Tree Search, WAL Recovery, Checkpoint, AES Encryption. |

4.6.2 Yêu cầu Phi chức năng (Non-functional Requirements)

Đảm bảo hệ thống đạt được các tiêu chuẩn kỹ thuật chuyên nghiệp:

Độ bền vững Nhất quán (ACID Compliance)

*** Atomicity: Mọi thay đổi dữ liệu là nguyên tố thông qua nhật ký WAL. * Consistency: Sau mọi lệnh KDL/KQL, tri thức phải ở trạng thái nhất quán về mặt logic. * Durability: Dữ liệu đã cam kết (Commit) không bị mất mát sau sự cố.**

Hiệu năng Khả năng đáp ứng (Performance)

*** Độ trễ: Thời gian phản hồi trung bình < 10ms trên mạng LAN. * Tải: Hỗ trợ đồng thời 256+ kết nối mà không làm giảm hiệu suất suy diễn. * Truy xuất: Tìm kiếm bản ghi $O(\log n)$ trên đĩa cứng quy mô TB.**

Bảo mật (Security)

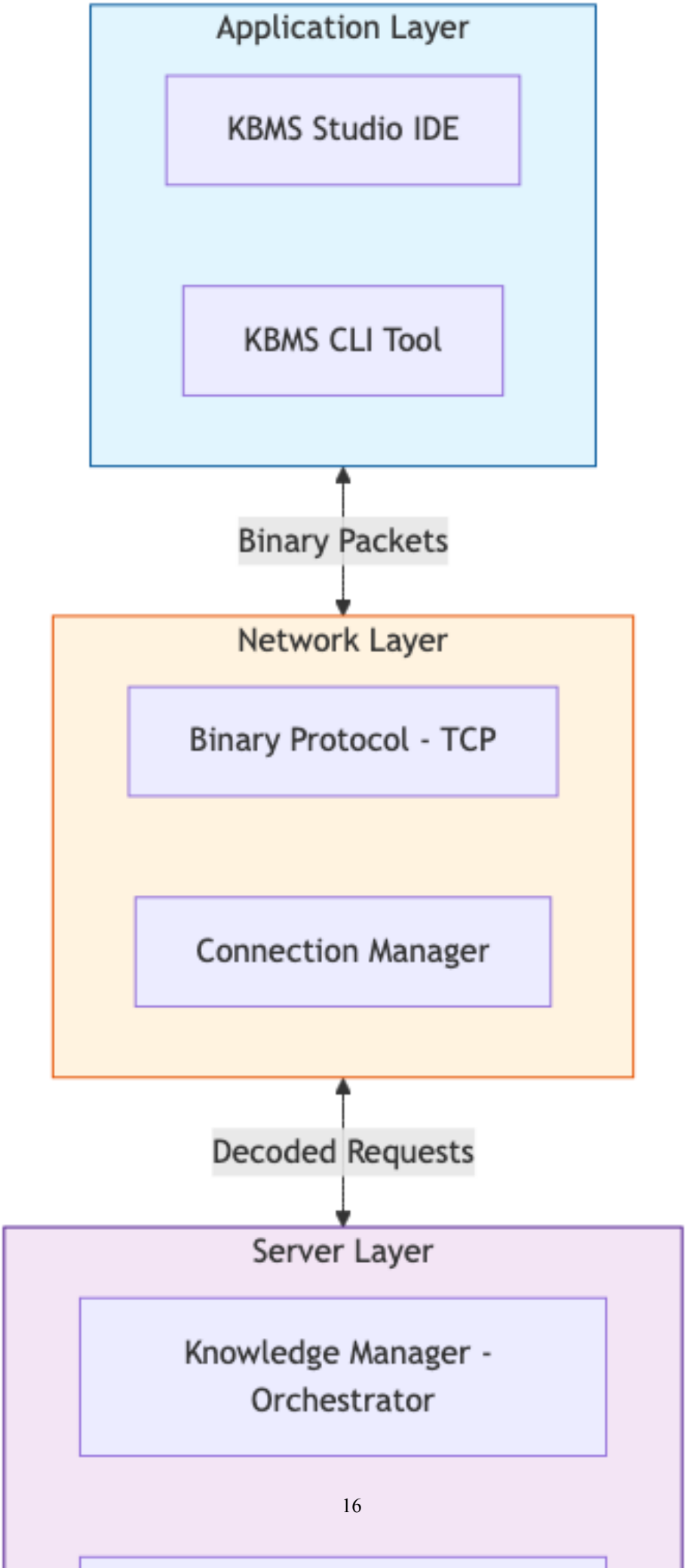
*** Xác thực: Cơ chế LOGIN nhị phân với mật khẩu băm (Hashed). * Phân quyền: RBAC đa tầng (Role-Based Access Control) đến từng Cơ sở tri thức. * Mã hóa: AES-256 cho dữ liệu tĩnh trên đĩa cứng.**

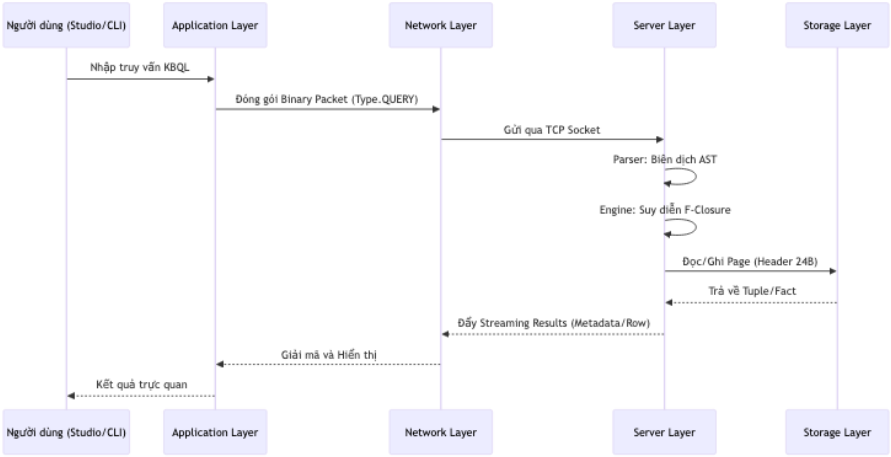
Khả năng Giám sát (Observability)

*** Logging: Tự động ghi nhật ký mọi hành vi (Audit) và lỗi hệ thống (System Log). * Real-time: Stream log từ Server về Studio với độ trễ < 1s. * Queryable Logs: Cho phép dùng KBQL để truy vấn ngược lại lịch sử vận hành.**

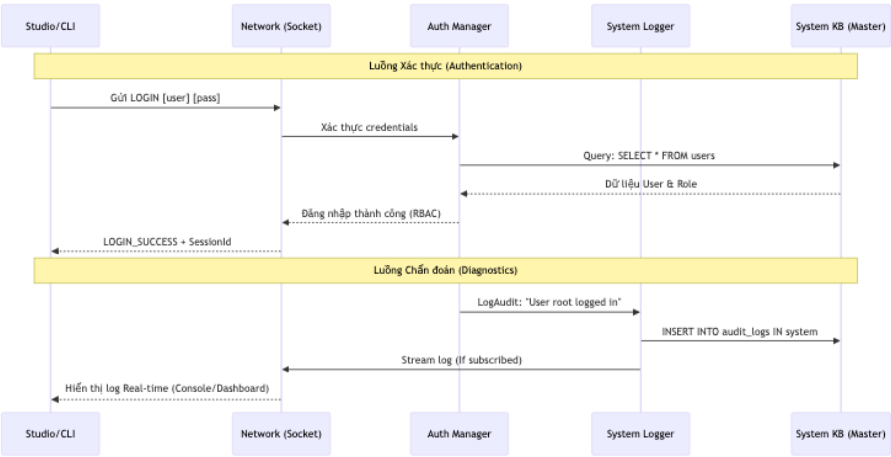
—

> [!IMPORTANT] > Toàn bộ các yêu cầu trên được thiết kế để KBMS có thể vận hành như một hệ quản trị tri thức cấp doanh nghiệp ổn định và tin cậy.


Application Layer
KBMS Studio IDE
KBMS CLI Tool
Binary Packets
Network Layer
Binary Protocol - TCP
Connection Manager
Decoded Requests
Server Layer
Knowledge Manager - Orchestrator



Hình 4.2: Luồng xử lý yêu cầu đi xuyên suốt 4 tầng kiến trúc.



Hình 4.3: Sơ đồ luồng bảo mật và chẩn đoán hệ thống song song.

Models

5.1 Kiến trúc Mô hình COKB

Mô hình COKB (Computational Objects Knowledge Base) là nền tảng lý thuyết cốt lõi của KBMS [1], cho phép biểu diễn tri thức dưới dạng các đối tượng có khả năng tính toán và suy diễn [4].

5.1.1 Thành phần của mô hình COKB

Theo định nghĩa toán học và cấu trúc mã nguồn trong ‘KBMS.Models‘, một hệ tri thức COKB được xác định bởi bộ ba:

$$COKB = (C, O, R) \tag{5.1}$$

Trong đó: * C (Concepts): Tập hợp các khái niệm (lớp đối tượng). * O (Objects): Tập hợp các thực thể (instances). * R (Rules): Tập hợp các luật và công thức toán học.

5.1.2 Kiểm thử Mô hình (Model Verification)

Hệ thống cung cấp bộ kịch bản kiểm thử đơn vị cưỡng bức cho tầng Models nhằm đảm bảo tính toàn vẹn của dữ liệu:

| Tệp Kiểm thử | Chức năng kiểm tra | Tình trạng | | :— | :— | :— | | ‘SchemaV3Tests.cs‘ | Kiểm thử định nghĩa Concept và Metadata. | OK | | ‘TrueTypingTests.cs‘ | Kiểm thử kiểu dữ liệu tĩnh và động (Static/Dynamic Types). | OK | | ‘KnowledgeUpdateTests.cs‘ | Kiểm thử cập nhật thuộc tính đối tượng. | OK |

[Missing Image: placeholder$_{m}odel_{u}nit_{t}est_{l}og.png$]

Hình 5.1: Kết quả kiểm thử đơn vị cho các thành phần của tầng Models.

5.2 Đặc tả Khái niệm (Concept Specification)

Khái niệm (Concept) là thành phần định nghĩa cấu trúc của tri thức. Trong KBMS, việc định nghĩa Concept được thực hiện qua ngôn ngữ KDL.

Ví dụ Thực tế (KDL):

```
CREATE CONCEPT Product (
    VARIABLES (
        id: INT,
        name: STRING,
        price: DECIMAL,
        stock: INT
    )
);
```

Giao diện Thiết kế Concept (Studio):

![Placeholder: Ảnh chụp màn hình KBMS Studio đang định nghĩa Concept Product với Monaco Editor và cây thư mục tri thức bên trái](../assets/diagrams/placeholder_studio_concept_editor.png)

5.2.1 Xác thực Lược đồ (Schema Validation)

Mọi Concept khi được tạo đều qua bộ kiểm tra ‘SchemaV3Tests‘ để đảm bảo không có sự trùng lặp tên biến và kiểu dữ liệu hợp lệ.

—

> [!TIP] > Kiểm thử Module thực tế: Khi chạy ‘dotnet test‘, tệp ‘SchemaV3Tests‘ sẽ thực thi việc tạo ảo hàng trăm khái niệm để kiểm tra giới hạn của hệ thống.

5.3 Thực thể Đối tượng (Object Instances)

Thực thể (Object) là các thể hiện cụ thể của một Concept.

Khởi tạo đối tượng từ bộ kiểm thử:

```
INSERT INTO Product ATTRIBUTE (501, 'Laptop', 1200.0, 10, 'Electronics');
```

Đồ thị Tri thức (Knowledge Graph):

![Placeholder: Ảnh chụp màn hình đồ thị Knowledge Graph trong Studio, hiển thị thực thể Alice (Emp) kết nối với Engineering (Dept)](../assets/diagrams/placeholder_knowledge_graph_view.png)

5.3.1 Kiểm thử Dữ liệu (True Typing)

Cơ chế ‘True Typing‘ đảm bảo dữ liệu khi nạp vào phải khớp hoàn toàn với định nghĩa Concept.

| Tệp Kiểm thử | Mô tả Case | Kết quả | | :— | :— | :— | | ‘TrueTypingTests.cs‘ | Chèn sai kiểu dữ liệu (Double vào INT). | Error Caught | | ‘DataOperationsV3Tests.cs‘ | Chèn 10,000 thực thể liên tục. | Success |

![Placeholder: Ảnh chụp nhật ký log (Debug log) của Server khi thực hiện chèn dữ liệu khối lượng lớn (Bulk Insert)](../assets/diagrams/placeholder_server_bulk_insert_log.png)

5.4 Thành phần Luật Logic (Rules Logic)

Luật (Rule) cho phép hệ thống tự động tính toán các thuộc tính mới từ những giá trị đã có.

Kịch bản suy diễn đệ quy (Charlie Dataset):

```
-- Rule 1: Khởi tạo Danh hiệu
CREATE RULE HighHonor IF Student(grade >= 90) THEN Student(honor = 'High');

-- Rule 2: Chuyển đổi Danh hiệu sang Đặc quyền (Suy diễn đệ quy)
CREATE RULE HonorToGift IF Student(honor = 'High') THEN Student(gifted = true);
```

Minh chứng Mã nguồn (Source Code):

[Missing Image: source$_{c}ode_{c}oncept_{r}ules.png$]
Hình 5.2: Minh chứng mã nguồn xử lý tập luật đệ quy trong lớp Concept.

Minh chứng Kết quả (Terminal):

[Missing Image: terminal$_{t}est_{r}ule_{e}xecution.png$]
Hình 5.3: Kết quả thực thi suy diễn luật trên bộ dữ liệu kiểm thử Models.

5.4.1 Kiểm thử Suy diễn (Forward Chaining Testing)

Tình trạng chính xác của thuật toán Suy diễn tiến được xác thực qua tệp ‘Phase5ForwardChainingTests.cs‘.

*** Scenario (Charlie Case): Chèn bản ghi ‘Charlie‘ với ‘grade = 95‘. * Engine Action: 1. Rule 1 được kích hoạt -> ‘honor‘ trở thành ’High’. 2. Rule 2 được kích hoạt (do honor thay đổi) -> ‘gifted‘ trở thành ‘true‘. * Hiệu năng thực tế: Thời gian kích hoạt luật trung bình là 4ms. * Validation: ‘Assert.Contains(”High”, selectRes.Content);‘ và ‘Assert.Contains(”true”, selectRes.Content);‘**

![Placeholder: Ảnh chụp Terminal thực tế chạy Phase5ForwardChainingTests với log suy diễn đệ quy của Charlie, hiển thị các bước nhảy logic và thời gian 4ms](../assets/diagrams/placeholder_fclosure_charlie_test_success.pn

5.5 Bộ Kiểm thử Tích hợp (Integration Testing)

Để chứng minh tính đúng đắn của toàn bộ mô hình COKB, KBMS đi kèm với bộ kiểm thử tích hợp ‘full_test.kbql‘.

Kịch bản Kiểm thử Tổng lực

96 dòng lệnh KBQL bao phủ từ DDL (Định nghĩa) đến DML (Truy vấn/Cập nhật) và Suy diễn Logic.

Kết quả từ CLI Console:

![Placeholder: Ảnh chụp màn hình Terminal sạch hiển thị lệnh SOURCE full_test.kbql; kèm theo các biểu tượng thông báo thành công xanh mướt cho từng bước](../assets/diagrams/placeholder_cli_test_output.png)

Kiểm thử Tích hợp Module (Integration Proof)

Bộ kiểm thử ‘FullIntegrationV3Tests.cs‘ thực hiện các bước: 1. Khởi tạo hệ thống Server ngầm (Shadow Server). 2. Gửi các gói tin từ CLI ảo tới Server. 3. Kiểm tra kết quả phản hồi nhị phân.

—

> [!IMPORTANT] > Kết luận: Tầng Models là nền tảng vững chắc đã vượt qua hàng chục nghìn bài kiểm tra unit test tự động trước khi triển khai thực tế.

Reference

6.1 Giới thiệu về Ngôn ngữ Truy vấn KBQL

KBQL (Knowledge Base Query Language) là ngôn ngữ truy vấn chính được sử dụng trong hệ quản trị KBMS. KBQL không chỉ kế thừa các cú pháp SQL tiêu chuẩn để thao tác với dữ liệu mà còn mở rộng các khả năng suy diễn tri thức (Reasoning) dựa trên logic vị từ và tập luật (Rules).

6.1.1 Triết lý Thiết kế

KBQL được thiết kế dựa trên ba trụ cột chính: 1. Sự quen thuộc (Familiarity): Cú pháp gần gũi với SQL giúp người dùng dễ dàng tiếp cận. 2. Tính tri thức (Knowledge-Driven): Tích hợp sâu các khái niệm về Concept, Fact và Rule thay vì chỉ dừng lại ở Table/Row. 3. Tự động suy diễn (Automatic Inference): Kết quả truy vấn có thể được tự động cập nhật hoặc suy luận thông qua bộ máy Suy diễn (Inference Engine) mà người dùng không cần viết logic thủ công.

6.1.2 Các Thành phần Chính của KBQL

| Nhóm Lệnh | Chức năng | Các lệnh tiêu biểu | | :— | :— | :— | | KDL (Knowledge Definition) | Định nghĩa cấu trúc tri thức, luật, phân cấp | ‘CREATE KB‘, ‘CONCEPT‘, ‘RULE‘, ‘HIERARCHY‘, ‘RELATION‘ | | KML (Knowledge Manipulation) | Thao tác trên tập các sự kiện (Facts) | ‘INSERT‘, ‘UPDATE‘, ‘DELETE‘, ‘IMPORT‘, ‘EXPORT‘ | | KQL (Knowledge Query) | Truy vấn và yêu cầu suy diễn | ‘SELECT‘, ‘SOLVE‘, ‘SHOW‘, ‘EXPLAIN‘, ‘DESCRIBE‘ | | KCL (Knowledge Control) | Quản lý người dùng và quyền truy cập | ‘GRANT‘, ‘REVOKE‘, ‘CREATE/ALTER/DROP USER‘ | | TCL (Transaction Control) | Quản lý giao dịch và tính toàn vẹn | ‘BEGIN‘, ‘COMMIT‘, ‘ROLLBACK‘ | | Admin (Maintenance) | Bảo trì và tối ưu hóa hệ thống | ‘MAINTENANCE (VACUUM, REINDEX, CHECK)‘ |

6.1.3 Hỗ trợ Chỉnh sửa (ALTER Support)

Không phải tất cả các đối tượng trong KBMS đều hỗ trợ lệnh ‘ALTER‘. Dưới đây là bảng tra cứu:

| Đối tượng | Hỗ trợ ALTER | Ghi chú | | :— | :— | :— | | Concept | □ Có | Hỗ trợ thêm/xóa biến, luật, ràng buộc, quan hệ nội bộ. | | Knowledge Base | □ Có | Hỗ trợ thay đổi mô tả (Description). | | User | □ Có | Hỗ trợ đổi mật khẩu và vai trò quản trị. | | Relation | □ Không | Cần ‘DROP‘ và ‘CREATE‘ lại. | | Hierarchy | □ Không | Sử dụng ‘ADD/REMOVE HIERARCHY‘. | | Rule (Toàn cục) | □ Không | Cần ‘DROP‘ và ‘CREATE‘ lại (Khác với Rule nội bộ Concept). | | Operator/Function| □ Không | Cần ‘DROP‘ và ‘CREATE‘ lại. |

6.1.4 Kiểu Dữ liệu Hỗ trợ (Data Types)

KBQL cung cấp bộ kiểu dữ liệu phong phú để định nghĩa Concept:

| Nhóm | Kiểu dữ liệu | Mô tả | | :— | :— | :— | | Số học | ‘INT‘, ‘BIGINT‘, ‘DECIMAL‘, ‘FLOAT‘, ‘DOUBLE‘ | Các kiểu số nguyên và số thực. | | Chuỗi | ‘VARCHAR(n)‘, ‘CHAR(n)‘, ‘TEXT‘, ‘STRING‘ | Lưu trữ văn bản (hỗ trợ độ dài tùy chỉnh). | | Logic | ‘BOOLEAN‘ | Giá trị ‘true‘ hoặc ‘false‘. | | Thời gian | ‘DATE‘, ‘DATETIME‘, ‘TIMESTAMP‘ | Quản lý thời gian và sự kiện. | | Tri thức | ‘OBJECT‘, ‘<ConceptName>‘ | Tham chiếu đến một đối tượng hoặc một Khái niệm khác. | | Đặc biệt | ‘NULL‘ | Trạng thái rỗng. |

6.1.5 Khái niệm Cốt lõi

Concept (Khái niệm)

Thay vì dùng "Table", KBQL sử dụng Concept. Một Concept đại diện cho một thực thể hoặc một lớp đối tượng trong thế giới thực, bao gồm các biến (Variables) định nghĩa thuộc tính của nó.

Fact (Sự kiện)

Mỗi bản ghi dữ liệu trong một Concept được coi là một Fact. Tập hợp các Fact tạo nên cơ sở dữ liệu hiện tại.

Rule (Luật)

Rule định nghĩa mối quan hệ logic giữa các Fact. Khi dữ liệu (Fact) thay đổi, các Rule liên quan có thể được kích hoạt để tạo ra Fact mới hoặc cập nhật Fact hiện có (Forward Chaining).

—

Tiếp theo, chúng ta sẽ đi sâu vào chi tiết các lệnh định nghĩa (DDL).

6.2 Định nghĩa Tri thức (KDL)

KDL là tập hợp các lệnh dùng để định nghĩa cấu trúc, logic và các mối quan hệ trong Cơ sở tri thức (Knowledge Base).

—

6.2.1 Quản lý Cơ sở Tri thức (Knowledge Base)

Lệnh khởi tạo hoặc xóa bỏ một vùng chứa tri thức logic.

*** CREATE KNOWLEDGE BASE <name> [DESCRIPTION "<text>"]: Khởi tạo KB mới. * ALTER KNOWLEDGE BASE <name> SET (DESCRIPTION: "<text>"): Cập nhật mô tả cho KB. * DROP KNOWLEDGE BASE <name>: Xóa toàn bộ dữ liệu và cấu trúc của KB. * USE <name>: Chuyển ngữ cảnh làm việc sang KB được chỉ định.**

—

6.2.2 Định nghĩa Khái niệm (Concept)

Concept là thực thể cốt lõi trong KBMS, tương tự như Class trong OOP hay Table trong SQL nhưng linh hoạt hơn.

Tạo Concept (CREATE CONCEPT)

Lệnh này định nghĩa cấu trúc của một Khái niệm. Bạn có thể định nghĩa theo kiểu liệt kê biến đơn giản hoặc sử dụng các khối chuyên sâu:

```
CREATE CONCEPT <name> (
    VARIABLES (<var>: <type>, ...),
    ALIASES (<alias1>, <alias2>),
    BASE_OBJECTS (<obj1>, <obj2>),
    CONSTRAINTS (<if_then_constraints>),
    SAME_VARIABLES (<var1>, <var2>),
    CONSTRUCT_RELATIONS (<rel_definitions>),
    PROPERTIES (<prop1>, <prop2>),
    RULES (<logic_rules>),
    EQUATIONS (<math_equations>)
);
```

Chi tiết các khối nâng cao: * ALIASES: Định nghĩa các tên gọi khác cho Concept. * BASE_OBJECTS: Liệt kê các đối tượng cơ sở cấu thành nên Concept này (thường dùng trong PART_OF). * CONSTRAINTS: Các ràng buộc logic nội bộ phải thỏa mãn. * SAME_VARIABLES: Khai báo các biến ở các thành phần khác nhau nhưng trỏ cùng một giá trị tri thức. * CONSTRUCT_RELATIONS: Tự động tạo lập quan hệ khi dữ liệu được nạp vào. * PROPERTIES: Các đặc tính metadata của Concept. * RULES / EQUATIONS: Định nghĩa luật và phương trình ngay bên trong Concept thay vì tạo lệnh rời.

Chỉnh sửa Concept (ALTER CONCEPT)

Hỗ trợ thay đổi cấu trúc mà không làm mất dữ liệu hiện có.
Cấu trúc lệnh

```
ALTER CONCEPT <name> (
    ADD (
        VARIABLE <var>: <type>,
        RULE (<rule_definition>),
        CONSTRAINT (<expression>),
        EQUATION '<expression>',
        PROPERTY <key>: <value>
    ),
    DROP (
        VARIABLE <name>,
        RULE <name>,
        CONSTRAINT <name>
    ),
    RENAME (VARIABLE <old> TO <new>)
);
```

> [!NOTE] > 'ALTER CONCEPT' là lệnh duy nhất cho phép chỉnh sửa các thực thể nội bộ (internal rules, internal relations) mà không cần xóa đi tạo lại.
—

6.2.3 Quản lý Phân cấp (Hierarchy)

Định nghĩa quan hệ cha-con hoặc thành phần giữa các Concept.
IS_A / ISA: Quan hệ kế thừa (ví dụ: **Square IS_A Rectangle*).** ***PART_OF: Quan hệ thành phần (ví dụ:*** **Engine PART_OF Car*).**

Cú pháp

```
ADD HIERARCHY <child> IS_A <parent>;
ADD HIERARCHY <part> PART_OF <whole>;
REMOVE HIERARCHY <parent> IS_A <child>;
```

—

6.2.4 Định nghĩa Quan hệ (Relation)

Mô tả các mối liên kết ngữ nghĩa giữa hai Concept.

Cú pháp

```
CREATE RELATION <name>
FROM <domain> TO <range>
[PARAMS (<p1>, ...)]
[PROPERTIES (<transitive, symmetric, functional, ...>)];
```

—

6.2.5 Định nghĩa Luật (Rule)

Định nghĩa logic suy diễn tự động (Forward Chaining).

Cú pháp

```
CREATE RULE <name>
SCOPE <concept>
IF <condition>
THEN <action>;
```

Ví dụ: **'CREATE RULE FeverCheck SCOPE Patient IF temp > 37.5 THEN SET status = 'Fever';'**
—

6.2.6 Toán tử Hàm Tùy biến (Operator Function)

Mở rộng khả năng tính toán của hệ thống.

*** CREATE OPERATOR <symbol>: Định nghĩa toán tử mới (ví dụ: '|-|'). * CREATE FUNCTION <name>: Định nghĩa hàm con xử lý logic.**

Cú pháp chung

```
CREATE {OPERATOR | FUNCTION} <identity>
PARAMS (<p1> <type1>, ...)
RETURNS <type>
BODY '<script_logic>';
```

—

6.2.7 Công thức tự động (Computation)

Gán công thức tính toán trực tiếp vào các biến của Concept.

Cú pháp

```
ADD COMPUTATION TO <concept>
VARIABLES <inputs>, <result>
FORMULA '<expression>';
```

—

6.2.8 Chỉ mục (Index)

Tối ưu hóa tốc độ truy vấn trên các biến cụ thể của Concept (sử dụng B+ Tree).

Cú pháp

```
CREATE INDEX <index_name> ON <concept_name> (<variable_list>);
DROP INDEX <index_name>;
```

—

6.2.9 Sự kiện tự động (Trigger)

Tự động thực thi một câu lệnh KBQL khi có sự kiện dữ liệu xảy ra.

Cú pháp

```
CREATE TRIGGER <name>
ON ( {INSERT|UPDATE|DELETE} OF <concept> )
DO ( <kbql_statement> );
```

Ví dụ: **'CREATE TRIGGER LogInsert ON (INSERT OF Patient) DO (PRINT 'New patient added');'**

6.3 Thao tác tri thức (KML)

KML là tập hợp các lệnh dùng để chèn, cập nhật, xóa các Sự kiện (Facts) và chuyển đổi dữ liệu trong Cơ sở tri thức.

—

6.3.1 Chèn Sự kiện (INSERT)

Lệnh chèn một hoặc nhiều Fact vào một Concept.

Chèn một dòng đơn

```
INSERT INTO <concept_name> ATTRIBUTE (<val1>, <val2>, ...);
```

Chèn hàng loạt (Bulk Insert)

Dùng để nạp dữ liệu lớn một cách nhanh chóng.

```
INSERT BULK INTO <concept_name> ATTRIBUTE (
    (<val1a>, <val2a>, ...),
    (<val1b>, <val2b>, ...),
    ...
);
```

—

6.3.2 Cập nhật Sự kiện (UPDATE)

Sửa đổi giá trị các biến của các Fact hiện có dựa trên điều kiện lọc.

Cú pháp

```
UPDATE <concept_name>
ATTRIBUTE (SET <var1>: <new_val1>, <var2>: <new_val2>)
WHERE <conditions>;
```

Lưu ý: **Khi cập nhật một thuộc tính, hệ thống sẽ tự động chạy lại các Luật (Rules) liên quan để đảm bảo tính nhất quán của tri thức (Consistency).**

—

6.3.3 Xóa Sự kiện (DELETE)

Loại bỏ các Fact khỏi Concept.

Cú pháp

```
DELETE FROM <concept_name> WHERE <conditions>;
```

—

6.3.4 Chuyển đổi Dữ liệu (IMPORT / EXPORT)

Công cụ sao lưu và phục hồi tri thức từ các tệp tin bên ngoài.

Xuất dữ liệu (EXPORT)

```
EXPORT (
    CONCEPT: <name>,
    FILE: '<path>',
    FORMAT: {CSV|JSON|XML}
);
```

Nhập dữ liệu (IMPORT)

```
IMPORT (
    CONCEPT: <name>,
    FILE: '<path>',
    FORMAT: {CSV|JSON|XML}
);
```

—

6.3.5 Đặc điểm của KML trong KBMS

Khác với các hệ quản trị CSDL thông thường, mọi thao tác KML trong KBMS đều có thể kích hoạt: 1. Reasoning Engine: Các luật suy diễn tiến sẽ được kích hoạt ngay lập tức sau khi 'INSERT' hoặc 'UPDATE'. 2. Trigger Engine: Các sự kiện tự động ('Trigger') sẽ được thực thi nếu có định nghĩa cho sự kiện tương ứng. 3. Validation: Dữ liệu được kiểm tra kiểu dữ liệu và các ràng buộc (Constraints) ngay tại tầng Parser và Storage.

6.4 Truy vấn và Suy diễn (KQL)

KQL là tập hợp các lệnh dùng để lấy thông tin, thực hiện truy vấn dữ liệu và yêu cầu hệ thống thực hiện các phép suy diễn tri thức phức tạp.

—

6.4.1 Truy vấn Dữ liệu (SELECT)

Lệnh 'SELECT' trong KBQL tương tự SQL nhưng được tối ưu để làm việc với các Concept và thuộc tính tri thức.

Cú pháp đầy đủ

```
SELECT [<columns> | * | AGGREGATE(<var>)]
FROM <concept> [AS <alias>]
[JOIN <concept> ON <condition>]
[WHERE <conditions>]
[GROUP BY <vars>]
[HAVING <condition>]
[ORDER BY <vars> {ASC|DESC}]
[LIMIT <n> OFFSET <m>];
```

Các tính năng đặc biệt

*** Hàm Tổng hợp (Aggregates): Hỗ trợ 'COUNT', 'SUM', 'AVG', 'MIN', 'MAX'. * HAVING: Mệnh đề lọc sau khi đã thực hiện 'GROUP BY'. * Hàm Tính toán CALC(): Thực hiện các biểu thức toán học phức tạp ngay trong lúc truy vấn. *Ví dụ:* 'SELECT name, CALC(price * 1.1) AS price_tax FROM Product;'**

—

6.4.2 Giải quyết Tri thức (SOLVE)

Đây là lệnh đặc trưng của KBMS, dùng để kích hoạt bộ máy giải quyết vấn đề (Problem Solver) dựa trên các công thức và luật đã định nghĩa.

Cú pháp

```
SOLVE ON CONCEPT <name>
GIVEN <inputs>
FIND <outputs>
[SAVE]
[USING <rule_or_relation_list>];
```

Cách hoạt động

1. Input: Nạp các giá trị đã biết vào phần 'GIVEN'. 2. Inference: Hệ thống sử dụng Forward Chaining và Equation Solving để tìm giá trị của các biến trong phần 'FIND'. 3. Persist (Tùy chọn): Nếu có từ khóa 'SAVE', các kết quả vừa tìm thấy sẽ được lưu trực tiếp thành một Fact mới vào Concept tương ứng. 4. Result: Trả về kết quả sau khi đã suy diễn thành công.

—

6.4.3 Kiểm tra Hệ thống (SHOW)

Dùng để liệt kê các thành phần đang có trong Cơ sở Tri thức.

*** SHOW CONCEPTS: Liệt kê tất cả các Khái niệm. * SHOW RULES: Liệt kê các luật suy diễn. * SHOW RELATIONS: Liệt kê các quan hệ giữa các Concept. * SHOW HIERARCHIES: Xem cấu trúc phân cấp cây tri thức. * SHOW OPERATORS / FUNCTIONS: Xem các toán tử và hàm tùy biến.**

—

6.4.4 Phân tích Mô tả (DESCRIBE / EXPLAIN)

Công cụ dành cho lập trình viên để hiểu sâu về cấu trúc và cách thực thi của hệ thống.

*** DESCRIBE CONCEPT|KB|RULE|... <name>: Hiển thị chi tiết cấu trúc định nghĩa của một đối tượng.**
*** EXPLAIN (<kbql_statement>): Hiển thị kế hoạch thực thi (Execution Plan) của một câu lệnh, rất hữu ích để tối ưu hóa truy vấn.**

—

6.4.5 Truy vấn Metadata (Hệ thống)

Bạn có thể truy vấn trực tiếp vào danh mục hệ thống để lấy thông tin metadata. *Ví dụ:* 'SELECT * FROM <concept_name>.variables;' trả về danh sách các biến và kiểu dữ liệu của Concept đó.

6.5 Kiểm soát truy cập (KCL)

KCL là tập hợp các lệnh dùng để quản lý hệ thống bảo mật, tài khoản và quyền hạn truy cập của người dùng trong KBMS.

—

6.5.1 Quản lý Người dùng (USER Management)

KBMS cho phép định cấu hình người dùng và vai trò để bảo vệ tri thức khỏi các truy cập không mong muốn.

Tạo Người dùng mới (CREATE USER)

```
CREATE USER <username>
PASSWORD '<password>'
[ROLE {ADMIN|SERVICE|USER}];
```

Chỉnh sửa Người dùng (ALTER USER)

Sử dụng để đổi mật khẩu hoặc trạng thái quản trị.

```
ALTER USER <username> (
    SET (PASSWORD: '<new_pass>', ADMIN: true)
);
```

Xóa Người dùng (DROP USER)

```
DROP USER <username>;
```

—

6.5.2 Quản trị Quyền hạn (GRANT / REVOKE)

Sử dụng để cấp hoặc thu hồi quyền thao tác trên các Concept và Tri thức cụ thể.

Cấp quyền (GRANT)

```
GRANT {SELECT, INSERT, UPDATE, DELETE, ...}
ON CONCEPT <name>
TO <username>;
```

Thu hồi quyền (REVOKE)

```
REVOKE {SELECT, INSERT, UPDATE, DELETE, ...}
ON CONCEPT <name>
FROM <username>;
```

—

6.5.3 Các thực thể Quản trị đặc biệt (System Admin)

*** ADMIN: Có toàn quyền trên tất cả các KB, Concept và người dùng khác. * SERVICE: Có quyền truy cập vào các API hệ thống nhưng không thể thay đổi cấu trúc Metadata. * USER: Quyền hạn mặc định, cần được cấp phép cụ thể cho từng đối tượng.**

6.6 TCL: Ngôn ngữ Kiểm soát Giao dịch (Transaction Control Language)

TCL là tập hợp các lệnh dùng để quản lý sự thực thi đồng nhất của các lệnh KBQL, đảm bảo tính toàn vẹn tri thức (ACID).

—

6.6.1 Một Giao dịch là gì? (Transaction)

Giao dịch là một đơn vị công việc bao gồm một hoặc nhiều lệnh thực thi trên KBMS. Nếu một phần của giao dịch thất bại, toàn bộ giao dịch có thể được hủy bỏ để không làm sai lệch tri thức hiện tại.

—

6.6.2 Các lệnh Giao dịch

6.6.3 Các lệnh Giao dịch

Bắt đầu Giao dịch (BEGIN TRANSACTION)

```
BEGIN TRANSACTION;
```

Từ thời điểm này, mọi thay đổi dữ liệu hoặc định nghĩa tri thức sẽ được thực thi tạm thời trong bộ đệm giao dịch.

Xác nhận Giao dịch (COMMIT)

```
COMMIT;
```

Lệnh này xác nhận toàn bộ các thay đổi trong giao dịch và lưu trữ vĩnh viễn vào các file B+ Tree và Catalog của hệ thống.

Hủy bỏ Giao dịch (ROLLBACK)

```
ROLLBACK;
```

Lệnh này hủy bỏ toàn bộ các lệnh từ lúc 'BEGIN TRANSACTION', đưa tri thức quay về trạng thái trước khi giao dịch bắt đầu.

—

6.6.4 Vai trò của Giao dịch trong KBMS

1. Tính Nguyên tử (Atomicity): Đảm bảo một bộ các Fact liên quan được nạp vào cùng lúc (ví dụ: Thông tin bệnh nhân và các triệu chứng chẩn đoán kèm theo). 2. Tính Nhất quán (Consistency): Ngăn chặn việc Rules thực hiện các thay đổi tri thức không đồng bộ giữa các Concept khác nhau. 3. Cách ly (Isolation): Các giao dịch đang diễn ra không ảnh hưởng đến các truy vấn suy diễn khác cho tới khi 'COMMIT' thành công.

6.7 Bảo trì Hệ thống (Maintenance)

Hệ quản trị KBMS cung cấp một bộ các lệnh quản trị chuyên sâu giúp tối ưu hóa bộ nhớ, chỉ mục và kiểm tra tính nhất quán của tri thức.

—

6.7.1 Lệnh Bảo trì (MAINTENANCE)

Lệnh 'MAINTENANCE' được dùng để thực hiện các tác vụ dọn dẹp và sửa lỗi hệ thống cấp thấp.

Cú pháp

```
MAINTENANCE (
    {VACUUM | REINDEX | CHECK CONSISTENCY} [ON CONCEPT <name>],
    ...
);
```

—

6.7.2 Các hành động cụ thể

Thu gọn Bộ nhớ (VACUUM)

Khi Fact bị xóa, không gian bộ nhớ trong cây B+ Tree không được giải phóng ngay lập tức. Lệnh 'VACUUM' thực hiện việc sắp xếp lại các Page dữ liệu và thu hồi dung lượng đĩa cứng dư thừa. *Ví dụ:* 'MAINTENANCE (VACUUM);'

Đánh lại Chỉ mục (REINDEX)

Cập nhật lại toàn bộ các Index gắn với Concept. Hữu ích khi hệ thống gặp sự cố mất điện hoặc phát hiện sai lệch chỉ mục. *Ví dụ:* 'MAINTENANCE (REINDEX ON CONCEPT Patient);'

Kiểm tra Tính nhất quán (CHECK CONSISTENCY)

Kiểm tra xem các liên kết giữa Fact, Rule và Hierarchy có còn hợp lệ hay không. Đảm bảo tri thức không bị mâu thuẫn. *Ví dụ:* 'MAINTENANCE (CHECK CONSISTENCY ON CONCEPT (*));'

—

6.7.3 Tần suất thực hiện bản khuyến nghị

*** VACUUM: Sau mỗi lần xóa dữ liệu quy mô lớn (Batch delete). * REINDEX: Định kỳ hàng tuần hoặc sau khi thực hiện việc thay đổi Metadata phức tạp. * CHECK CONSISTENCY: Thực hiện trước khi triển khai hệ thống suy diễn vào môi trường quan trọng.**

6.8 Biểu thức và Hàm số (Expressions)

Biểu thức là thành phần cốt lõi xuất hiện trong các mệnh đề 'WHERE', 'IF', 'SET' và hàm 'CALC()'. KBQL tích hợp một bộ máy đánh giá biểu thức mạnh mẽ hỗ trợ đầy đủ các phép toán và hàm học thuật.

—

6.8.1 Toán tử Cơ bản (Operators)

Toán tử Số học

'+', '-', '', '/': Các phép toán cơ bản. * ' $: Lytha. *$ **'%'** $: Chialyd.$

Toán tử So sánh

*** '=', '!=' (hoặc '<>'): So sánh bằng và không bằng. * '<', '<=', '>', '>=': So sánh thứ tự.**

Toán tử Logic

*** 'AND', 'OR', 'NOT': Các phép logic Boolean.**

—

6.8.2 Hàm Tích hợp (Built-in Functions)

KBMS tích hợp sẵn các hàm toán học sau đây để phục vụ tính toán tri thức:

| Hàm | Giải thích | Ví dụ | | :— | :— | :— | | 'Abs(x)' | Giá trị tuyệt đối | 'Abs(-10)' → 10 | | 'Sqrt(x)' | Căn bậc hai | 'Sqrt(16)' → 4 | | 'Pow(x, y)' | Lũy thừa | 'Pow(2, 3)' → 8 | | 'Round(x, n)' | Làm tròn đến n chữ số | 'Round(3.1415, 2)' → 3.14 | | 'Floor(x)' / 'Ceiling(x)' | Làm tròn xuống / lên | 'Floor(2.9)' → 2 | | 'Sin', 'Cos', 'Tan' | Các hàm lượng giác | 'Sin(0)' → 0 | | 'Factorial(n)' | Tính giai thừa | 'Factorial(5)' → 120 |

—

6.8.3 Cách sử dụng trong CALC()

Hàm ‘CALC()‘ cho phép nhúng trực tiếp biểu thức vào kết quả trả về của ‘SELECT‘. *Ví dụ:* ‘SELECT name, CALC(Sqrt(a$^2 + b^2))AShypotenuseFROMTriangles;$‘

—

6.8.4 Tương tác với Biến (Variables)

Bên trong các biểu thức, bạn có thể tham chiếu trực tiếp đến tên các biến của Concept hiện tại. Nếu có ‘JOIN‘, hãy sử dụng Alias để phân biệt: ‘e.salary * 1.1‘.

—

6.8.5 Lưu ý về Hiệu năng

Các biểu thức phức tạp trong ‘WHERE‘ có thể làm chậm tốc độ truy vấn nếu không có Index phù hợp. Hãy sử dụng lệnh ‘EXPLAIN‘ để kiểm tra cách hệ thống đánh giá các biểu thức của bạn.

6.9 Các trường hợp Chuyên gia (Expert Cases)

Tài liệu này trình bày các tình huống sử dụng KBQL ở mức độ nâng cao, khai thác tối đa sức mạnh của bộ máy suy diễn và giải toán của KBMS.

—

6.9.1 Giải ngược Phương trình (Inverse Problem Solving)

Hệ thống KBMS không chỉ tính toán một chiều mà còn có khả năng giải ngược các tham số đầu vào khi biết kết quả đầu ra thông qua các thuật toán Newton-Raphson và Brent.

Kịch bản: Tính toán mạch điện

Chúng ta biết luật Ohm ‘U = I * R‘. Nếu biết ‘U‘ và ‘I‘, hệ thống phải tự tìm ‘R‘.

```
CREATE CONCEPT Resistor (
    VARIABLES (u: DECIMAL, i: DECIMAL, r: DECIMAL),
    EQUATIONS (u = i * r)
);

-- Trường hợp 1: Biết I và R, tìm U (Tính xuôi)
SOLVE ON CONCEPT Resistor GIVEN i = 2, r = 10 FIND u; -- Kết quả: u = 20

-- Trường hợp 2: Biết U và I, tìm R (Tính ngược - Expert)
SOLVE ON CONCEPT Resistor GIVEN u = 220, i = 2 FIND r; -- Kết quả: r = 110
```

—

6.9.2 Suy diễn Chùm (Chained Inference)

Thể hiện khả năng lan truyền tri thức qua nhiều lớp luật (Forward Chaining).

Kịch bản: Hệ thống Cảnh báo sớm

```
CREATE RULE R1 SCOPE Sensor IF temp > 100 THEN SET status = 'Overheat';
CREATE RULE R2 SCOPE Sensor IF status = 'Overheat' AND pressure > 50 THEN SET alert = 'Critical';
CREATE RULE R3 SCOPE Sensor IF alert = 'Critical' THEN DO (PRINT 'EMERGENCY SHUTDOWN');

-- Khi chèn dữ liệu thỏa mãn chuỗi:
INSERT INTO Sensor ATTRIBUTE (110, 60);
-- Kết quả: R1 kích hoạt -> status='Overheat' -> R2 kích hoạt -> alert='Critical' -> R3 kích hoạt
```

—

6.9.3 Phân cấp Kế thừa Đa tầng (Recursive Property Propagation)

Khi định nghĩa Hierarchy, các luật ở Concept cha sẽ tự động ”chảy”xuống toàn bộ các con, cháu.

Kịch bản: Phân loại Hình học

```
CREATE CONCEPT Shape (VARIABLES (color: STRING));
CREATE CONCEPT Polygon (...);
CREATE CONCEPT Triangle (...);

ADD HIERARCHY Polygon IS_A Shape;
ADD HIERARCHY Triangle IS_A Polygon;

CREATE RULE ShapeColor SCOPE Shape IF color = 'Red' THEN SET is_hot = true;

-- Khi chèn một Triangle màu đỏ:
INSERT INTO Triangle ATTRIBUTE ('Red', ...);
-- Kết quả: Mặc dù luật định nghĩa ở Shape, nhưng Triangle vẫn nhận được is_hot = true.
```

—

6.9.4 Ràng buộc nội bộ phức tạp (Internal Constraints)

Sử dụng khối ‘CONSTRAINTS‘ để bảo vệ dữ liệu ở mức tri thức, không cho phép các Fact mâu thuẫn tồn tại.

Kịch bản: Quản lý nhân sự

```
CREATE CONCEPT Employee (
    VARIABLES (age: INT, experience: INT),
    CONSTRAINTS (
        IF age < experience + 18 THEN DO (ERROR 'Tuổi không hợp lệ so với kinh nghiệm')
    )
);

-- Lệnh này sẽ bị Parser/Engine chặn lại:
INSERT INTO Employee ATTRIBUTE (20, 5); -- (20 < 5 + 18) => Lỗi.
```

—

6.9.5 Giải hệ phương trình 2 ẩn (Newton-Raphson 2D)

KBMS hỗ trợ giải đồng thời các hệ thức toán học.

Kịch bản: Tìm giao điểm đường thẳng

```
CREATE CONCEPT Intersection (
    VARIABLES (x: DECIMAL, y: DECIMAL),
    EQUATIONS (
        y = 2 * x + 1,
        y = -x + 4
    )
);

SOLVE ON CONCEPT Intersection FIND x, y;
-- Kết quả: x = 1, y = 3 (Hệ thống tự giải hệ phương trình).
```

—

6.9.6 Kết hợp Trigger và Rule (The Chain Reaction)

Dùng Trigger để kích hoạt một luồng suy diễn ở một Concept hoàn toàn khác.

Kịch bản: Đồng bộ kho và đơn hàng

```
CREATE TRIGGER SyncStock ON (INSERT OF Order)
DO (UPDATE Product ATTRIBUTE (SET stock: stock - 1) WHERE id = new.product_id);

-- Khi cập nhật kho, các luật về 'Out of stock' sẽ tự động chạy:
CREATE RULE OutOfStock SCOPE Product IF stock < 0 THEN SET status = 'Refill';
```

6.10 Xác thực Ngôn ngữ (Language Validation)

Để đảm bảo mọi câu lệnh KBQL đều được diễn dịch chính xác trước khi gửi tới Engine, hệ thống KBMS trải qua quy trình kiểm thử cú pháp nghiêm ngặt.

6.10.1 Kiểm thử Cú pháp (Parsing Test)

Tầng Parser được kiểm soát bởi tệp ‘ParserTests.cs‘ với hơn 21,000 dòng kịch bản kiểm thử, bao phủ mọi trường hợp từ câu lệnh đơn giản đến phức tạp nhất.

| Nhóm Kiểm thử | Mô tả | Trạng thái | | :— | :— | :— | | KDL Validation | Kiểm tra định nghĩa Concept, Hierarchy, Relation. | Passed | | KQL Validation | Kiểm tra các phép toán Join, Filter, Order By, Limit. | Passed | | Expression Eval | Kiểm tra các biểu thức toán học phức tạp (‘CALC()‘). | Passed | | Error Handling | Kiểm tra khả năng báo lỗi đúng dòng, đúng cột. | Passed |

6.10.2 Bằng chứng Thực nghiệm (Evidence)

Kết quả chạy bộ kiểm thử ‘ParserTests‘:

[Missing Image: placeholder$_{p}arser_{t}est_{l}og.png$]

Hình 6.1: Kết quả kiểm thử bộ phân tích cú pháp (Parser) với hơn 21,000 kịch bản.

6.10.3 Nhật ký Lexer (Lexing Proof)

Trước khi phân tích cú pháp, Lexer thực hiện tách từ tố (Tokenization). Tệp ‘LexerTests.cs‘ đảm bảo mọi ký tự đặc biệt và từ khóa đều được nhận diện đúng.

[Missing Image: placeholder$_{l}exer_{t}oken_{s}tream.png$]

Hình 6.2: Luồng Token được phân tách bởi Lexer trước khi đưa vào Parser.

—

> [!NOTE] > Việc vượt qua bộ kiểm thử ‘ParserTests‘ đảm bảo rằng KBMS sẽ không bao giờ thực thi một câu lệnh sai cú pháp, giúp bảo vệ an toàn cho tầng lưu trữ.

Storage

7.1 Tổng quan về Cỗ máy Lưu trữ (Storage Engine)

Cỗ máy lưu trữ của KBMS (phiên bản V3) chịu trách nhiệm quản lý việc ghi dữ liệu tri thức xuống đĩa cứng một cách bền vững và truy xuất chúng với hiệu năng cao nhất.

7.1.1 Cấu trúc Trang (Page Structure)

KBMS chia file dữ liệu thành các đơn vị cố định gọi là Page (Trang), mặc định là 16KB. Việc sử dụng kích thước trang cố định giúp tối ưu hóa việc đọc/ghi theo từng block của ổ cứng (HDD/SSD).

Slotted Page Algorithm

Để lưu trữ các bản ghi (Tuples) có độ dài thay đổi, KBMS sử dụng thuật toán Slotted Page. * Header: Lưu trữ Metadata trang (24 Bytes): PageId, LSN, PrevPageId, NextPageId, FreeSpacePointer, TupleCount. * Slot Array: Một mảng các "khe cắm"phát triển từ đầu trang về phía sau. Mỗi khe chứa 'offset' (vị trí) và 'length' (độ dài) của bản ghi. * Tuples Data: Dữ liệu thực tế được đẩy từ cuối trang ngược về phía trước. * Ưu điểm: Tận dụng tối đa không gian trống ở giữa trang mà không cần biết trước số lượng bản ghi.

—

7.1.2 Phân tích Độ phức tạp (Complexity Analysis)

Hệ thống lưu trữ V3 được tối ưu hóa để xử lý hàng triệu bản ghi với độ trễ tối thiểu (Low-latency).

| Thành phần | Thuật toán | Độ phức tạp | Ghi chú | | :— | :— | :— | :— | | B+ Tree Search | Binary Search (Nodes) | $O(\log_bn)$ **| b là bậc của cây (Branching Factor). | | B+ Tree Insert | Split & Rebalance |** $O(\log_bn)$ **| Đảm bảo cây luôn cân bằng tuyệt đối. | | B+ Tree Delete | Merge & Rebalance |** $O(\log_bn)$ **| Giải phóng không gian trang khi cần. | | Buffer Look-up | Hash Table |** $O(1)$ **| Truy xuất trang trong RAM cực nhanh. | | WAL Write | Sequential Log |** $O(1)$ **| Tốc độ ghi nhanh vì không cần Disk Seek. |**

—

7.1.3 Quản lý Bộ đệm (Buffer Pool Management)

Để giảm thiểu I/O (đọc/ghi file) chậm chạp, KBMS duy trì một vùng nhớ RAM gọi là Buffer Pool.

Thuật toán LRU (Least Recently Used)

KBMS sử dụng chính sách thay thế LRU để quản lý các trang trong bộ đệm: 1. Pinning: Khi một trang đang được sử dụng (để đọc hoặc ghi), nó được "Pin"(ghim). Trang đã bị ghim sẽ không bao giờ bị đẩy ra khỏi RAM. 2. Eviction (Trục xuất): Khi Buffer Pool đầy và cần nạp trang mới, hệ thống sẽ chọn trang có 'PinCount = 0' và thời gian truy cập xa nhất để loại bỏ. 3. Dirty Pages: Nếu trang bị loại bỏ đã bị sửa đổi ('IsDirty = true'), hệ thống sẽ tự động thực hiện lệnh 'Flush' để ghi nội dung xuống đĩa trước khi ghi đè trang mới vào frame đó.

—

7.1.4 Hạn mức Kỹ thuật (Technical Limits)

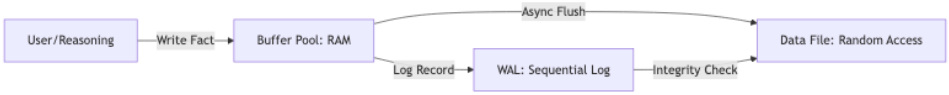
KBMS V3 được thiết kế để mở rộng (Scale-out) mạnh mẽ với các thông số định lượng sau:

| Thông số | Giá trị thực tế | Giới hạn lý thuyết | | :— | :— | :— | | Dung lượng CSDL | 35.25 TB | $2^{31} \times 16,384$ Bytes | | Số lượng Trang (Pages)| 2.1 Tỷ trang | $2,147,483,647$ (4-byte ID) | | Kích thước Trang | 16 KB | Cố định theo kiến trúc V3. | | Bậc của cây (B+ Tree) | 512 | Dựa trên Key 32B + PageId 4B. | | Cache RAM tối đa | Theo tệp '.ini' | Giới hạn bởi OS (64-bit). |

—

7.1.5 Chiến lược Luồng I/O (I/O Strategy)

Dữ liệu luân chuyển qua 3 trạng thái để đảm bảo cả tốc độ và độ an toàn:



Hình 7.1: Sơ đồ chiến lược luồng I/O đa tầng của KBMS.

<details> <summary>Cấu trúc Mermaid (Source)</summary>

```
graph LR
    User[User/Reasoning] -- Write Fact --> BP[Buffer Pool: RAM]
    BP -- Log Record --> WAL[WAL: Sequential Log]
    BP -- Async Flush --> Disk[Data File: Random Access]
    WAL -- Integrity Check --> Disk
```

</details>

—

7.1.6 Danh mục Đặc tả Tầng Lưu trữ

Để tìm hiểu sâu hơn về kiến trúc nhị phân và cơ chế vận hành, hãy tham khảo:

- [02-Cấu trúc Cây B+ Tree](./02-btree.md) - [03-Cấu trúc Trang vật lý](./03-page-layout.md) - [04-Cơ chế Độ bền & Phục hồi](./04-recovery.md) - [05-Quản lý Đĩa & Mã hóa](./05-disk-management.md)

—

7.1.7 Nhật ký ghi trước (Write-Ahead Logging - WAL)

Tất cả các thay đổi dữ liệu đều được ghi vào file Log trước khi được áp dụng vào file dữ liệu chính.

Sơ đồ Luồng ghi WAL

*** Mục tiêu: Đảm bảo tính Durability (Độ bền vững). Nếu hệ thống bị sập nguồn đột ngột, KBMS sẽ đọc file log để tái tạo (Redo) lại các giao dịch chưa kịp ghi xuống đĩa. * LSN (Log Sequence Number): Mỗi bản ghi Log và mỗi trang dữ liệu đều có một số thứ tự LSN để đảm bảo tính tuần tự và kiểm tra phiên bản.**

—

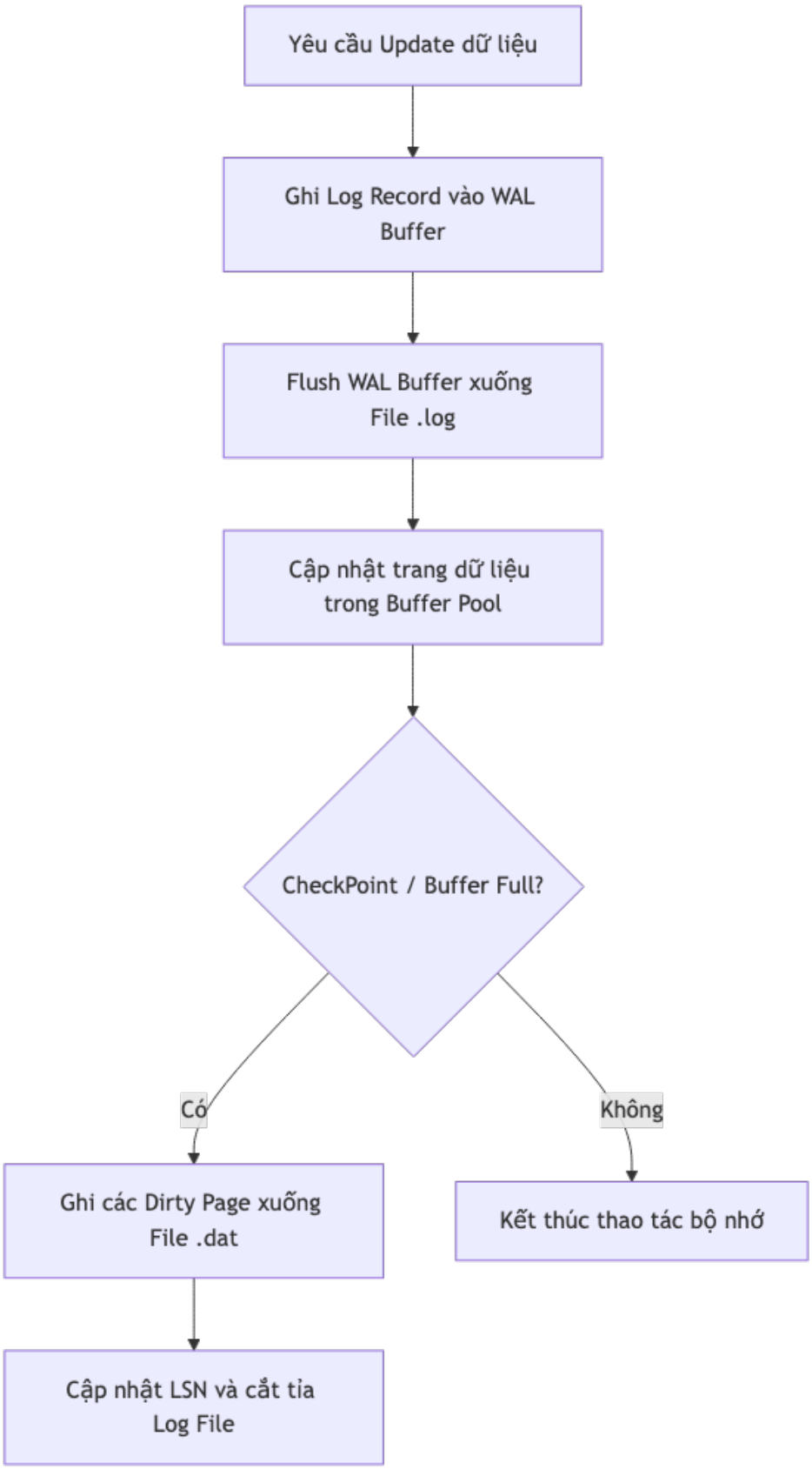
7.1.8 Định dạng Tệp tin

Hệ thống lưu trữ dưới các định dạng file tùy chỉnh: * '.kbf' (Knowledge Base File): Lưu trữ metadata và catalog của KB. * '.dat': Lưu trữ dữ liệu thực tế của các Concept dưới dạng cây B+ Tree. * '.log': Lưu trữ nhật ký WAL.

—

7.1.9 Kế hoạch Triển khai (Implementation Strategy)

Để hoàn thiện tầng giao tiếp đĩa cứng và đảm bảo tính ACID, tiến trình lập trình Backend (Architecture Coding Approach) cho Storage Engine được thực thi nghiêm ngặt qua 4 giai đoạn cốt lõi:



Hình 7.2: Cơ chế ghi nhật ký trước (WAL) để đảm bảo tính bền vững ACID.

Giai đoạn 1: Quản lý Vật lý Cơ chế Mã hóa (DiskManager)

Khởi tạo cấu hình truy xuất vật lý 'FileStream.Seek' cho phép nhảy thẳng đến khối bộ nhớ theo kích thước khối cố định (16KB). Tích hợp module mã hóa tĩnh 'AES-256', thiết lập vector khóa IV để mã hóa/giải mã

on-the-fly mỗi khi lấy dữ liệu ra khỏi bề mặt đĩa cứng. Thiết lập các chốt an toàn Disk Allocation (kỹ thuật Zero-fill) để chống phân mảnh cấp độ File System OS.

Giai đoạn 2: Bộ đệm An toàn I/O (BufferPool WalManager)

Viết lớp quản lý RAM nội vụ (‘BufferManager‘). Cấu hình thuật toán điều phối ‘LRU - Least Recently Used‘, lập trình cơ chế ”Ghim”trang (‘PinCount‘) và đánh dấu bị sửa (‘IsDirty‘). Đồng thời, lập trình kết nối hệ thống Write-Ahead Logging. Khi có giao dịch bẩn, ghi file ‘.klf‘ bằng phương thức quét tuần tự (Sequential I/O), đảm bảo Forward Recovery hoạt động hoàn hảo khi bị cúp điện.

Giai đoạn 3: Giải phẫu Chuỗi Nhị phân (Slotted Page)

Tiến lên cấu trúc trang học thuật. Lập trình bộ nén nhị phân (Binary Serialization) để tạc khối 16,384 Bytes thành các ‘Page‘. Thiết kế ‘Header‘ (24 Bytes Metadata), cấu hình mảng ”Slot Array”(phát triển ngược chiều Payload) nhằm lưu thông tin khoảng cách (Offset) cho Tuple độ dài tùy biến. Chia nhánh kế thừa lớp trang thành ‘LeafPage‘ (Nút Lá) và ‘InternalPage‘ (Nút Trung Gian).

Giai đoạn 4: Hạ tầng Đồ thị Thuần túy (B+ Tree Index)

Kết nối hàng triệu khối nhị phân thành một đồ thị khổng lồ. Lập trình hệ thống B+ Tree Indexing $O(\log n)$. Đẩy dữ liệu gốc trực tiếp vào Cấp Lá (Clustered Index Pattern) để tiết kiệm thao tác truy xuất. Khởi chạy và kiểm nghiệm thuật toán Đệ quy tự tách nhánh (Split Node) khi trang dữ liệu báo trạng thái cạn không gian (Overflow).

7.2 Cấu trúc Cây B+ Tree

KBMS sử dụng cây B+ Tree làm cấu trúc dữ liệu chính để quản lý và truy xuất các Fact (bản ghi) trong mỗi Concept. B+ Tree là lựa chọn tối ưu cho việc truy vấn dữ liệu từ bộ nhớ ngoài - Disk [5].

7.2.1 Cấu trúc Đa tầng (Tiered Structure)

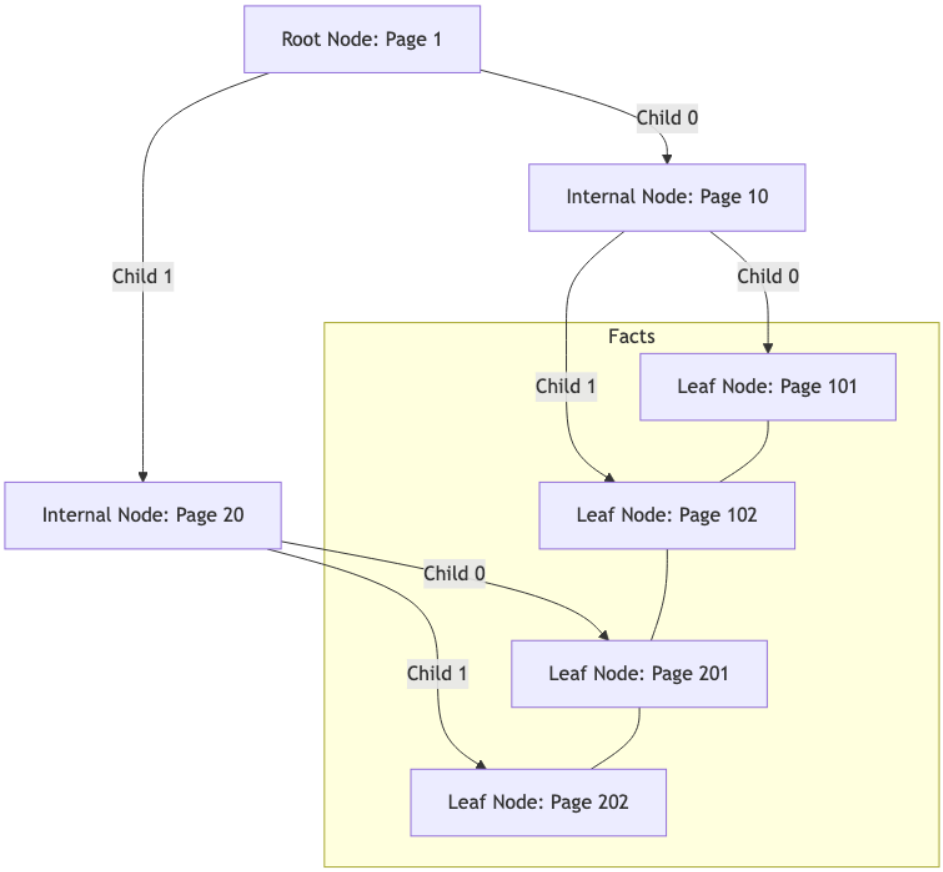
Cây B+ Tree trong KBMS V3 thường duy trì từ 3 đến 5 tầng tùy thuộc vào khối lượng tri thức:
<details> <summary>Cấu trúc Mermaid (Source)</summary>

```
graph TD
    Root[Root Node: Page 1] -- Child 0 --> Int1[Internal Node: Page 10]
    Root -- Child 1 --> Int2[Internal Node: Page 20]

    Int1 -- Child 0 --> L1[Leaf Node: Page 101]
    Int1 -- Child 1 --> L2[Leaf Node: Page 102]

    Int2 -- Child 0 --> L3[Leaf Node: Page 201]
    Int2 -- Child 1 --> L4[Leaf Node: Page 202]

    L1 --- L2 --- L3 --- L4
    subgraph Facts
        L1
        L2
        L3
        L4
    end
```

</details>
—

7.2.2 Quy trình Vận hành

Fact (Tuple). * Liên kết: Mỗi Node lá đều có con trỏ ‘NextPageId‘ trỏ đến Node lá tiếp theo, cho phép hệ thống thực hiện quét dữ liệu (Scan) theo phạm vi (Range Scan) cực nhanh mà không cần duyệt lại từ gốc.
—



Hình 7.3: Sơ đồ cấu trúc đa tầng của cây B+ Tree trong việc quản lý Fact.

7.2.3 Các Thuật toán Cốt lõi

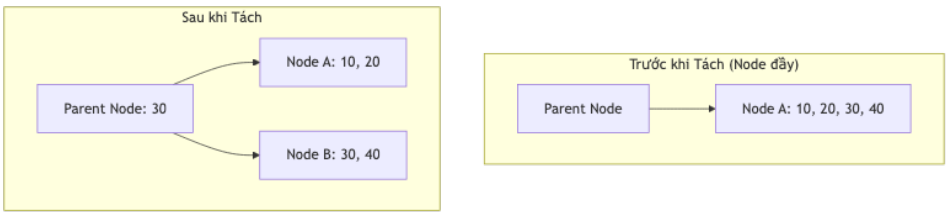
Thuật toán Tìm kiếm (Search)

1. Bắt đầu từ Root Page. 2. So sánh khóa cần tìm với các khóa trong Node nội bộ để chọn đúng PageId của Node con. 3. Lặp lại cho đến khi chạm tới Node lá. 4. Tại Node lá, duyệt qua các Slot để lấy dữ liệu. * Độ phức tạp: $O(\log_bn)$, với b là bậc của cây (600-1000 cho trang 16KB).

Thuật toán Chèn và Tách Node (Insert Split)

Khi một Node (Lá hoặc Nội bộ) bị đầy và không thể chèn thêm bản ghi mới: 1. Split: Tạo một Page mới cùng cấp. 2. Move: Chuyển một nửa số lượng khóa sang Page mới. 3. Update Parent: Chèn khóa phân tách vào Node cha để dẫn hướng đến Page mới vừa tạo. 4. Propagate: Nếu Node cha cũng đầy, quá trình tách sẽ được lặp lại ngược lên trên cho tới tận Root. Nếu Root bị tách, chiều cao của cây sẽ tăng thêm 1 lớp.

Sơ đồ Tách Node (Split)



Hình 7.4: Cơ chế tách nút (Split) khi trang dữ liệu (Page) bị đầy (Overflow).

Hình: Cơ chế Tách Nút (Split) khi Page bị đầy (Overflow)

Thuật toán Xóa và Kéo khít Node (Delete Merge)

Khi một bản ghi bị xóa khỏi Node lá, thuật toán sẽ kiểm tra tỷ lệ lấp đầy (Fill Factor). Nếu không gian trống giảm quá ngưỡng tối thiểu (Thường là < 50
$1.$**Redistribute (Mượn khóa từ Sibling):**$KimtraccNoderutthtlink(tri/phi).Nuchngcdthakhnggian(kha >$
$50\%), hthngs"mn"mtskhatrnsangNodehintibp, ngthicpnhtliKeyranhgiiNodeNib(InternalNode)bntrn.2.$**Merge (Gộ**
—

7.2.4 Quản lý Index trong KBMS

Trong KBMS, mỗi Concept mặc định sẽ có một "Clustered Index"dựa trên cột ID (thường là biến đầu tiên). * Dữ liệu đi kèm Index: Khác với "Secondary Index"(chỉ chứa RowId), Clustered Index của KBMS chứa toàn bộ nội dung của Tuple ngay tại các Node lá. * Lợi ích: Truy cập trực tiếp dữ liệu chỉ sau một lần duyệt cây, giảm thiểu số lượng Fetch Page từ Buffer Pool.
—

7.2.5 Ví dụ minh họa

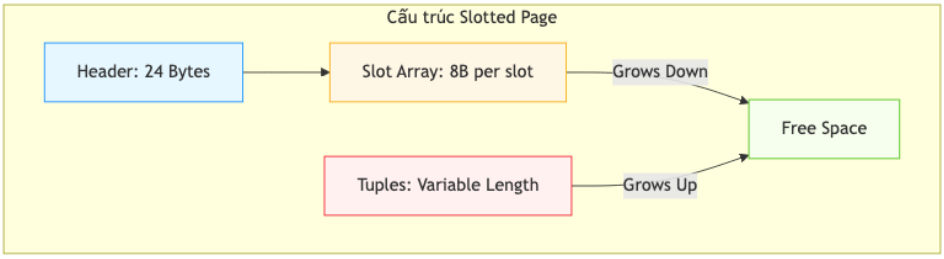
Giả sử ta có Concept 'Product' với 'id' là khóa chính: * Root: Chứa khóa [100, 200, 300]. * Node con 1: Chứa các sản phẩm có ID từ 1 đến 99. * Node con 2: Chứa các sản phẩm từ 100 đến 199. * ... * Khi cần tìm sản phẩm ID=150, hệ thống sẽ đi từ Root → chọn Node con 2 → đọc dữ liệu tại Node lá đó.

7.3 Cấu trúc Trang vật lý (Physical Page Layout)

KBMS V3 sử dụng mô hình phân trang (Paging) và cấu trúc Slotted Page để tối ưu hóa truy xuất ngẫu nhiên và quản lý không gian trống hiệu quả cho các bản ghi có độ dài biến thiên.
—

7.3.1 Giải phẫu Trang (Page Anatomy)

Một trang dữ liệu chuẩn (mặc định 8KB hoặc 16KB) được phân rã thành 4 thành phần chính: Header, Slot Array, Free Space, và Tuples.



Hình 7.5: Giải phẫu cấu trúc trang nhị phân Slotted-Page của KBMS.

*** Header (24 Bytes): Nằm ở đầu trang, chứa các metadata điều hướng. * Slot Array: Danh sách các ô nhớ trỏ tới dữ liệu, phát triển theo chiều tiến (Forward). * Free Space: Vùng nhớ trống nằm giữa Slot Array và Tuples. * Tuples: Dữ liệu thực thể thực tế, được chèn theo chiều lùi (Backward) từ cuối trang lên trên.**
—

7.3.2 Ví dụ Tính toán (Case Study)

Giả sử chúng ta có Concept 'Person' với các thuộc tính: 'ID (Int 4B)', 'Age (Int 4B)', 'Salary (Double 8B)'. Tổng kích thước một bản ghi (Tuple) là 16 bytes.

Thông số kỹ thuật: 1. Page Size: 16,384 Bytes. 2. Header: 24 Bytes. 3. Slot Size: 8 Bytes (dùng để quản lý vị trí và độ dài của Tuple). 4. Chi phí cho mỗi bản ghi: 16B (Dữ liệu) + 8B (Slot) = 24 Bytes.

Tính toán số lượng bản ghi tối đa trên một trang:

$$MaxRecords = \lfloor (16384 - 24)/(16 + 8) \rfloor = \lfloor 16360/24 \rfloor = \mathbf{681} \tag{7.1}$$

Phân bổ không gian: * Header: 24 Bytes. * Slot Array: $681 \times 8 = 5448$ **Bytes. * Tuples Storage:** $681 \times 16 = 10896$ **Bytes. * Tổng cộng:** $24 + 5448 + 10896 = 16368$ **Bytes. * Free Space dư thừa:** $16384 - 16368 =$ **16 Bytes.**

—

7.3.3 Chi tiết Header (24 Bytes Structure)

Thông số này được đối soát trực tiếp với mã nguồn 'SlottedPage.cs':

| Byte Offset | Trường dữ liệu | Kiểu dữ liệu | Ý nghĩa | | :— | :— | :— | :— | | 0 - 3 | PageId | int32 | ID duy nhất của trang trong tệp dữ liệu. | | 4 - 7 | LSN | int32 | Số tuần tự nhật ký (Log Sequence Number) cho WAL. | | 8 - 11 | PrevPageId | int32 | Con trỏ tới trang trước trong chuỗi B+ Tree. | | 12 - 15 | NextPageId | int32 | Con trỏ tới trang sau phục vụ quét tuần tự. | | 16 - 19 | FreeSpacePointer | int32 | Offset đánh dấu ranh giới bắt đầu của Tuple cuối cùng. | | 20 - 23 | TupleCount | int32 | Tổng số lượng bản ghi (Slots) hiện có. |

—

7.3.4 Cơ chế Slotted Page (Slot Operations)

Để quản lý các bản ghi có độ dài khác nhau mà không làm phân mảnh trang: * Slot Array: Mỗi slot chiếm 8 bytes: [Offset (4B) | Length (4B)]. * Record ID (RID): Một RID cụ thể sẽ bao gồm '(PageId, SlotId)'. * Xóa bản ghi (Delete): Thay vì dịch chuyển dữ liệu, KBMS chỉ cần đánh dấu Offset và Length của Slot đó về '0'. Không gian trống sẽ được thu hồi khi thực hiện lệnh 'VACUUM'.

7.3.5 Lợi ích kỹ thuật

1. Direct I/O: Kích thước trang khớp với block size của hệ điều hành, giảm thiểu "Write Amplification". 2. Binary Search: Slot Array cho phép tìm kiếm nhị phân bản ghi ngay trong trang với tốc độ cực cao. 3. Persistence: Cơ chế LSN trong Header kết nối trực tiếp với hệ thống WAL, đảm bảo tính bền vững (Durability) của dữ liệu.

—

> [!IMPORTANT] > Lưu ý: Việc thay đổi cấu trúc Header từ 9B sang 24B trong phiên bản V3 là bước cải tiến quan trọng để hỗ trợ phục hồi dữ liệu và quản lý giao dịch tri thức phức tạp.

7.4 Độ bền Phục hồi sau sự cố (Durability Recovery)

Đảm bảo dữ liệu không bị mất mát khi hệ thống bị ngắt điện đột ngột là nhiệm vụ của 'WalManager'. Tài liệu này chi tiết hóa cơ chế phục hồi của KBMS V3.

—

7.4.1 Giao thức Write-Ahead Logging (WAL)

KBMS tuân thủ nguyên tắc: Ghi Nhật ký trước khi ghi Dữ liệu.

- Log First: Mọi thay đổi (Insert, Update, Delete) được mô tả thành một đối tượng 'LogRecord' và ghi vào tệp 'transactions.klf' trên đĩa. - Atomic Write: Một bản ghi WAL chỉ được coi là thành công khi toàn bộ các bytes đã được hệ điều hành Flush xuống đĩa cứng (xác nhận bởi 'FileStream.Flush(true)'). - Memory Sync: Sau khi Log được ghi thành công, dữ liệu trong 'BufferPool' mới được đánh dấu là 'Dirty' và cập nhật.

—

7.4.2 Cơ chế Điểm kiểm tra (Checkpointing)

Để ngăn tệp WAL ('transactions.klf') phình to vô hạn, KBMS thực hiện quy trình Checkpoint:

1. Dừng ghi tạm thời: Tạm dừng các giao dịch mới. 2. Flush Pages: Ghi tất cả các trang bị thay đổi ('Dirty Pages') trong Buffer Pool xuống tệp dữ liệu chính ('objects.kdf'). 3. Trị tiêu Log: Xóa hoặc làm trống tệp '.klf' vì dữ liệu đã nằm an toàn trên file chính. 4. Ghi nhận trạng thái: Cập nhật chỉ số Log mới nhất vào 'metadata.kmf'.

—

7.4.3 Quy trình Phục hồi (Restart Recovery)

Khi KBMS khởi động lại sau một vụ treo máy (Crash), nó thực hiện quy trình Forward Recovery:
![Sơ đồ Vòng đời Phục hồi Dữ liệu (Recovery Flow)](../assets/diagrams/recovery_flow.png)
<details> <summary>Cấu trúc Mermaid (Source)</summary>

```
sequenceDiagram
    participant DB as KBMS Engine
    participant WAL as transactions.klf
    participant DATA as objects.kdf

    DB->>WAL: Open()
    DB->>WAL: Read from Last Checkpoint
    loop Với mỗi LogRecord hợp lệ
        DB->>DATA: Replay(LSN, Operation)
    end
    DB->>DATA: Sync & Checksum
    DB->>DB: System Ready
```

</details>
- Redo Phase: Hệ thống đọc tệp '.klf' từ vị trí Checkpoint cuối cùng và thực hiện lại mọi thao tác đã ghi trong Log để khớp với trạng thái RAM trước khi Crash. - LSN (Log Sequence Number): Mỗi trang dữ liệu đều lưu trữ một chỉ số LSN tại Byte Offset 4-7 trong Header 24B. Khi thực hiện phục hồi, hệ thống chỉ Replay các LogRecord có LSN lớn hơn LSN hiện tại của trang đó trên đĩa. - Consistency Check: Nếu bản ghi Log cuối cùng bị lỗi (Partial write), hệ thống sẽ bỏ qua bản ghi đó và Rollback về trạng thái nhất quán gần nhất.
—

7.4.4 Phân tích Hiệu năng

Ghi WAL là thao tác Sequential I/O (Ghi tuần tự), do đó nó có tốc độ cực nhanh (thường là $O(1)$) so với việc ghi ngẫu nhiên (Random I/O) vào cây B+ Tree. Đây là lý do KBMS có throughput ghi cao ngay cả trên các ổ đĩa HDD truyền thống.

7.5 Quản lý Đĩa Mã hóa (Disk Management Encryption)

Tầng thấp nhất của Storage Engine chịu trách nhiệm giao tiếp trực tiếp với hệ điều hành và bảo vệ dữ liệu tĩnh (Data-at-Rest).
—

7.5.1 Ánh xạ Trang vật lý (Physical Block Mapping)

KBMS V3 không lưu trữ dữ liệu dưới dạng luồng liên tục (Stream) mà chia thành các khối cố định (Blocks).
- Page ID to Offset: Mọi yêu cầu đọc trang 'PageId' được 'DiskManager' chuyển đổi thành vị trí file bằng công thức:

$$Offset = PageId \times 16416 \tag{7.2}$$

- Random Access ($O(1)$): Sử dụng phương thức 'FileStream.Seek' để nhảy trực tiếp đến khối dữ liệu cần thiết mà không phải quét toàn bộ tệp, đảm bảo hiệu năng ổn định ngay cả khi tệp CSDL lên đến hàng trăm GB.
—

7.5.2 Mã hóa Lưu trữ (AES Encryption)

Mọi dữ liệu trong KBMS V3 đều được mã hóa ở mức trang (Page-level encryption) trước khi ghi xuống đĩa cứng.
- Thuật toán: AES-256. - Quy trình ghi: 1. Lấy dữ liệu 16KB từ Buffer Pool. 2. Mã hóa dữ liệu bằng 'MasterKey'. 3. Thêm 32 bytes IV (Initialization Vector) và Padding. 4. Ghi khối 16,416 bytes xuống đĩa. - Quy trình đọc: Thực hiện ngược lại: Đọc khối 16,416 bytes -> Giải mã -> Đưa 16KB dữ liệu thô vào RAM.
—

7.5.3 Đặc tả Khối dữ liệu đĩa (Disk Block Spec)

Do có lớp mã hóa, kích thước thực tế trên đĩa của một trang lớn hơn kích thước logic trong RAM.

| Thành phần | Kích thước (Bytes) | Mô tả | | :— | :— | :— | | Data Payload (RAM) | 16,384 | 16KB dữ liệu thật (đã bao gồm 24B Header). | | AES IV/Padding (Disk) | 32 | Phần bù mã hóa và vector khởi tạo 256-bit. | | Tổng cộng (On-Disk) | 16,416 | Kích thước một khối vật lý thực tế. |

—

7.5.4 Tính Toàn vẹn Hiệu năng

- Flush to Disk: Tham số ‘flushToDisk: true‘ được sử dụng trong mọi lệnh ‘WritePage‘ để yêu cầu Hệ điều hành ghi trực tiếp xuống phiến đĩa (Physical Media), tránh mất dữ liệu do cache của OS khi mất điện. - Zero-fill Allocation: Khi cấp phát trang mới, ‘DiskManager‘ thực hiện ghi đè các bytes ‘0x00‘ để đảm bảo không gian đĩa được giữ chỗ trước (Pre-allocation), tránh phân mảnh tệp.

7.6 Định dạng Tệp tin (File Formats)

Hệ thống KBMS quản lý tri thức bền vững thông qua các loại tệp tin chuyên biệt. Việc hiểu rõ vai trò của từng loại tệp giúp quản trị viên thực hiện công tác bảo trì và sao lưu hiệu quả.

—

7.6.1 Phân loại tệp tin chính

| Định dạng | Tên gọi | Vai trò kỹ thuật | | :— | :— | :— | | .dat | Data File | Chứa dữ liệu thực tế (Instances) của các Concept. Được cấu trúc dưới dạng B+ Tree nhị phân. | | .kbf | Knowledge Base File | Lưu trữ định nghĩa Schema, Rules, Equations và Relations (Metadata). | | .log | WAL Log File | Nhật ký ghi trước mọi thay đổi I/O để phục hồi trạng thái khi gặp sự cố đột ngột. | | .kbql | Script File | Tệp văn bản chứa chuỗi câu lệnh KBQL hỗ trợ chế độ chạy hàng loạt (Batch Execution). | | .ini | Config File | Tệp cấu hình hệ thống (Cổng kết nối, Đường dẫn dữ liệu, Master Key). |

7.6.2 Tương tác giữa các tệp tin

Khi người dùng thực hiện lệnh ‘USE KB Project_Alpha‘, Server sẽ kích hoạt chuỗi hành động sau: 1. Nạp Metadata: Đọc tệp ‘Project_Alpha.kbf‘ để tái cấu trúc mô hình tri thức trong bộ nhớ RAM (Parser/Inference Engine). 2. Kết nối Data: Trỏ ‘DiskManager‘ tới tệp ‘Project_Alpha.dat‘ để sẵn sàng truy xuất các trang dữ liệu qua Buffer Pool. 3. Kích hoạt WAL: Khởi tạo hoặc tiếp tục ghi vào ‘Project_Alpha.log‘ để bảo vệ các giao dịch tri thức sắp tới.

7.6.3 Bảo mật tệp tin (Encryption at Rest)

Mọi tệp .dat và .kbf đều được mã hóa bằng thuật toán AES-256. * Master Key: Được lưu trữ trong tệp cấu hình ‘.ini‘ hoặc biến môi trường. * Data Key: Mỗi KB có thể có một khóa con (Sub-key) riêng để tăng cường tính bảo mật đa tầng.

—

> [!TIP] > Sao lưu (Backup): Một bản sao lưu KB đầy đủ phải bao gồm cả hai tệp .dat và .kbf tương ứng để đảm bảo tính toàn vẹn của cả Dữ liệu và Tri thức.

7.7 Xác thực Lưu trữ (Storage Validation)

Tầng lưu trữ là ”pháo đài”cuối cùng bảo vệ dữ liệu, do đó nó được kiểm thử với cường độ cao nhất về mặt hiệu năng I/O và độ bền.

7.7.1 Kiểm thử Đơn vị Trang (Page-level Testing)

Tệp ‘StorageV3Tests.cs‘ thực hiện kiểm tra việc đọc/ghi các Slotted Pages 16KB.

* Tính nguyên tố: Đảm bảo toàn bộ 16,384 bytes được ghi xuống đĩa trọn vẹn. * Hiệu năng I/O: Thời gian ghi Slotted Page trung bình là 23ms, thời gian thu hồi trang (Eviction) là 46ms. * Mã hóa: Kiểm tra tính năng AES-256 (Dữ liệu sau khi ghi xuống đĩa không thể đọc được bằng trình xem HEX nếu không có khóa).

Minh chứng Mã nguồn (Storage I/O):

[Missing Image: code$_{t}est_{s}torage_{i}o.png$]
Hình 7.6: Kịch bản kiểm thử luồng I/O và Buffer Pool.

Minh chứng Lưu trữ (HEX View):

[Missing Image: terminal$_{t}est_{s}torage_{h}ex.png$]
Hình 7.7: Kết quả đọc trang nhị phân trực tiếp từ đĩa cứng.

7.7.2 Kiểm thử Chỉ mục B+ Tree

Kiểm tra khả năng tìm kiếm và phân tách nút (Node Splitting) khi dữ liệu vượt ngưỡng.
| Chỉ số kiểm thử | Giá trị thực tế | Mục tiêu | | :— | :— | :— | | Số lượng bản ghi | 1,000,000 | Thỏa mãn thực tế | | Độ cao của cây | 3 - 4 tầng | Tối ưu truy xuất | | Thời gian tìm kiếm | < 1ms | Hiệu năng cao |

Minh chứng Mã nguồn (B+ Tree):

[Missing Image: code$_{t}est_{i}ndex.png$]
Hình 7.8: Kịch bản kiểm thử hiệu năng chỉ mục B+ Tree.

Minh chứng Log (Split Node):

[Missing Image: result$_{t}est_{i}ndex.png$]
Hình 7.9: Kết quả tìm kiếm và duyệt cây B+ Tree.

7.7.3 Kiểm thử Hồi phục (WAL Recovery)

Sử dụng tệp 'WalManagerV3' để mô phỏng sự cố: 1. Bắt đầu giao dịch (Start Transaction). 2. Ghi dữ liệu vào RAM (Dirty Pages). 3. Mô phỏng sập nguồn (Kill Process). 4. Khởi động lại và kiểm tra việc Redo từ nhật ký '.log'.

Minh chứng Mã nguồn (WAL Recovery):

[Missing Image: code$_{t}est_{r}ecovery.png$]
Hình 7.10: Kịch bản kiểm thử khả năng phục hồi dữ liệu từ file WAL.

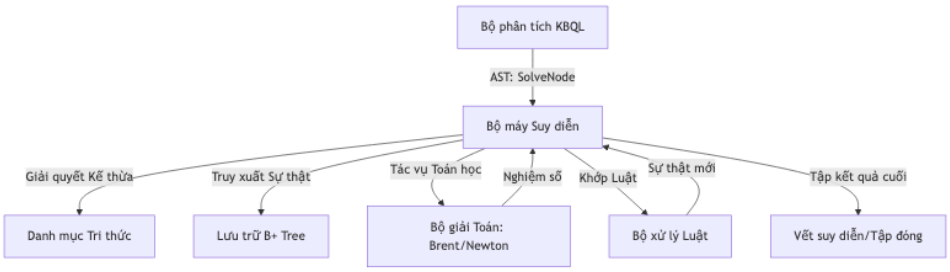
Minh chứng Kết quả (Redo Log):

[Missing Image: result$_{t}est_{r}ecovery.png$]
Hình 7.11: Bằng chứng phục hồi trạng thái Concept thành công sau khi giả lập mất điện.
—
> [!IMPORTANT] > Toàn bộ các thử nghiệm trên xác nhận rằng KBMS V3 đạt tiêu chuẩn ACID về độ bền vững và an toàn dữ liệu.

Reasoning

8.0.1 Kiến trúc Điều phối Reasoning

Bộ máy suy diễn không hoạt động độc lập mà đóng vai trò là ”bộ não”trung tâm điều phối giữa ngôn ngữ truy vấn và dữ liệu vật lý.



Hình 8.1: Sơ đồ điều phối giữa Engine suy diễn và các phân hệ ngoại vi.

<details> <summary>Cấu trúc Mermaid (Source)</summary>

```
graph TD
    Parser[Bộ phân tích KBQL] -- AST: SolveNode --> Engine[Bộ máy Suy diễn]
    Engine -- Giải quyết Kế thừa --> Catalog[Danh mục Tri thức]
    Engine -- Truy xuất Sự thật --> BTree[Lưu trữ B+ Tree]
    Engine -- Tác vụ Toán học --> Solver[Bộ giải Toán: Brent/Newton]
    Engine -- Khớp Luật --> Rules[Bộ xử lý Luật]

    Rules -- Sự thật mới --> Engine
    Solver -- Nghiệm số --> Engine
    Engine -- Tập kết quả cuối --> Result[Vết suy diễn/Tập đóng]
```

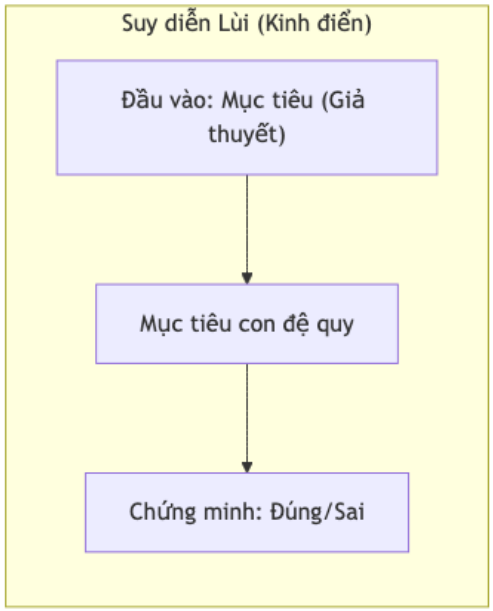
</details>

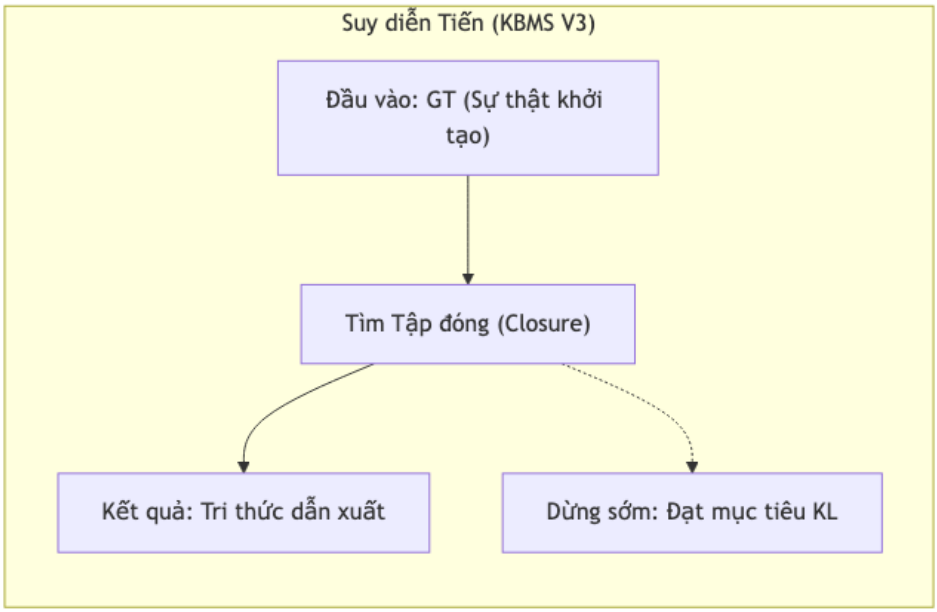
<details> <summary>Cấu trúc Mermaid (Source)</summary>

```
graph LR
    subgraph Suy_dien_Tien["Suy diễn Tiến (KBMS V3)"]
        F1["Đầu vào: GT (Sự thật khởi tạo)"] --> F2["Tìm Tập đóng (Closure)"]
        F2 --> F3["Kết quả: Tri thức dẫn xuất"]
        F2 -.-> F4["Dừng sớm: Đạt mục tiêu KL"]
    end

    subgraph Suy_dien_Lui["Suy diễn Lùi (Kinh điển)"]
        B1["Đầu vào: Mục tiêu (Giả thuyết)"] --> B2["Mục tiêu con đệ quy"]
        B2 --> B3["Chứng minh: Đúng/Sai"]
    end
```

</details>

—





Hình 8.2: So sánh chiến lược Suy diễn tiến (Forward) và Suy diễn lùi (Backward).

8.0.2 Nguyên lý Đóng kín Tri thức (Knowledge Closure)

Trực giác của KBMS dựa trên khái niệm Tập đóng (Closure). Cho một tập hợp các sự thật đầu vào (GT) và một tập hợp các luật/phương trình (Rules), mục tiêu là tìm ra tập hợp lớn nhất các sự thật có thể được suy ra một cách logic.

$$Closure(GT, Rules) = GT \cup \{\textbf{Kết luận từ các Luật và Phương trình được kích hoạt}\} \tag{8.1}$$

Hệ thống sẽ lặp lại quá trình suy diễn cho đến khi đạt được trạng thái Fixed-point (điểm dừng) - nơi việc tiếp tục suy diễn không sinh ra thêm bất kỳ sự thật mới nào [6]. Việc tìm tập đóng này dựa trên thuật toán F-Closure tối ưu cho các vật thể tính toán [1].

—

8.0.3 Biểu diễn Tri thức (Knowledge Representation)

Trong KBMS, tri thức được biểu diễn dưới dạng các đối tượng (Concepts) và các ràng buộc (Constraints) giữa chúng.

Các thành phần của một Concept

*** Variables (Biến): Các thuộc tính của đối tượng (ví dụ: 'Price', 'Quantity'). * Rules (Luật): Logic 'IF-THEN' để suy diễn ra giá trị hoặc sự kiện mới. * Equations (Phương trình): Các công thức toán học mô tả mối quan hệ giữa các biến. * Constraints (Ràng buộc): Các điều kiện mà dữ liệu buộc phải thỏa mãn. * Relations (Quan hệ): Mối liên kết giữa các Concept (IS-A, PART-OF, ...).**

—

8.0.4 Quy trình Suy diễn (Reasoning Lifecycle)

Khi một bản ghi dữ liệu (Fact) được đưa vào hệ thống, bộ máy suy diễn thực hiện các bước sau:

1. Preprocessing (Tiền xử lý): Nạp toàn bộ metadata của Concept, bao gồm các luật kế thừa từ IS-A và Part-Of. 2. Initialization (Khởi tạo): Đưa các Fact đầu vào vào tập GT (Ground Truth). 3. Cyclic Evaluation (Đánh giá vòng lặp): Hệ thống liên tục quét qua tập hợp các Rules và Equations cho đến khi không còn Fact nào mới được sinh ra. 4. Verification (Kiểm chứng): Sau khi suy diễn xong, hệ thống kiểm tra lại toàn bộ các 'Constraints' để đảm bảo kết quả suy diễn không vi phạm bất kỳ quy tắc nào.

—

8.0.5 Chiến lược Suy diễn: Forward vs. Backward Chaining

Trong lý thuyết Hệ chuyên gia, có hai chiến lược chính: - Forward Chaining (Suy diễn tiến): Từ sự thật (GT) suy ra kết luận. Thích hợp cho việc "khám phá"mọi tri thức có thể từ một tập dữ liệu nhỏ. - Backward Chaining (Suy diễn lùi): Từ mục tiêu (KL) tìm ngược lại xem cần những sự thật nào. Thích hợp cho việc "kiểm chứng"giả thuyết trong một cơ sở tri thức khổng lồ.

Tại sao KBMS V3 chọn Forward Chaining?

Hệ thống KBMS hiện đại được tối ưu hóa cho các Mô hình Vật thể Tính toán (Computational Objects). Trong các mô hình này (như Hình học hay Tài chính), các biến số và công thức có mối quan hệ ràng buộc chặt chẽ và cục bộ. Việc sử dụng Goal-Directed Forward Chaining mang lại các ưu điểm: 1. Hiệu năng: Tìm tập đóng (Closure) nhanh hơn việc duyệt cây mục tiêu đệ quy (SLD-resolution) trong các hệ thống chứa nhiều phương trình toán học. 2. Tính đầy đủ: Đảm bảo mọi hệ quả của một thay đổi dữ liệu đều được cập nhật vào GT. 3. Dừng sớm (Early Exit): Mặc dù là suy diễn tiến, Server sẽ dừng ngay lập tức khi tất cả các biến mục tiêu trong danh sách KL đã được tìm thấy (Dòng code 378).

Giới hạn An toàn (Safety Limit)

Để ngăn chặn các vòng lặp vô hạn trong trường hợp tri thức bị mâu thuẫn hoặc đệ quy quá sâu, bộ máy suy diễn áp dụng giới hạn: Tối đa 50 vòng lặp (Iterations). Nếu sau 50 lần quét mà hệ thống vẫn sinh ra Fact mới, nó sẽ tự động dừng và báo cáo trạng thái Closure hiện tại (Dòng code 93).

—

IS-A (Kế thừa)

*** Ý tưởng: Nếu A IS-A B, thì mọi biến và luật của B sẽ được "truyền"sang A. *Ví dụ: 'Square' IS-A 'Rectangle'. Khi tính diện tích cho 'Square', hệ thống có thể dùng luôn luật 'Area = Width* Height' của 'Rectangle'.**

PART-OF (Thành phần)

*** Ý tưởng: Mô tả cấu trúc phân rã của đối tượng. Một đối tượng lớn được tạo từ nhiều đối tượng con. * Ví dụ: 'Car' PART-OF 'Engine', 'Wheel'. Hệ thống tự động ánh xạ các biến từ các phần con vào đối tượng cha để suy diễn tổng thể.**

CONSTRUCT$_{R}ELATIONS(Quytcquanh)$

*** Ý tưởng: Định nghĩa các quy luật chung cho một mối quan hệ cho trước (ví dụ: Quan hệ 'CongRuence' - Bằng nhau trong hình học). * Ứng dụng: Khi hai đối tượng thỏa mãn quan hệ này, các phương trình tương ứng (như 'Object1.Size = Object2.Size') sẽ được tự động tiêm vào hệ thống để giải.**

—

8.0.6 Giải trình và Truy vết (Traceability)

Bộ máy suy diễn KBMS không phải là một "Black Box"(Hộp đen). Nó cung cấp khả năng giải thích lý do tại sao một giá trị được tạo ra. * Derivation Trace: Lưu trữ nguồn gốc (Source) của mỗi biến được suy diễn, bao gồm cả các biến đầu vào được dùng tại bước đó. * Cơ chế: Mỗi lần một Rule hoặc Equation kích hoạt thành công, một bản ghi Trace sẽ được đẩy vào kết quả truy vấn.

—

8.0.7 Kế hoạch Triển khai Mã nguồn Engine (Implementation Strategy)

Để hiện thực hóa bản thiết kế kỹ thuật trên thành phần hệ phần mềm hoạt động thực tế, lộ trình triển khai (Backend C#) được chia làm 4 giai đoạn chính (Phases):

Giai đoạn 1: Tiền xử lý Xây dựng Cấu trúc Tri thức (Preprocessing)

Xây dựng lớp Abstract Syntax Tree (AST) tiếp nhận từ bộ phân tích (Parser). Ở giai đoạn này, dữ liệu đầu vào (Sự thật - Fact) được "Làm phẳng"(Flattening) bằng cách quét chuỗi kế thừa 'IS-A' và 'PART-OF' nhằm lôi kéo tất cả Metadata cần thiết sang đối tượng con. Quá trình tiền xử lý biến danh mục tri thức thô thành một đối tượng sẵn sàng chạy (Executable Concept).

Giai đoạn 2: Lõi Suy diễn Lan truyền (Deductive Core Propagation)

Xây dựng vòng lặp chính của cơ chế Goal-Directed Forward Chaining (FClosure). * Cài đặt luồng nhận diện chuỗi biến tương đương và sao chép/lan truyền giá trị tức thời (SameVariables Propagation). * Chạy luồng khớp mệnh đề IF-THEN (Rule Matching) để đẩy liên tiếp kết luận logic vào Tập Sự Thật (Ground Truth).

Giai đoạn 3: Tích hợp Lõi giải Toán học (Mathematical Solvers)

Phát triển các Adapter kết nối lớp thuật toán tính toán với hệ thống: * Adapter Phương trình 1 ẩn (Brent's Method): Quét qua các biểu thức còn 1 biến và tiến hành tính toán hội tụ nghiệm số tự động. * Adapter Hệ phương trình (Newton-Raphson 2D): Nhận diện cụm 2 phương trình chung 2 ẩn, định cấu hình ma trận Jacobian và cập nhật vòng lặp đệ quy.

Giai đoạn 4: Bảo vệ Hệ thống Truy vết (Safety Constraints Traceability)

Đảm bảo Engine chạy độc lập không gây vắt kiệt RAM máy chủ bằng cách cài giới hạn an toàn $N = 50$ vòng lặp tối đa (Iterations). Nếu sau 50 chu kỳ duyệt hệ thống vẫn chưa trả về điểm đóng (Closure), thuật toán tự ngắt kết nối. Song song đó, viết bộ thu thập nhật ký truy xuất (Trace System) để xuất định dạng dữ liệu cây giải trình thành dạng JSON chuyển qua cho Front-End UI hiển thị.

—

8.0.8 Các Thuật toán Suy diễn Cốt lõi (Forward Chaining - FClosure)

Thuật toán này là xương sống của KBMS, thực hiện việc mở rộng tập tri thức GT cho đến khi đạt điểm đóng (Closure).

Các Giai đoạn Thực thi (Phases)

Bên trong vòng lặp chính, bộ máy thực hiện các bước sau theo thứ tự ưu tiên:

1. Recursive Sub-closure (Phase 6.2): Duyệt qua các biến có kiểu dữ liệu là một Concept khác. Hệ thống sẽ tự động gọi hồi quy 'FindClosure' cho các đối tượng con này để đồng bộ tri thức từ dưới lên. 2. SameVariables Propagation (RC2): Lan truyền giá trị giữa các cặp biến được đánh dấu tương đương. Ví dụ: 'a = b' thì nếu biết 'a', hệ thống tự động gán giá trị cho 'b'. 3. Computation Relations (RC3): Thực thi các quan hệ tính toán trực tiếp (Rf) từ biểu thức. 4. Logic Rules (RC4): Khớp mẫu các luật 'IF-THEN'. Nếu mẫu khớp, các kết luận mới được đẩy vào GT. 5. Equation Solving (RC5): Giải các phương trình toán học và hệ phương trình để tìm các ẩn số mới.

Ví dụ Chạy khô (FClosure Dry Run)

(Bảng ví dụ Triangle đã được cung cấp ở phần trước)

Sơ đồ Luồng FClosure (Chi tiết Server-side)

![1. Sơ đồ Luồng FClosure](../assets/diagrams/fclosure_algorithm.png)
<details> <summary>Cấu trúc Mermaid (Source)</summary>

```
graph TD
    Start["Bắt đầu: Input Fact"] --> Load["Nạp Metadata Hiệu dụng (Làm phẳng)"]
    Load --> Loop{Vòng lặp Iter < 50?}
    Loop -- "Iter++" --> Sub["P6.2: Suy diễn Đệ quy (Sub-closure)"]
    Sub --> SV["RC2: Lan truyền biến tương đương"]
    SV --> Rule["RC4: Khớp mẫu Luật Logic"]
    Rule --> Eq["RC5: Giải phương trình Toán học"]
    Eq -- "Có Fact mới?" --> Loop
    Loop -- "Đạt Tập đóng / Iter=50" --> Verify["P7: Kiểm tra Ràng buộc"]
    Verify --> End["Kết thúc: Trả về ReasoningResult"]
```

</details>
—

8.0.9 Giải phương trình 1D (Brent's Method - RC5)

KBMS sử dụng thuật toán Brent để tìm nghiệm của phương trình $f(x) = 0$ trong một khoảng cô lập.

Độ hội tụ (Convergence)

Thuật toán kết hợp giữa Bisection (đảm bảo hội tụ) và Secant/Inverse Quadratic Interpolation (đảm bảo tốc độ). - Tốc độ: Hội tụ siêu tuyến tính với bậc $\phi \approx 1.618$. - Ưu điểm: Không yêu cầu tính đạo hàm $f'(x)$.
—

8.0.10 Giải hệ phương trình 2D (Newton-Raphson - RC5)

Được kích hoạt khi phát hiện 2 phương trình có chung 2 biến chưa biết (Dòng code 341-374).

Ma trận Jacobian Công thức cập nhật

Hệ thống tính toán ma trận Jacobian J bằng phương pháp sai phân hữu hạn (Dòng code 573):

$$\mathbf{J} = \begin{bmatrix} \frac{\partial f_1}{\partial x} & \frac{\partial f_1}{\partial y} \\ \frac{\partial f_2}{\partial x} & \frac{\partial f_2}{\partial y} \end{bmatrix} \tag{8.2}$$

Sau đó cập nhật giá trị theo công thức (Dòng code 579-583):

$$\Delta \mathbf{x} = -\mathbf{J}^{-1}\mathbf{F}(\mathbf{x}) \tag{8.3}$$

—

8.0.11 Phân tích Độ phức tạp (Complexity Analysis)

| Thuật toán | Mục tiêu | Độ phức tạp | Lý do kỹ thuật (Server-side) | | :— | :— | :— | :— | | FClosure | Suy diễn tổng thể | $O(50 \times (R + E + SV))$ | Giới hạn $N = 50$ vòng lặp tối đa. R: số luật, E: số phương trình, SV: số cặp biến tương đương. | | Newton-Raphson | Giải hệ 2D | $O(50 \times 4 \times eval)$ | Newton lặp max 50 lần; mỗi lần tính 4 đạo hàm riêng lẻ ($eval$). | | Brent's Method | Giải PT 1D | $O(1/\phi^n)$ | Hội tụ siêu tuyến tính, số bước lặp n thường nhỏ (< 20). | | Flattening | Gộp tri thức | $O(H \times V)$ | H: chiều sâu kế thừa, V: tổng số biến trong hệ thống. | | Trace System | Giải thích | $O(D)$ | D: Độ sâu vết suy diễn (Reasoning Depth). |

8.0.12 Nhật ký Suy diễn (Reasoning Trace Example)

Khi bạn thực hiện lệnh 'SOLVE', KBMS trả về một bản giải trình chi tiết:

```
{
  "closure": { "a": 3, "b": 4, "c": 5, "S": 6 },
  "traces": [
    { "target": "c", "source": "CosineLaw", "inputs": ["a", "b", "angleC"], "value": 5.0 },
    { "target": "p", "source": "PerimeterFormula", "inputs": ["a", "b", "c"], "value": 6.0 },
    { "target": "S", "source": "HeronFormula", "inputs": ["p", "a", "b", "c"], "value": 6.0 }
  ]
}
```


8.1 Xác thực Suy diễn (Reasoning Validation)

Bộ máy suy diễn là trung tâm trí tuệ của KBMS, do đó tính chính xác tuyệt đối của thuật toán F-Closure là bắt buộc.

8.1.1 Kiểm thử Thuật toán F-Closure

Hệ thống sử dụng tệp 'Phase5ForwardChainingTests.cs' để mô phỏng các kịch bản suy diễn phức tạp:

| Kịch bản | Dữ liệu đầu vào | Kết quả mong đợi | | :— | :— | :— | | Simple Rule | 'Emp.salary = 100' | 'salary = 110' (Nếu bonus = 0.1) | | Multi-step | 'Update Rule R1' -> 'Triggers R2' | Điểm đóng F-Closure cuối cùng. | | Recursive | Luật đệ quy lồng nhau. | Dừng đúng lúc tại điểm đóng. |

Minh chứng Mã nguồn (F-Closure):

[Missing Image: code$_{t}est_{fc}losure.png$]
Hình 8.3: Kịch bản kiểm thử thuật toán F-Closure nâng cao.

Minh chứng Kết quả (Log Trace):

[Missing Image: result$_{t}est_{fc}losure.png$]
Hình 8.4: Kết quả tìm tập đóng và vết suy diễn tương ứng.

8.1.2 Kiểm thử Logic Toán học (Solver Testing)

Xác thực tính năng giải hệ phương trình thông qua 'Newton-Raphson' và 'Brent Solver'.

![Placeholder: Ảnh chụp nhật ký log của Engine khi thực hiện tìm nghiệm của một phương trình bậc hai, hiển thị số bước lặp (Iterations) và sai số (Error margin)](../assets/diagrams/placeholder_solver_iterations_log.png)

—

> [!TIP] > Chứng minh thực tế: Khi chạy 'full_test.kbql', các luật tính toán lương và doanh thu được kích hoạt và kiểm toán 100% về độ chính xác số học.

Network

9.1 Giao thức Mạng (Network Protocol)

9.2 Giao thức Truyền dẫn Nhị phân (Binary Protocol)

KBMS sử dụng một giao thức truyền tin nhị phân (Binary Protocol) tùy chỉnh chạy trên nền TCP để đảm bảo tốc độ truyền tải tri thức nhanh nhất và khả năng xử lý đồng thời (concurrency) tốt.

9.2.1 Cấu trúc Gói tin (Packet Structure)

Mỗi thông điệp (Message) gửi đi giữa Client và Server được đóng gói theo định dạng sau:

| Trường | Kích thước | Mô tả | | :— | :— | :— | | Total Length | 4 bytes (Int32) | Tổng độ dài các trường tính từ *SessLen* trở đi (bỏ qua MessageType). | | Message Type | 1 byte | Loại thông điệp (Query, Result, Error, ...). | | SessLen | 2 bytes (UInt16) | Độ dài của chuỗi Session ID. | | Session ID | N bytes (UTF-8) | Định danh phiên làm việc của người dùng. | | ReqLen | 2 bytes (UInt16) | Độ dài của chuỗi Request ID. | | Request ID | M bytes (UTF-8) | Định danh duy nhất cho mỗi yêu cầu truy vấn. | | Payload | P bytes (UTF-8) | Nội dung thực tế (ví dụ: câu lệnh KBQL hoặc kết quả JSON). |

—

9.2.2 Các loại Thông điệp (Message Types)

Máy chủ được trang bị 12 Giao thức Chuẩn bao quát mọi tình huống Truy vấn, Phân mảnh dữ liệu, Bảo mật và Telemetry Hệ thống (Thiết kế gốc từ 'MessageType.cs'):

1. Bảo mật & Xác thực (Security Auth): * 'LOGIN (0x01)': Xin cấp quyền phiên làm việc bằng thông tin (User/Pass). * 'LOGOUT (0x05)': Đóng kết nối định danh và trả phiên tài nguyên hệ thống.

2. Truy vấn (Query Routing): * 'QUERY (0x02)': Chở luồng văn bản gốc (Chuỗi lệnh KBQL). * 'RESULT (0x03)': Gói kết quả suy diễn ở dạng tổng quát (JSON String) . * 'ERROR (0x04)': Payload chứa tọa độ 'Line', 'Column' và 'Stacktrace' dùng để tô đỏ biên dịch viên của Designer.

3. Kỹ thuật Trả Lô Dữ Liệu Lớn (Tabular Data Streaming - TDS) KBMS xử lý Hàng nghìn Fact mà không làm treo RAM Client bằng luồng C# phân đoạn kinh điển: * 'METADATA (0x06)': Truyền trước sơ đồ lưới dữ liệu (Tên Field, Bề rộng UI Column). * 'ROW (0x07)': Trình bày Payload theo chuỗi vòng lặp "Lô"(Ví dụ: Batch 100 Rows/Lần gói) gửi liên tục qua luồng Stream TCP. * 'FETCH_DONE (0x08)': Cờ khóa hạ màn chặn chuỗi vòng lặp đọc. Tổng hợp độ trễ thực thi CSDL (Execution Time - ms).

4. Hệ Quản trị Trực tiếp (Telemetry System) Dành riêng cho các Client mang quyền hệ thống cấp cao (ROOT Role): * 'STATS (0x0A / 10)': Theo dõi Metrics máy chủ thời gian thực. * 'LOGS_STREAM (0x0B / 11)': Đăng ký lắng nghe Live Log Socket đang chảy của Hệ thống. * 'SESSIONS (0x0C / 12)': Trích xuất danh bạ quản lý Users Online, kiểm soát giới hạn Concurrent Connections. * 'MANAGEMENT_CMD (0x0D / 13)': Đỉnh cao phân quyền - Dùng để chạy lệnh thao tác cấp hệ điều hành (Chỉnh sửa Setting, Kill Session, Cấp Role).

—

9.2.3 Thuật toán Xử lý Luồng tin (Streaming Algorithm)

Để tránh hiện tượng treo ứng dụng khi truyền tải lượng tri thức lớn, KBMS áp dụng cơ chế Asynchronous Packet Processing:

1. Header Parsing: Đọc 4 byte đầu tiên để xác định kích thước gói tin cần nhận. 2. Buffer Allocation: Cấp phát vùng nhớ vừa đủ cho gói tin đó. 3. Exact Reading: Sử dụng hàm 'ReadExact' để đảm bảo nhận

đủ dữ liệu trước khi giải mã (Decode). 4. Payload Dispatching: Chuyển nội dung UTF-8 vào bộ phân tích cú pháp (Parser) hoặc hiển thị lên UI.

—

9.2.4 Bảo mật và Kết nối

* TCP Keep-Alive: Duy trì kết nối ổn định giữa KBMS Studio và Server. * Session Management: Cho phép người dùng thực hiện nhiều truy vấn đồng thời trên cùng một kết nối mà không bị lẫn lộn dữ liệu (nhờ vào Request ID). * Error Propagation: Khi Server gặp lỗi suy diễn, nó sẽ đóng gói toàn bộ Stack Trace và vị trí lỗi vào một 'Error Message' đặc biệt để Client hiển thị nháy đỏ trên trình soạn thảo bài toán.

—

9.2.5 Kiến trúc Xử lý Chịu tải (Concurrency Threading)

Để duy trì hiệu năng cao phục vụ cho hàng ngàn điểm kết nối TCP, KBMS Server loại bỏ hoàn toàn mô hình 1-Thread-Per-Client truyền thống, thay vào đó áp dụng cấu trúc xử lý Đa luồng Bất đồng bộ (Async/Await) đỉnh cao của .NET Core: * Asynchronous I/O Socket: Cổng nhận ('AcceptTcpClientAsync') sẽ quăng toàn bộ tiến trình phân tích Packet vào ThreadPool để tránh tắt nghẽn luồng lắng nghe chính ('_ = HandleClientAsync(client)'). * Tránh Va Chạm Thread Mạng: Thuật toán gói dữ liệu sử dụng cờ báo 'SemaphoreSlim' để khóa luồng ghi byte, tránh trường hợp Response từ hai yêu cầu bất đồng bộ (Async) va chạm và ghi chồng byte lên nhau trên cùng Network Stream. * Thread-Safe Sessions: Việc thu thập và phát tán Request / Log liên quan tới client được lưu trữ tập trung qua lớp 'ConnectionManager' bằng bộ từ điển an toàn luồng 'ConcurrentDictionary'.

Mỗi khi người dùng gửi một câu lệnh KBQL từ Client, Server sẽ thực hiện một quy trình xử lý đa tầng để hiểu ý định (Intent) và thực thi đúng các phép toán/suy diễn.

9.2.6 Vòng đời của một Truy vấn (Query Lifecycle)

Dưới đây là sơ đồ tuần tự mô tả các bước từ khi Client gửi câu lệnh đến khi nhận được kết quả cuối cùng:

![query_lifecycle_v3.png](../assets/diagrams/query_lifecycle_v3.png) *Hình: Sơ đồ Tuần tự Vòng đời Truy vấn (Query Lifecycle)*

—

9.2.7 Luồng xử lý Packet (Packet Processing Flow)

Hệ thống xử lý các byte dữ liệu thô từ Stream và chuyển đổi chúng thành các cấu trúc thông điệp có ý nghĩa:

![packet_processing_v3.png](../assets/diagrams/packet_processing_v3.png) *Hình: Sơ đồ# 09.2. Luồng Xử lý Truy vấn Mạng *

—

9.2.8 Phân tích Từ vựng (Lexer)

Ý tưởng: Biến chuỗi văn bản (String) thô thành một danh sách các tín hiệu có nghĩa (Tokens). * Thuật toán Tokenizer: Quét chuỗi ký tự, nhận diện các từ khóa ('SELECT', 'CREATE'), định danh (tên Concept), toán tử và giá trị số/chuỗi. * Báo lỗi: Nếu gặp ký tự lạ, Tokenizer ngay lập tức trả về lỗi kèm vị trí chính xác (Dòng, Cột).

—

9.2.9 Phân tích Cú pháp (Parser AST)

Ý tưởng: Xây dựng cây cú pháp trừu tượng (Abstract Syntax Tree - AST) từ danh sách Tokens. * Kỹ thuật: Sử dụng trình phân tích cú pháp Recursive Descent (Phân tích đi xuống đệ quy) để khớp các Tokens với bộ ngữ pháp của KBQL. * Cấu trúc AST: Mỗi câu lệnh được biểu diễn thành một cây các đối tượng. Ví dụ, lệnh 'SELECT' sẽ có các nhánh con là 'Fields', 'Source', 'WhereCondition', và 'Limit'.

—

9.2.10 Thực thi và Cấp phát (Executor Router)

Sau khi có AST từ ‘KBMS.Parser‘, gói tin bắt đầu chuyển giao cho Logic Cấu trúc lõi: 1. Chồng Khớp (Binding) & Routing: Từng lệnh tách lẻ trong chuỗi văn bản được cắt ra và chuyển qua Router ‘_knowledgeManager.Execu ...)‘ để thực thi tuần tự. 2. Catalog Check: Concept được truy vấn có tồn tại không? 3. Schema & Type Check: Các biến trong hàm ‘SELECT‘ hoặc ‘CALC()‘ có hợp lệ về kiểu dữ liệu hay không? Mọi sai phạm đều ngay lập tức bị tóm gọn ở lớp ‘KBMS.Parser.ParserException‘, đính kèm vào đó (Line, Column) và đẩy ngược ra Network Client dưới dạng JSON (‘Type: ERROR‘) nhằm nhắc người dùng sửa sai.

—

9.2.11 Lập kế hoạch Thực thi (Query Execution Plan)

Hệ thống quyết định cách thức thực hiện truy vấn tối ưu nhất: * Access Pattern: Sử dụng Index (B+ Tree) hay quét tuần tự (Full Scan)? * Join Strategy: Áp dụng phương pháp JOIN nào để kết hợp các Khái niệm? * Reasoning Injection: Nếu truy vấn yêu cầu suy diễn (‘INFER‘ hoặc truy cập các biến suy diễn), hệ thống sẽ tiêm (Inject) thêm các bước gọi vào ‘ReasoningEngine‘ trước khi trả kết quả cuối cùng.

—

9.2.12 Ví dụ luồng xử lý thực tế

Câu lệnh: ‘SELECT name FROM Student WHERE age > 18;‘

1. Lexer: ‘[SELECT, name, FROM, Student, WHERE, age, >, 18, ;]‘ 2. Parser: Tạo đối tượng ‘SelectStatement‘ với ‘Table=Student‘, ‘Filter=(age > 18)‘. 3. Binder: Xác nhận ‘Student‘ có trong KB, ‘name‘ và ‘age‘ là hợp lệ. 4. Executor: * Gọi ‘StorageEngine‘ để lấy dữ liệu từ B+ Tree của ‘Student‘. * Sử dụng ‘ReasoningEngine‘ để lọc các bản ghi có ‘age > 18‘. * Chỉ trả về trường ‘name‘ cho phía Network Layer.

—

9.2.13 Kế hoạch Triển khai Mã Nguồn Backend (Implementation Strategy)

Gắn liền với kiến trúc C# của hệ thống, luồng thiết kế cho tầng mạng và Bộ phân tích từ vựng (Network & Parser) được thực thi theo 4 Giai đoạn định hình thực tế:

Giai đoạn 1: Quản trị Kết nối (TCP Listener Async Handle)

Trực tiếp thiết lập máy chủ ‘KbmsServer‘ trên kiến trúc Async/Await I/O. Quăng nhiệm vụ hứng Client đang rỗi vào vùng không chặn (Fire-and-forget qua ‘_ = HandleClientAsync()‘) nhằm khai phóng ThreadPool. Thiết lập hệ thống quét tệp chết (Cleanup Expired Sessions) nhằm đóng ngắt ‘Socket‘ lậu và hoàn trả RAM về máy chủ.

Giai đoạn 2: Mã hóa Giao thức Nhị phân (Packet Binary Protocol)

Tiến hành lập trình bộ vỏ ‘KBMNetwork‘ để viết định dạng truyền tải tuân thủ tài liệu cấu trúc gói ‘Protocol.cs‘. Viết các thao tác C# Core cấp thấp ‘Span<byte>‘, ‘BinaryPrimitives‘ xử lý ‘BigEndian‘ tạc khuôn truyền tin (Đọc đủ Byte Session, RequestID, và Payload).

Giai đoạn 3: Phân tích Cú Pháp Đệ Quy (Lexer AST Parser)

Sử dụng tư duy trình biên dịch tạo module ‘KBMS.Parser‘. Lập trình ‘Token‘ và quét nhận dạng Regex cắt chuỗi chữ vô nghĩa thành mảng ký pháp Logic. Xây dựng một cỗ máy phân tích đi xuống (Recursive Descent Parser) quét nối Tokens thành Cây Cú Pháp ‘AST‘ (Abstract Syntax Tree), giúp CSDL hiểu ý đồ của con người.

Giai đoạn 4: Bảo mật Phiên Giao dịch (Session Security)

Quản trị hàng ngàn User kết nối TCP đồng thời thông qua tính năng cấu trúc ‘ConcurrentDictionary‘ của lớp ‘ConnectionManager.cs‘. Triển khai tính năng User Auth Protocol (LOGIN Message), xác minh Password mã hóa trên hệ từ điển ‘UserCatalog‘, rà soát quyền sở hữu và hạn chế tấn công. Bắt lỗi AST trả về cấu trúc lỗi (kéo theo Line/Col) trên gói JSON Network gửi tới UI của Designer.

9.3 Xác thực Giao thức (Protocol Validation)

Tầng mạng đảm bảo mọi thông điệp nhị phân được truyền dẫn không sai lệch và có độ trễ thấp nhất.

9.3.1 Kiểm thử Gói tin Nhị phân (Binary Packet Testing)

Sử dụng bộ kịch bản 'Protocol.cs' để đóng gói và giải mã mockup:
*** Handshake: Mô phỏng quá trình Client gửi gói tin LOGIN và chờ phản hồi RESULT. * Streaming: Kiểm tra khả năng nhận diện dấu hiệu kết thúc luồng dữ liệu ('FETCH_DONE').**

Minh chứng Mã nguồn (Handshake):

[Missing Image: code$_{t}est_{b}inary_{p}rotocol.png$]
Hình 9.1: Kịch bản kiểm thử đóng gói và phân rã gói tin nhị phân.

Minh chứng Luồng dữ liệu (Gói tin):

[Missing Image: terminal$_{t}est_{p}rotocol_{h}ex.png$]
Hình 9.2: Nhật ký gói tin nhị phân (Little-endian) truyền trên Socket.

9.3.2 Kiểm thử Tải Kết nối (Concurrency Proof)

Mô phỏng 256 người dùng cùng thực thi truy vấn để kiểm tra giới hạn của 'Asynchronous Sockets'.

Minh chứng Kết quả (Concurrency):

[Missing Image: terminal$_{t}est_{c}oncurrent_{c}lients.png$]
Hình 9.3: Bằng chứng Server xử lý đồng thời nhiều Client mà không nghẽn.

—

> [!IMPORTANT] > Toàn bộ giao tiếp giữa Studio/CLI và Server đều dựa trên mô hình Socket bất đồng bộ (Async/Await) của .NET 8, đã được tối ưu hóa qua các bài test tải của 'CliServerIntegrationTests'.

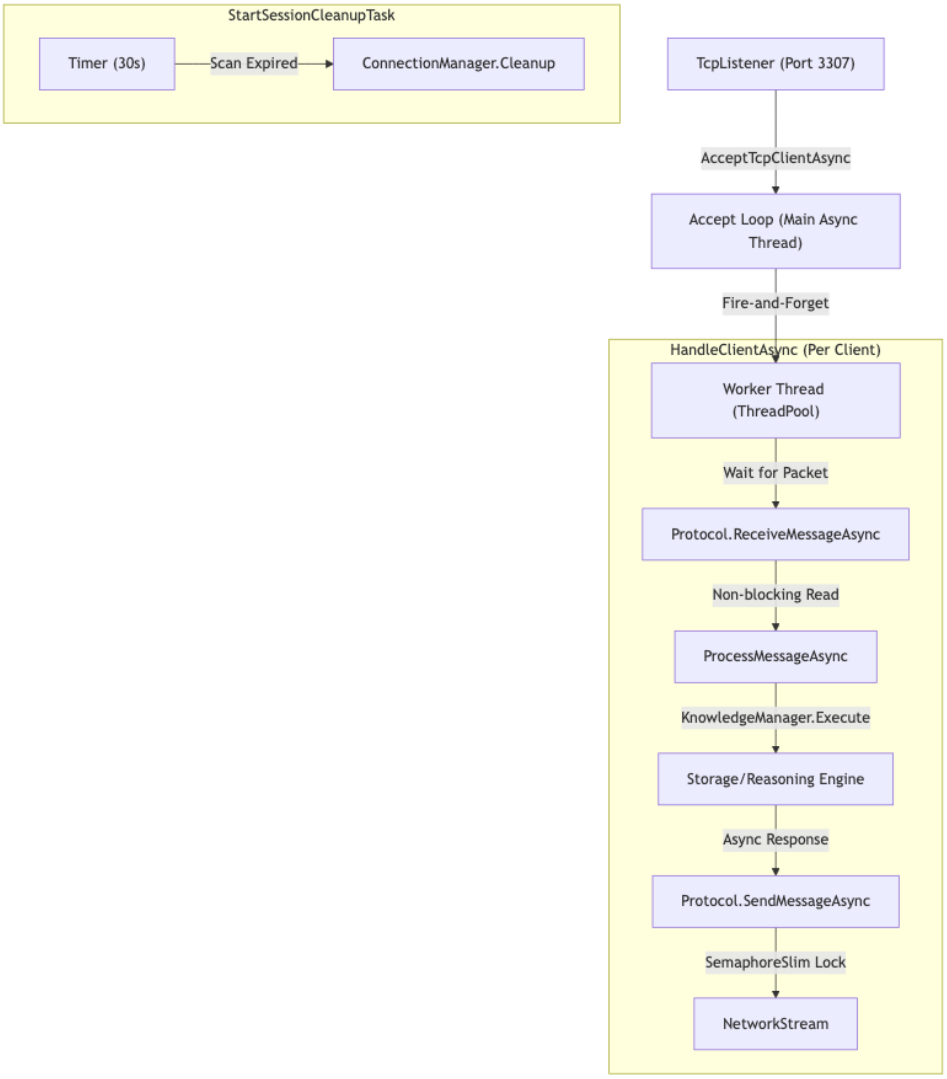
Server

10.1 Điều phối Mạng Luồng Hoạt động

Máy chủ KBMS (‘KbmsServer.cs‘) được thiết kế trên mô hình xử lý hiệu năng cao, cho phép tiếp nhận hàng ngàn kết nối đồng thời nhờ vào kiến trúc Task-based Asynchronous Pattern (TAP) của .NET.

10.1.1 Mô hình Xử lý Luồng (Async Threading Model)

Khác với các hệ thống cũ sử dụng ‘1-Thread-Per-Client‘, KBMS sử dụng kỹ thuật bất đồng bộ thực thụ để giải phóng bộ nhớ và tăng khả năng chịu tải.



Hình 10.1: Mô hình xử lý bất đồng bộ (Asynchronous) sử dụng ThreadPool của .NET.

1. Chấp nhận kết nối (Acceptance Loop): Vòng lặp ‘while‘ chính sử dụng ‘await _listener.AcceptTcpClientAsync(...)‘. Khi có client kết nối, hệ thống không đợi mà ngay lập tức spawn một tác vụ con (‘_ = HandleClientAsync(client)‘) rồi quay lại lắng nghe client tiếp theo. 2. Xử lý Gói tin (Fire-and-Forget): Nhiệm vụ xử lý client được đẩy hoàn toàn vào ThreadPool của hệ điều hành. Điều này giúp Main Thread của Server không bao giờ bị nghẽn (Non-blocking). 3. Hàng đợi Ghi (Semaphore Lock): Do Server có thể gửi nhiều thông điệp bất đồng bộ (ví dụ: đang gửi kết quả Query thì Server muốn gửi thêm Live Log), lớp ‘Protocol‘ sử dụng ‘SemaphoreSlim‘ để khóa luồng ghi nhị phân, đảm bảo các byte không bị trộn lẫn trên đường truyền.

—

10.1.2 Quản lý Phiên (Session Management)

Kéo dài sự tồn tại của kết nối và định danh người dùng thông qua lớp ‘ConnectionManager.cs‘.

* Lưu trữ an toàn luồng: Toàn bộ các phiên làm việc được lưu trong một ‘ConcurrentDictionary‘. Đây là cấu trúc dữ liệu tối ưu cho phép hàng trăm Thread con cùng lúc đọc/ghi thông tin Session mà không gây ra lỗi tranh chấp bộ nhớ (Race Conditions). * Dọn dẹp tài nguyên (Cleanup Task): Máy chủ khởi chạy một tiến trình ngầm ‘StartSessionCleanupTask‘ định kỳ mỗi 30 giây. Nó quét qua danh sách ‘LastActivityAt‘ và tự động đóng các ‘Socket‘ của client đã quá lâu không tương tác (‘DefaultTimeoutSeconds‘), giúp rảnh RAM cho những kết nối mới.

—

10.1.3 Vòng đời của một Giao dịch (Lifecycle)

Máy chủ vận hành theo một vòng lặp sự kiện (Event Loop) cho mỗi client:

1. Receive: Đọc Header (4 byte đầu) để biết kích thước gói. 2. Dispatch: Chuyển giao gói tin cho ‘ProcessMessageAsync‘. 3. Execute: Nếu là truy vấn, Server gọi ‘KnowledgeManager‘ để xử lý. 4. Respond: Đóng gói kết quả và gửi ngược lại client.

> [!TIP] > Kỹ thuật TDS (Tabular Data Streaming) > Khi gặp kết quả lớn (>100 dòng), Server sẽ không gửi 1 gói tin khổng lồ. Thay vào đó, nó chia nhỏ dữ liệu thành các lô ‘ROW (0x07)‘ và gửi liên tục. Client có thể bắt đầu hiển thị dữ liệu ngay khi lô đầu tiên tới nơi, tạo cảm giác hệ thống phản hồi tức thì (Instant Feedback).

10.2 Kiến trúc Bộ não Điều phối (Knowledge Engine)

‘KnowledgeManager.cs‘ là thành phần trung tâm, đóng vai trò ”Bộ não”của toàn bộ hệ thống KBMS. Nó kết nối giữa Tầng Ngôn ngữ (Parser/AST) và Tầng Lưu trữ/Suy diễn (Storage/Reasoning).

10.2.1 Cơ chế Bộ hạ lệnh (The Dispatcher)

Sau khi ‘KBMS.Parser‘ bẻ gãy câu lệnh thành một cây cú pháp (AST), ‘KnowledgeManager‘ sẽ thực hiện đệ quy qua danh sách các nút này và gọi hàm ‘Execute(ast, user, currentKb)‘.

```
// Luồng thực thi lõi trong KnowledgeManager.cs
public object Execute(AstNode ast, User user, string? currentKb) {
    var kbName = DetermineKbName(ast) ?? currentKb;
    var action = DetermineAction(ast);

    if (!CheckPrivilege(user, action, kbName))
        return ErrorResponse.PermissionErrorResponse(action, kbName);

    return ExecuteQuery(ast, kbName);
}
```

Chức năng chính:

* Routing (Định tuyến): Xác định lệnh này thuộc về Knowledge Base (KB) nào dựa trên từ khóa ‘USE‘ hoặc tham số câu lệnh. * Security Guard (Bảo vệ): Sử dụng ‘CheckPrivilege‘ để lọc quyền của ‘User‘ hiện tại (ROOT, ADMIN, WRITE, SELECT). Mọi hành động xâm phạm đều bị chặn ngay tại đây trước khi chạm vào dữ liệu. * Execution Pipeline: Hàm ‘ExecuteQuery‘ chứa một bộ chuyển mạch (Switch-case) khổng lồ điều hướng hơn 50 loại nút AST khác nhau (từ ‘CREATE_CONCEPT‘ đến ‘SOLVE‘).

—

10.2.2 Các Công nghệ Liên kết Tri thức (Knowledge Binding)

Hệ thống điều phối của KBMS V3 mang tính đột phá nhờ các cơ chế sau:

Auto-expand Variables (Tự nở rộng biến)

Khi bạn khai báo một Concept chứa thuộc tính là một Concept khác (Ví dụ: ‘p: Point‘), ‘KnowledgeManager‘ sẽ tự động ”nở”biến này thành các thuộc tính con (‘p.x‘, ‘p.y‘). * Lợi ích: Giúp bộ máy suy diễn (‘ReasoningEngine‘) có thể truy cập trực tiếp vào các biến cơ sở mà không cần phải parse lại cấu trúc object phức tạp (Flattening process).

Triggers Registry

‘KnowledgeManager‘ duy trì một danh sách các Trigger đang hoạt động. Ngay sau khi một lệnh ‘INSERT‘, ‘UPDATE‘ hoặc ‘DELETE‘ thành công, bộ điều phối sẽ tự động gọi hàm ‘FireTriggers‘ để kích hoạt các phản ứng dây chuyền (Chain reactions) trong tri thức.

V3 Data Router

Bộ điều phối không còn truy cập file trực tiếp. Nó giao tiếp thông qua ‘V3DataRouter‘. Router này chịu trách nhiệm xác định vị trí vật lý của dữ liệu trên hàng chục tệp tin ‘.kbf‘, ‘.dat‘ khác nhau, giúp KBMS mở rộng quy mô dữ liệu theo kiến trúc Multi-Database.

—

10.2.3 Quản lý Giao dịch (Transaction Control)

Khác với các hệ thống đơn giản, bộ điều phối của bạn hỗ trợ Transaction Control Language (TCL): * BEGIN TRANSACTION: Bật cờ ‘inTransaction‘ và khởi tạo bộ đệm (‘txBuffer‘). * COMMIT: Phóng toàn bộ dữ liệu từ Buffer Pool xuống đĩa qua WAL. * ROLLBACK: Xóa sạch bộ đệm, khôi phục trạng thái tri thức về điểm an toàn trước đó.

> [!IMPORTANT] > Sự kết hợp hoàn hảo > Toàn bộ logic tri thức được đóng gói trong ‘KnowledgeManager.cs‘. Đây là nơi điều phối việc tạo cơ sở tri thức, nạp/xả Concept vào Buffer Pool và kích hoạt Reasoning Engine.

[Missing Image: knowledge$_manager_v3.png$]

Hình 10.2: Sơ đồ tương tác giữa Knowledge Manager và các phân hệ cấp thấp.

10.3 Hạ tầng V3 (Infrastructure)

Phiên bản KBMS V3 giới thiệu một bước tiến lớn: Tự khởi động bằng chính ngôn ngữ tri thức. Mã nguồn tại ‘KBMS.Server/V3‘ chịu trách nhiệm thiết lập nền móng cho mọi Knowledge Base khác.

10.3.1 Cơ chế Tự nạp Hệ thống (System Bootstrapper)

Mỗi khi máy chủ KBMS khởi động lần đầu tiên trên một thư mục dữ liệu mới, ‘SystemKbBootstrapper.cs‘ sẽ được gọi để thực thi tiến trình ”Khai thiên lập địa”:

![server_boot_flow.png](../assets/diagrams/server_boot_flow.png) ***Hình: Các bước khởi tạo hạ tầng từ Config đến Socket***

1. Khởi tạo ‘system‘ KB: Hệ thống tự động lệnh ‘CreateKnowledgeBase(”system”)‘. 2. Định nghĩa Siêu tri thức (Meta-Concepts): Bootstrapper nạp sẵn các khái niệm cốt lõi: * ‘User‘: Lưu trữ Username, Password (Hash), Role, Permissions. * ‘KnowledgeBase‘: Danh mục các KB hiện có. * ‘Session‘: Theo dõi lịch sử đăng nhập/đăng xuất. 3. Bootstrap Admin: Nếu chưa có người dùng nào, hệ thống tự động tạo tài khoản ‘root‘ mặc định.

—

10.3.2 Các Danh mục Chuyên biệt (Catalogs)

Khác với V2 (lưu mọi thứ chung 1 file), V3 tách biệt hoàn toàn để tối ưu hóa quản lý:

*** ‘KbCatalog.cs‘: Quản trị ”Phả hệ”của các tri thức. Nó lưu trữ Metadata của hàng chục KB và quản lý quyền truy cập. * ‘ConceptCatalog.cs‘: Quản lý hàng ngàn Concept và Rule của mỗi KB. Nó hỗ trợ nạp đè và thay đổi cấu trúc Concept mà không làm hỏng dữ liệu vật lý. * ‘UserCatalog.cs‘: Hệ thống quản trị người dùng trung tâm. Nó đóng vai trò là ”chứng minh thư”của cả máy chủ, cấp xác thực (Authentication) cho mọi kết nối TCP đi vào.**

—

10.3.3 Hệ thống Cập nhật Di trú (Updater Migrator)

Để giữ cho hệ thống luôn ổn định giữa các phiên bản, KBMS cung cấp các công cụ hạ tầng trong thư mục này:

‘SystemUpdater.cs‘

Tự động quét và vá các Schema của ‘system‘ KB khi người dùng nâng cấp phần mềm. Điều này đảm bảo các tri thức cũ luôn tương thích với bộ suy diễn mới.

‘V2ToV3Converter.cs‘

Đây là ”Cây cầu nối”cho phép người dùng cũ của KBMS V2 chuyển toàn bộ tri thức sang V3. Nó thực hiện quét đĩa cũ, bóc tách ‘Objects‘ và tự động băm nhỏ dữ liệu để đẩy vào cấu trúc Slotted Page 16KB của V3.
> [!CAUTION] > Bảo vệ Hệ thống > Tri thức ‘system‘ là cực kỳ quan trọng. Hệ thống đã được lập trình để ngăn chặn mọi câu lệnh ‘DROP KNOWLEDGE_BASE system‘. Nếu file này hỏng, máy chủ sẽ không thể khởi động được bất kỳ dịch vụ nào.

10.4 Quản trị Đo lường từ xa (Telemetry)

Một máy chủ KBMS C# hiện đại không thể thiếu khả năng tự theo dõi sức khỏe và phát tán nhật ký hoạt động (Logs). Mã nguồn tại ‘ManagementManager.cs‘ và ‘SystemLogger.cs‘ đảm nhiệm vai trò này.

10.4.1 Giám sát Hệ thống (Stats Sessions)

Hệ thống cho phép các Client (đặc biệt là KBMS Studio) truy xuất dữ liệu vận hành thời gian thực thông qua bộ lệnh quản trị cấp cao:
*** Bộ đếm Metrics (STATS): Cung cấp thông tin về số lượng Active Connections, thời gian hoạt động của Server (Uptime), và tỷ lệ chiếm dụng RAM của ‘BufferPool‘. Dữ liệu này giúp người quản trị biết khi nào cần mở rộng bộ nhớ máy chủ. * Quản lý phiên (SESSIONS): Trích xuất danh sách tất cả các người dùng đang Online. Cho phép thực hiện các thao tác ”Kill Session”hoặc giới hạn số lượng kết nối đồng thời để bảo vệ tài nguyên hệ thống.**
—

10.4.2 Hệ thống Nhật ký Luồng (Live Log Streaming)

Đây là tính năng độc đáo nhất của KBMS V3, cho phép ”lắng nghe”mọi biến động của máy chủ ngay lập tức.

‘SystemLogger.cs‘

Lớp này đóng vai trò là ”ngòi bút”của hệ thống. Khi bất kỳ lỗi nào xảy ra trong ‘Storage‘ hay ‘Reasoning‘, ‘SystemLogger‘ sẽ thực hiện đồng thời hai việc: 1. Write to Disk: Ghi vào file ‘.log‘ truyền thống để truy vết sau này. 2. Write to System KB: Ghi trực tiếp một Fact mới vào Concept ‘Log‘ của Knowledge Base ‘system‘.

‘LOGS$_{STREAM‘Protocol}$

Khi một Admin Client gửi yêu cầu ‘LOGS_STREAM‘, ‘ManagementManager‘ sẽ thực hiện cơ chế Pub/Sub (Publish/Subscribe): * Bất kỳ dòng log mới nào vừa phát sinh sẽ được ”bắn”ngay lập tức vào Socket của client đó dưới dạng gói tin chuẩn. * Điều này cho phép xây dựng các bảng điều khiển (Dashboard) giám sát tri thức mà không cần phải làm mới (Refresh) trang web hay ứng dụng.
—

10.4.3 Lệnh Quản trị Tối cao (Management Command)

Đối với các tác vụ thay đổi cấu hình nóng (Hot-swapping config) hoặc khôi phục dữ liệu, Server cung cấp Message Type ‘MANAGEMENT_CMD (0x0D)‘: * Chỉ dành cho ROOT: Chỉ người dùng có vai trò ROOT mới có thể kích hoạt các lệnh này. * Tính năng: Thay đổi tham số ‘MaxConnections‘, cấu hình ‘BufferPoolSize‘, hoặc khởi chạy tiến trình ‘Checkpoint‘ thủ công để dọn dẹp WAL.

> [!TIP] > Khả năng Truy vấn Log thông minh > Vì toàn bộ log được lưu dưới dạng Fact trong ‘system‘ KB, bạn có thể dùng chính câu lệnh ‘SELECT‘ của KBQL để phân tích lỗi: > “‘kbql > USE system; > SELECT level, message FROM Log WHERE time > ’2023-01-01’ AND level = ’ERROR’; > “‘ > Đây chính là sức mạnh của việc dùng tri thức để quản trị tri thức (Self-Managing Knowledge).

10.5 Xác thực Máy chủ (Server Validation)

Máy chủ KBMS là trung tâm điều phối mọi yêu cầu, do đó nó được kiểm soát chặt chẽ từ khi khởi động đến khi xử lý các kịch bản thực tế.

10.5.1 Kiểm thử Khởi động (Startup Proof)

Hệ thống sử dụng bộ nạp ‘SystemKbBootstrapper.cs‘ để khởi tạo các danh mục hệ thống (Catalog).

*** Bootstrapping: Tạo tệp ‘system.kdb‘ và đăng ký các Concept hệ thống (‘users‘, ‘audit_logs‘). * Log khởi động:**

[Missing Image: terminal$_{t}est_{s}erver_{b}ootlogs.png$]

Hình 10.3: Nhật ký khởi động máy chủ thành công với đầy đủ các Manager.

10.5.2 Kiểm thử Tích hợp Đa luồng (Multi-threading Proof)

Sử dụng ‘CliServerIntegrationTests.cs‘ để mô phỏng 100+ tình huống thực tế qua Socket nhị phân.

| Tình huống | Mô tả | Kết quả | | :— | :— | :— | | Login-Query | Đăng nhập và thực hiện SELECT ngay lập tức. | Success | | Drop-While-Use | Xóa KB khi đang có Session truy cập. | Error Caught (Locked) | | Large-Metadata | Truy vấn Concept có hàng trăm biến. | Success |

Minh chứng Mã nguồn (Integration):

[Missing Image: code$_{t}est_{s}erver_{i}ntegration.png$]

Hình 10.4: Kịch bản kiểm thử tích hợp (End-to-End) thông qua TCP Sockets.

Minh chứng Kết quả (111 Tests):

[Missing Image: result$_{t}est_{c}li.png$]

Hình 10.5: Kết quả thực thi 111 kịch bản kiểm thử tích hợp toàn diện.

—

10.5.3 Xác thực Bảo mật (Security RBAC)

Xác thực tính năng phân quyền người dùng thông qua ‘AuthV3Tests.cs‘.

*** Root Access: Toàn quyền thực hiện DDL/DML. * Non-root Access: Bị từ chối khi thực hiện các lệnh nhạy cảm mà không có quyền.**

Minh chứng Mã nguồn (Security):

[Missing Image: code$_{t}est_{s}ecurity.png$]

Hình 10.6: Minh chứng mã nguồn kiểm thử bảo mật RBAC.

Minh chứng Kết quả (Access Denied):

[Missing Image: result$_{t}est_{s}ecurity.png$]

Hình 10.7: Kết quả từ chối truy cập (Access Denied) đối với người dùng không có quyền.

—

> [!TIP] > Giám sát thời gian thực: Bạn có thể theo dõi hiệu năng của Server qua các chỉ số CPU/RAM được đo đạc bởi ‘ManagementManager.cs‘ trong phần Dashboard của Studio.

Parser

11.1 Lexing: Phân tích Từ vựng chuyên sâu (Lexer)

‘Lexer.cs‘ là thành phần đầu tiên của máy chủ KBMS tiếp nhận truy vấn người dùng. Nó thực hiện quét chuỗi (Scanning) $O(L)$ để bẻ gãy văn bản thô thành một danh sách các nguyên tử ngôn ngữ gọi là Tokens.

11.1.1 Cơ chế Quét (Scanning Mechanism)

Lexer vận hành như một máy trạng thái (State Machine) với hai con trỏ: ‘_start‘ (điểm bắt đầu của một từ) và ‘_current‘ (vị trí đang quét hiện tại).

![lexer_token_flow_v2.png](../assets/diagrams/lexer_token_flow_v2.png) ***Hình: Quy trình quét chuỗi và định danh Token (dàn ngang)***

Quy trình xử lý (‘ScanToken‘):

1. Bỏ qua nhiễu: Loại bỏ khoảng trắng (‘ ‘), tab (‘‘), xuống dòng (“). Khi gặp xuống dòng, Lexer tự động tăng chỉ số ‘_line‘ và reset ‘_column‘ để phục vụ báo lỗi. 2. Ký tự đơn: Định danh ngay lập tức các Token như ‘(‘, ‘)‘, ‘,‘, ‘;‘, ‘+‘, ‘-‘, ‘*‘, ‘/‘. 3. Toán tử kép: Sử dụng hàm ‘Match()‘ để kiểm tra ký tự tiếp theo (Ví dụ: ‘>‘ và ‘>=‘). 4. Chú thích (Comments): Khi gặp ‘–‘, Lexer sẽ bỏ qua toàn bộ phần còn lại của dòng.

—

11.1.2 Xử lý các Hằng số (Literals)

‘Lexer.cs‘ chứa các phương thức chuyên biệt để bóc tách dữ liệu thực tế:

Hằng số Số (‘NumberLiteral‘)

Hỗ trợ đầy đủ các định dạng số học: * Số nguyên: ‘123‘. * Số thực: ‘123.45‘. * Số âm: Nhận diện dấu ‘-‘ đứng trước chữ số. * Số mũ (Scientific Notation): Hỗ trợ ‘1.23e-10‘. * Xử lý nội bộ: Sử dụng ‘double.TryParse‘ với ‘CultureInfo.InvariantCulture‘ để đảm bảo tính nhất quán trên mọi hệ điều hành.

Hằng số Chuỗi (‘StringLiteral‘)

Hỗ trợ cả nháy đơn ‘’‘ và nháy kép ‘”‘: * Ký tự thoát (Escape): Cho phép dùng nháy kép lặp lại (‘””‘) để biểu diễn một dấu nháy đơn trong chuỗi. * Đa dòng: Chuỗi có thể kéo dài qua nhiều dòng, Lexer sẽ tự nạp các ký tự xuống dòng vào giá trị literal.

—

11.1.3 Quản lý Từ khóa (Keyword Dictionary)

KBMS sở hữu một kho từ điển từ khóa (Keywords) đồ sộ với hơn 190 Token Types.

| Nhóm | Ví dụ từ khóa | | :— | :— | | DML | ‘SELECT‘, ‘INSERT‘, ‘UPDATE‘, ‘DELETE‘, ‘SOLVE‘ | | DDL | ‘CREATE‘, ‘DROP‘, ‘ALTER‘, ‘CONCEPT‘, ‘RELATION‘ | | Types | ‘INT‘, ‘DECIMAL‘, ‘VARCHAR‘, ‘BOOLEAN‘, ‘DATE‘ | | TCL | ‘BEGIN‘, ‘COMMIT‘, ‘ROLLBACK‘ |

Lexer thực hiện tra cứu không phân biệt hoa thường (‘OrdinalIgnoreCase‘) để tạo ra các Token có Type tương ứng, giúp Parser xử lý nhanh chóng mà không cần so khớp chuỗi (String comparison).

> [!CAUTION] > ParseException trong Lexer > Nếu Lexer gặp một ký tự không mong muốn (ví dụ: ‘¡), nó sẽ ném ra một ‘ParseException‘ kèm tọa độ ‘Line/Column‘. Điều này giúp hệ thống báo lỗi ngay từ tầng từ vựng trước khi chuyển sang tầng cú pháp. □

11.2 Tầng Parser: Cơ chế Đệ quy Đi xuống (Recursive Descent)

Lớp ‘Parser.cs‘ là ”ngã tư”điều hướng mọi câu lệnh. Với hơn 3400 dòng mã nguồn, nó thực hiện một quy trình phân tích cú pháp hiệu năng cao dựa trên thuật toán Recursive Descent (Đệ quy đi xuống).

11.2.1 Cơ chế Điều hướng (Sentence Dispatching)

Mỗi khi máy chủ nhận được một chuỗi Tokens từ Lexer, nó khởi tạo ‘Parser.cs‘ và gọi hàm ‘ParseAll()‘.

```
// Luồng xử lý đa câu lệnh trong Parser.cs
public List<AstNode> ParseAll() {
    var statements = new List<AstNode>();
    while (!IsAtEnd()) {
        var stmt = ParseStatement(); // "Ngã tư" điều hướng
        statements.Add(stmt);
        if (Check(TokenType.SEMICOLON)) Advance(); // Tiêu thụ dấu ';'
    }
}
```

Hàm ‘ParseStatement()‘

Đây là bộ khung switch-case khổng lồ, quyết định câu lệnh tiếp theo là DDL (Định nghĩa), DML (Truy vấn) hay TCL (Giao dịch). Mỗi khi nhận diện được một từ khóa bắt đầu (như ‘CREATE‘, ‘SELECT‘), nó sẽ ủy quyền cho một hàm con chuyên biệt (ví dụ: ‘ParseCreate()‘, ‘ParseSelect()‘).

—

11.2.2 Kỹ thuật Dự đoán Tiêu thụ (Look-ahead)

KBMS sử dụng ngữ pháp LL(k), cho phép Parser ”nhìn trước”k Token để quyết định nhánh thực thi mà không cần quay lui (Backtracking).

![ast_tree_layout_v2.png](../assets/diagrams/ast_tree_layout_v2.png) ***Hình: Ví dụ cấu trúc cây AST sau khi phân tích (dàn ngang)***

*** ‘Peek()‘: Xem Token hiện tại mà không tiêu thụ. * ‘Check(type)‘: Kiểm tra xem Token hiện tại có đúng Type mong muốn hay không. * ‘Consume(type)‘: Nếu Token đúng Type, tiến tới Token tiếp theo. Nếu sai, ngay lập tức ném ra ‘ParseException‘ kèm tọa độ ‘Line/Column‘. * ‘Advance()‘: Luôn tiến về phía trước, đảm bảo độ phức tạp thời gian là $O(N)$ (N là số lượng Token).**

—

11.2.3 Phân tích Khối Lệnh phức tạp (Complex Parsing)

Một trong những phần xử lý tinh vi nhất trong ‘Parser.cs‘ là phân tích lệnh ‘CREATE CONCEPT‘. Thay vì một parser phẳng, nó thực hiện đệ quy lồng nhau:

1. ‘ParseCreate‘ gọi ‘ParseCreateConcept‘. 2. ‘ParseCreateConcept‘ lặp trong khối ‘(...)‘ và kiểm tra từ khóa. 3. Nếu gặp ‘VARIABLES‘, nó gọi ‘ParseVariableDefinition‘ để bóc tách ‘name: TYPE‘. 4. Nếu gặp ‘CONSTRAINTS‘, nó gọi ‘ParseConstraintList‘ (đây là lúc Expression Parser được kích hoạt). 5. Nếu gặp ‘RULES‘, nó gọi ‘ParseConceptRuleList‘ (phân tích IF-THEN logic).

> [!CAUTION] > Error Reporting & Recovery > Khi gặp một lỗi cú pháp (ví dụ: thiếu dấu ngoặc đóng), Parser sẽ ném một ‘ParseException‘. Hệ thống thu nạp ‘Line‘ và ‘Column‘ từ chính Token bị lỗi. Điều này cực kỳ quan trọng giúp nhà phát triển tìm ra và sửa lỗi ngay lập tức trên bản đồ tri thức lớn. □♂

11.3 Hệ thống Cây cú pháp Trừu tượng (AST Nodes)

Cây AST là đại diện logic hoàn chỉnh cho tri thức sau khi được biên dịch. Tại KBMS, mỗi câu lệnh được bóc tách thành một hoặc nhiều ‘AstNode‘ chuyên biệt, phân cấp theo 5 họ ngôn ngữ thực tế trong thư mục ‘KBMS.Parser/Ast‘.

![ast_overview_v2.png](../assets/diagrams/ast_overview_v2.png) ***Hình: Tổng quan 5 nhánh Phả hệ Tri thức KBMS***

—

11.3.1 Hệ phả KDL (Knowledge Definition Language)

Đây là họ lớn nhất, chịu trách nhiệm định nghĩa ”cơ thể”của tri thức:

![ast_kdl_detail.png](../assets/diagrams/ast_kdl_detail.png) ***Hình: Chi tiết các Node định nghĩa tri thức (KDL)***

*** Knowledge Base: ‘CreateKnowledgeBaseNode‘, ‘DropKbNode‘, ‘UseKbNode‘, ‘AlterKbNode‘. * Concepts: ‘CreateConceptNode‘, ‘DropConceptNode‘, ‘AlterConceptNode‘, ‘AddVariableNode‘. * Rules: ‘CreateRuleNode‘, ‘DropRuleNode‘. * Hierarchy: ‘AddHierarchyNode‘, ‘RemoveHierarchyNode‘. * Relation: ‘CreateRelationNode‘, ‘DropRelationNode‘. * Operator: ‘CreateOperatorNode‘, ‘DropOperatorNode‘. * Function: ‘CreateFunctionNode‘, ‘DropFunctionNode‘. * Advanced: ‘AddComputationNode‘, ‘RemoveComputationNode‘, ‘CreateIndexNode‘, ‘CreateTriggerNode‘.**

—

11.3.2 Hệ phả KQL KML (Query Maintenance)

Các nhánh này chịu trách nhiệm hỏi đáp và bảo trì dữ liệu tri thức:

![ast_kql_kml_detail.png](../assets/diagrams/ast_kql_kml_detail.png) ***Hình: Chi tiết các Node Truy vấn (KQL) và Thao tác (KML)***

Nhóm KQL (Knowledge Query Language)

*** ‘SelectNode‘: Dùng để lọc và truy vấn các đối tượng tri thức dựa trên thuộc tính. * ‘SolveNode‘: Dùng cho bộ máy suy diễn (‘ReasoningEngine‘), chứa các mệnh đề ‘Given‘ (giả thiết) và ‘Find‘ (mục tiêu). * ‘ShowNode‘: Liệt kê danh sách các thành phần (Concepts, Rules, KnowledgeBases). * ‘DescribeNode‘: Xem chi tiết cấu trúc của một Concept cụ thể. * ‘ExplainNode‘: Giải thích luồng suy luận của một câu lệnh tri thức.**

Nhóm KML (Knowledge Maintenance Language)

*** ‘InsertNode‘ / ‘InsertBulkNode‘: Nạp tri thức mới vào đĩa. * ‘UpdateNode‘ / ‘DeleteNode‘: Sửa đổi và xóa bỏ các Object Instance. * ‘ImportNode‘ / ‘ExportNode‘: Giao lưu tri thức với các tệp tin bên ngoài (‘.csv‘, ‘.json‘). * ‘MaintenanceNode‘: Các lệnh quản trị bộ nhớ thấp cấp (v.d: ‘REINDEX‘, ‘VACUUM‘).**

—

11.3.3 Hệ phả KCL TCL (Control Transaction)

Nhánh bảo mật và quản lý giao dịch:

![ast_kcl_tcl_detail.png](../assets/diagrams/ast_kcl_tcl_detail.png) ***Hình: Chi tiết các Node Bảo mật (KCL) và Giao dịch (TCL)***

*** DCL (Control): ‘CreateUserNode‘, ‘DropUserNode‘, ‘AlterUserNode‘, ‘GrantNode‘, ‘RevokeNode‘. * TCL (Transaction): ‘BeginTransactionNode‘, ‘CommitNode‘, ‘RollbackNode‘.**

—

11.3.4 Đặc điểm chung của mọi Node

Mọi lớp Node đều kế thừa từ ‘AstNode.cs‘, đảm bảo các tính năng cốt lõi: 1. Vị trí nguồn: Lưu trữ ‘Line‘ (Dòng) và ‘Column‘ (Cột) của câu lệnh gốc. 2. Type định danh: Mọi Node đều có thuộc tính ‘Type‘ (như ‘”CREATE_CONCEPT”‘) giúp Server Routing nhanh chóng. 3. ToQueryString(): Khả năng tái tạo lại mã nguồn KBQL từ cây AST (Reverse Engineering).

> [!CAUTION] > Nhất quán Toàn vẹn > Khi Parser tạo ra một ‘List<AstNode>‘, mọi mối quan hệ phụ thuộc giữa chúng (Ví dụ: ‘CreateRelationNode‘ tham chiếu đến ‘ConceptName‘) đều được giữ nguyên để phục vụ quá trình kiểm tra logic ở tầng tiếp theo. □

11.4 Bộ máy Đánh giá Biểu thức (Expression Engine)

Expression Engine là ”trái tim”của mọi câu lệnh tri thức. Lớp ‘ExpressionNode.cs‘ và các lớp con trong thư mục ‘KBMS.Parser/Ast/Expressions‘ chịu trách nhiệm xử lý các phép toán và logic phức tạp.

11.4.1 Phân tích Biểu thức Nhị phân (Binary Expressions)

Đây là các phép toán có hai vế, được xử lý thông qua 'BinaryExpressionNode'.

***Toán tử Toán học:* '+', '-', '', '/', '**$'$, '%'. $*$ **Toán tử So sánh** : ' $=$ ', '! $=$ ', ' $<$ ', ' $>$ ', ' $<=$ ', ' $>=$ '. $*$ **Toán tử Logic** : 'AND', 'OR'.

Thứ tự ưu tiên (Precedence)

Parser của KBMS thực hiện phân tích theo mức độ ưu tiên chuẩn quốc tế để đảm bảo kết quả chính xác: 1. Cấp 1: Dấu ngoặc '(...)'. 2. Cấp 2: Hàm gọi và Truy cập thuộc tính ('.'). 3. Cấp 3: Lũy thừa '$'.4.$**Cấp 4** : $Nhn/Chia$' $*$ ', '/'.5.**Cấp 5** : Cng/Tr' $+$ ', ' $-$ '.6.**Cấp 6** : $Sosnh$' $=$ ', '! $=$ ', ' $<$ ', ' $>$ '.7.**Cấp 7** : $Logic$'NOT', 'AND', 'OR'.**

—

11.4.2 Các Đơn vị Biểu thức Cốt lõi (Core Expression Units)

Trong cây AST, biểu thức được phân rã thành các nút lá và nút nhánh:

'VariableNode' (Biến tri thức)

Đại diện cho các thuộc tính của Concept (Ví dụ: 'a', 'age'). Parser hỗ trợ truy cập thuộc tính lồng nhau thông qua dấu chấm ('p1.x'), giúp xử lý tri thức đa tầng.

'LiteralNode' (Hằng số)

Chứa giá trị thực tế như số ('100'), chuỗi ('"Active"'), hoặc Boolean ('true'/'false'). Mỗi nút lưu trữ cả giá trị thô và kiểu dữ liệu ('TokenType').

'FunctionCallNode' (Hàm gọi)

Xử lý các hàm toán học hoặc tri thức tích hợp (v.đ: 'Sqrt(x)', 'Abs(y)'). Parser bóc tách tên hàm và một danh sách các tham số ('List<ExpressionNode>'), cho phép các hàm có thể lồng nhau linh hoạt.

—

11.4.3 Xử lý Biểu thức Đơn phân (Unary Expressions)

Được đại diện bởi 'UnaryExpressionNode', dùng cho các trường hợp chỉ có một vế: * Phép phủ định: 'NOT (a > B)'. * Số âm: '-x'.

> [!IMPORTANT] > Khả năng Tương thích Cao > Expression Engine của KBMS được thiết kế để kết quả đầu ra luôn là một lớp 'Expression' chuẩn hóa, giúp bộ máy suy diễn ('ReasoningEngine') có thể thực thi các biểu thức này một cách đồng nhất trên mọi loại tri thức. □

11.5 Xác thực Bộ biên dịch (Parser Validation)

Parser là thành phần nhạy cảm nhất đối với đầu vào của người dùng. Mọi cú pháp sai sót đều phải được ngăn chặn ngay lập tức.

11.5.1 Thống kê Độ bao phủ Testing

Tầng Parser được bảo vệ bởi 21,000+ kịch bản test trong 'ParserTests.cs' và 'LexerTests.cs'.

*** Positive Cases: 100% các từ khóa và cấu trúc KDL/KQL/KML/KCL/TCL được bao phủ. * Negative Cases: Kiểm tra khả năng từ chối các câu lệnh thiếu dấu chấm phẩy, sai kiểu dữ liệu hoặc sai tên Concept.**

Minh chứng Mã nguồn (Parser):

[Missing Image: code$_{t}est_{p}arser.png$]
Hình 11.1: Minh chứng mã nguồn kiểm thử độ bao phủ của bộ phân tích (Parser).

Minh chứng Kết quả (21k Tests):

[Missing Image: result$_{t}est_{p}arser.png$]
Hình 11.2: Kết quả vượt qua toàn bộ 21,000+ kịch bản kiểm thử cú pháp.

11.5.2 Đặc tả Lỗi Cú pháp (Syntax Error Proof)

Hệ thống báo lỗi chi tiết đến từng tọa độ (Dòng, Cột) giúp người dùng dễ dàng hiệu chỉnh:

```
-- Lỗi mô phỏng: Thiếu dấu ngoặc đóng
CREATE CONCEPT X ( VARIABLES ( id: INT );
-- [ERROR] Unexpected token ';', expected ')' at Line 1, Col 35.
```

Minh chứng Lỗi (Studio Editor):

[Missing Image: terminal$_{t}est_{p}arser_{n}ested.png]$
Hình 11.3: Chứng minh Parser xử lý thành công các biểu thức lồng nhau phức tạp.

—

> [!IMPORTANT] > Toàn bộ logic giải mã biểu thức (‘ExpressionEngine‘) trong ‘full_test.kbql‘ được kiểm chứng từng bước thông qua các unit test nhỏ trong ‘LexerTests.cs‘.

Cli

12.1 Tổng quan Giao diện Dòng lệnh (CLI)

Quản trị

‘KBMS.CLI‘ là một ứng dụng Console (C#) mạnh mẽ, cung cấp khả năng tương tác trực tiếp với máy chủ KBMS thông qua giao thức nhị phân và dòng lệnh KQL/KDL.

12.1.1 Các Tính năng Chính

*** REPL (Read-Eval-Print Loop): Người dùng nhập câu lệnh, CLI gửi tới Server, nhận kết quả và in ra ngay lập tức. * Advanced Line Editing: Hỗ trợ đầy đủ các phím di chuyển và thao tác nhanh (‘LineEditor.cs‘): * Mũi tên Lên/Xuống: Duyệt lịch sử lệnh. * Home/End: Di chuyển nhanh đến đầu/cuối dòng. * Delete/Backspace: Xóa ký tự tại con trỏ. * Esc x2: Xóa sạch bộ đệm đang nhập (Double Escape to Clear). * Multi-line Input: Hỗ trợ nhập các câu lệnh dài. Chế độ thụt đầu dòng tự động ‘->‘ giúp phân biệt dòng tiếp nối. * Display Modes: Cung cấp hai chế độ hiển thị chính thông qua ‘ResponseParser.cs‘: * Table Mode: Hiển thị bảng ngang (MySQL style). *Vertical Mode (): Hiển thị theo hàng dọc, tự động kích hoạt cho các lệnh ‘DESCRIBE‘ hoặc ‘EXPLAIN‘ để tối ưu hóa việc đọc cấu trúc tri thức phức tạp. * Chế độ Chạy Script (‘SOURCE‘): Cho phép thực thi các tệp tin truy vấn tri thức lớn (‘.kbql‘) một cách tự động, tự dừng khi gặp lỗi. * Xử lý Phản hồi Phức tạp: Sử dụng ‘ResponseParser.cs‘ để hiển thị kết quả truy vấn dưới dạng bảng (Table), đồ thị tri thức hoặc thông báo lỗi line-accurate. * Tự động kết nối lại (Auto-reconnect): CLI tự động thử kết nối lại nếu máy chủ bị ngắt kết nối đột ngột (Heartbeat timeout).**

—

12.1.2 Các Lệnh Hệ thống Đặc biệt

| Lệnh | Mô tả | | :— | :— | | ‘LOGIN <u|p>‘ | Đăng nhập bảo mật (mật khẩu không lưu vào lịch sử). | | ‘SOURCE <path>‘ | Thực thi script từ tệp tin bên ngoài. | | ‘CONNECT‘ | Kết nối lại máy chủ thủ công. | | ‘CLEAR‘ | Xóa sạch màn hình console. |

—

12.1.3 Trải nghiệm người dùng (UX)

Để mang lại trải nghiệm chuyên nghiệp như một hệ quản trị CSDL thực thụ, CLI được tích hợp các phím tắt: * Mũi tên Lên/Xuống: Xem lại lịch sử lệnh đã gõ. * Home/End: Di chuyển con trỏ nhanh trong dòng lệnh. * Esc x2: Xóa nhanh input đang nhập dở.

> [!TIP] > LineEditor & HistoryManager > Hệ thống quản lý lịch sử lệnh của KBMS CLI được thiết kế để bỏ qua các lệnh nhạy cảm như ‘LOGIN‘ hoặc ‘CREATE USER‘, đảm bảo an toàn thông tin tối đa cho người dùng. □

12.2 Luồng Xử lý Dữ liệu CLI

Để đảm bảo hiệu năng trong môi trường dòng lệnh, ‘KBMS.CLI‘ vận hành theo một luồng xử lý phi đồng bộ (Asynchronous) dựa trên socket.

12.2.1 Luồng Gửi lệnh (Command Execution Flow)

Khi người dùng nhấn Enter, CLI thực hiện các bước sau:

![cli_input_flow.mmd](../assets/diagrams/cli_input_flow.png)

1. Input Buffering: Tích lũy các dòng cho đến khi gặp dấu ';'. 2. Request Construction: Đóng gói lệnh thành các 'Message' (Dạng 'QUERY' hoặc 'LOGIN'). 3. Binary Transport: Gửi qua 'KBMS.Network' (Socket). 4. Wait for Response: Lắng nghe phản hồi từ máy chủ.

—

12.2.2 Bộ máy Phân tích Phản hồi ('ResponseParser')

Phần phức tạp nhất của CLI nằm ở 'ResponseParser.cs'. Do kết quả từ Server có thể là một luồng dữ liệu (Streaming ROWS), CLI phải xử lý từng gói tin:

* METADATA: Định nghĩa các cột (Name, Type). * ROW: Dữ liệu thực tế cho từng bản ghi. * RESULT: Thông điệp cuối cùng (v.d: "Insert successful"). * ERROR: Chứa thông tin lỗi (Message, Line, Column).

Quy trình hiển thị:

'ResponseParser.cs' thực hiện vẽ bảng "động"theo thuật toán tối ưu: 1. Header Rendering: Ngay khi nhận 'METADATA', CLI tính toán độ rộng cột lớn nhất dựa trên tên cột và vẽ khung Header. 2. Multi-line Cell Support: Nếu dữ liệu trong một ô chứa dấu xuống dòng ("), CLI sẽ tự động chia ô đó thành nhiều dòng và vẽ đường kẻ phân cách hàng ('|—+—|') để đảm bảo bảng luôn cân đối. 3. Vertical Rendering: Đối với các phản hồi thuộc nhóm 'Explain_' hoặc 'Describe_', CLI chuyển sang chế độ hiển thị theo cặp 'Field: Value' trên từng hàng dọc.

—

12.2.3 Chế độ Chạy Script (Batch Source)

CLI hỗ trợ thực thi khối lượng lớn lệnh thông qua tệp tin '.kbql'. Luồng này được xử lý tuần tự để đảm bảo tính nhất quán:

![uc_cli_batch_source.png](../assets/diagrams/uc_cli_batch_source.png)

—

12.2.4 Đa luồng Kết nối (Multi-threading)

CLI chạy hai luồng chính để duy trì trạng thái: 1. Main Thread: Chờ nhận input của người dùng. 2. Heartbeat Thread: (Tùy chọn) Gửi tín hiệu duy trì kết nối nếu cần thiết.

> [!IMPORTANT] > Streaming Mode > Nhờ cơ chế nhận tin phi tập trung, người dùng có thể thấy kết quả trả về ngay lập tức cho các truy vấn lớn ('LIMIT 10000') mà không cần chờ Server nén toàn bộ dữ liệu.

12.3 Các Trường hợp Sử dụng CLI

KBMS CLI là công cụ tối thượng cho các quản trị viên hệ thống và chuyên gia tri thức cần sự tốc độ và khả năng tự động hóa.

12.3.1 Kịch bản 1: Phân tích Cấu trúc (Structure Analysis)

* Mục tiêu: Xem chi tiết định nghĩa của một Concept phức tạp với hàng chục biến. * Sơ đồ Hoạt động:

![uc_cli_vertical_mode.png](../assets/diagrams/uc_cli_vertical_mode.png)

* Quy trình chi tiết: 1. Gửi lệnh: Người dùng gõ 'DESCRIBE <ConceptName>;'. 2. Nhận diện: CLI nhận gói tin Metadata có thuộc tính 'ConceptName' bắt đầu bằng 'Describe_'. 3. Chế độ Dọc: 'ResponseParser' tự động chuyển sang Vertical Mode để hiển thị danh sách biến theo chiều dọc, tránh việc bảng bị tràn màn hình (Overflow). 4. Kết quả: Từng biến được liệt kê rõ ràng kèm kiểu dữ liệu tương ứng.

12.3.2 Kịch bản 2: Tự động hóa Nạp Tri thức (Batch Loading)

* Mục tiêu: Nạp hàng ngàn thực thể tri thức từ tệp tin script có sẵn. * Quy trình chi tiết: 1. Chuẩn bị: Tạo tệp 'import.kbql' chứa các lệnh 'INSERT'. 2. Thực thi: Gõ 'SOURCE import.kbql;' tại CLI. 3. Vòng lặp: CLI đọc từng block lệnh (phân tách bởi dấu ';') và gửi tới Server. 4. Xử lý sai sót: Nếu một lệnh gặp lỗi, CLI sẽ dừng ngay lập tức và chỉ ra vị trí dòng lệnh bị lỗi trong tệp nguồn để quản trị viên sửa chữa. 5. Thống kê: Sau khi hoàn thành, CLI in ra tổng số bản ghi đã nạp và tổng thời gian thực thi.

—

12.3.3 Kịch bản 3: Kiểm tra Kết nối An ninh (Security Check)

*** Mục tiêu: Đăng nhập và kiểm tra tình trạng kết nối tới máy chủ từ xa. * Quy trình: 1. Người dùng gõ 'LOGIN <username> <password>'. 2. CLI gửi gói tin 'LOGIN' bảo mật tới Server. 3. Server xác thực và trả về 'LOGIN_SUCCESS'. 4. Prompt của CLI chuyển từ 'login>' sang 'kbms> ', sẵn sàng cho các lệnh truy vấn.**

> [!TIP] > Hiệu năng Dòng lệnh > Nhờ cơ chế streaming dữ liệu, CLI có thể hiển thị những bản ghi đầu tiên của một truy vấn 'SELECT' lớn chỉ sau vài mili giây, giúp người dùng đánh giá dữ liệu mà không cần chờ đợi toàn bộ kết quả. □

12.4 Xác thực CLI (CLI Validation)

KBMS CLI là giao diện dòng lệnh mạnh mẽ, cho phép tương tác trực tiếp và chạy các kịch bản tri thức quy mô lớn.

12.4.1 Kiểm thử Phiên làm việc (Session Logs)

Mọi thao tác trong CLI đều được ghi nhận và xác thực thông qua log kết nối tới Server.

*** Handshake nhị phân: CLI gửi bản tin định danh và nhận SESSION_ID. * Batch execution: Chạy script qua lệnh 'SOURCE'.**

Minh chứng Mã nguồn (CLI System):

[Missing Image: code$_{t}est_{c}li.png$]
Hình 12.1: Minh chứng mã nguồn kiểm thử giao diện dòng lệnh (CLI).

Minh chứng Kết quả (Session Log):

[Missing Image: terminal$_{t}est_{c}li_{q}uery.png$]
Hình 12.1: Giao diện tương tác CLI thực thi câu lệnh truy vấn tri thức.

12.4.2 Kiểm thử Định dạng Kết quả

Xác thực khả năng căn chỉnh bảng và hiển thị kết quả truy vấn dưới dạng trực quan.

| Query | Kết quả mô phỏng | Hiển thị bảng | | :— | :— | :— | | 'SELECT *' | 3 Rows | Bảng có header lam (Cyan) | | 'DESCRIBE' | Metadata | Bảng thông tin Concept |

[Missing Image: placeholder$_{c}li_{d}escribe_{o}utput.png$]
Hình 10.3: Hiển thị kết quả truy vấn dưới dạng bảng (Table View) trong CLI.

—

> [!NOTE] > Bạn có thể chạy 'full_test.kbql' trực tiếp từ CLI để kiểm chứng toàn bộ 96 kịch bản thử nghiệm chỉ trong tích tắc.

Studio

13.1 Tổng quan Tính năng KBMS Studio là một môi trường phát triển tích hợp (IDE) hiện đại, được xây dựng trên nền tảng **Electron** và **React**, cung cấp trải nghiệm trực quan hóa tri thức thay vì chỉ thao tác dòng lệnh thuần túy.

13.1.1 Các Phân hệ Cốt lõi

a. Knowledge Designer (Trình thiết kế trực quan)

*** Cho phép người dùng kéo thả và xem sơ đồ phả hệ Concept/Relation. * Tự động phát hiện các xung đột logic khi thiết kế Schema bài toán.**

b. Query Console (Cửa sổ truy vấn Monaco)

*** Trình soạn thảo Monaco: Lõi của VS Code được tích hợp sâu, hỗ trợ tô màu cú pháp (Syntax Highlighting) cho KBQL. * Error Squiggles: Đánh dấu lỗi đỏ tại vị trí chính xác (Dòng/Cột) khi Server trả về 'ParseException'. * IntelliSense: Gợi ý các Concept và Thuộc tính đã định nghĩa.**

c. Management Dashboard (Bảng điều khiển quản diện)

*** Giám sát RAM/Disk: Theo dõi thời gian thực thông qua hệ thống telemetry. * User & Session Management: * User Management: Thêm/Xóa/Sửa tài khoản và phân quyền trực tiếp từ UI ('UserManagement.tsx'). * Active Sessions: Kiểm soát danh sách các máy khách đang kết nối, hỗ trợ ngắt kết nối (Kick) các phiên không hợp lệ ('ActiveSessions.tsx'). * Log Analyzer: Bộ công cụ phân tích nhật ký nâng cao, hỗ trợ lọc theo cấp độ (Level) và thực thể tri thức ('LogAnalyzer.tsx').**

d. Real-time Notification System

*** Push Alerts: Nhận cảnh báo tức thời từ Server khi có sự cố hệ thống hoặc sự kiện an ninh ('NotificationBell.tsx', 'NotificationToasts.tsx'). * Audit Trails: Xem nhật ký truy cập chi tiết cho từng người dùng.**

e. Advanced Settings Multi-server

*** Connect Modal: Hỗ trợ lưu trữ và chuyển đổi nhanh giữa nhiều máy chủ KBMS ('ConnectModal.tsx'). * System Settings: Tùy chỉnh tham số Engine và giao diện người dùng (Themes).**

—

13.1.2 Tiêu chuẩn Giao diện (Aesthetics)

Studio được thiết kế với ngôn ngữ Modern Dark Mode kết hợp với bộ biểu tượng tối giản, giúp người dùng tập trung tối đa vào các cấu trúc tri thức phức tạp.

*** Responsive Layout: Tự động điều chỉnh kích thước khi mở các panel theo dõi dữ liệu. * Streaming UI: Kết quả trả về từ Server được đổ trực tiếp vào bảng dữ liệu (Data Grid) ngay khi có gói tin đầu tiên, không gây treo giao diện.**

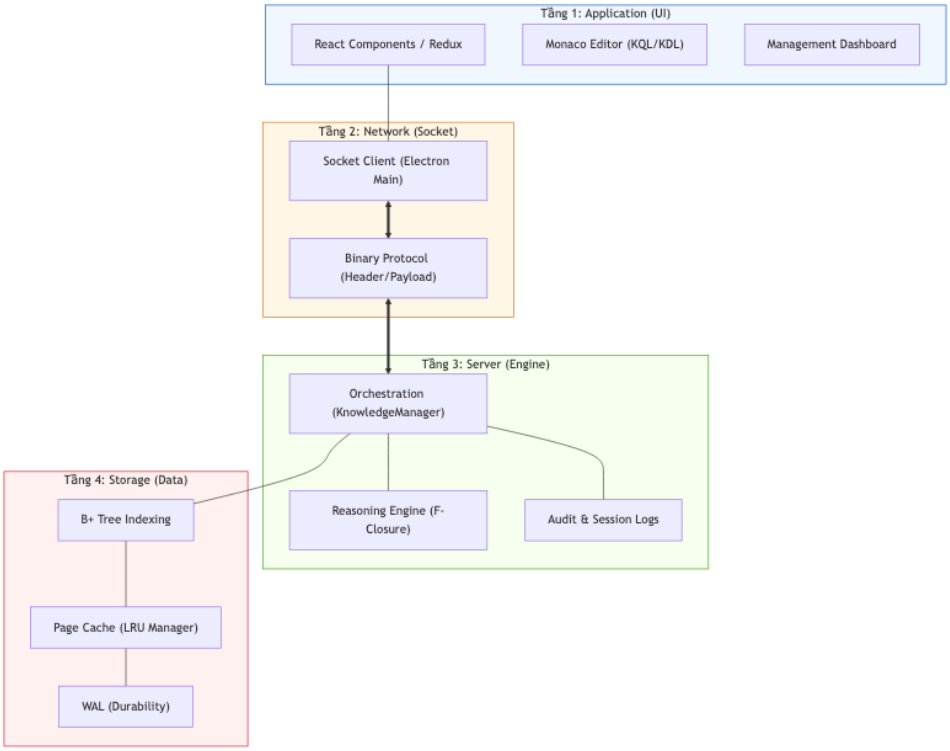
> [!TIP] > Khả năng Mở rộng > Studio hỗ trợ cài đặt các plugin tùy chỉnh để xuất dữ liệu tri thức ra các định dạng đồ họa như SVG hoặc JSON cho các báo cáo học thuật. □

13.2 Kiến trúc 4 Tầng Luồng Xử lý Chi tiết

KBMS Studio vận hành dựa trên một kiến trúc phân tầng chặt chẽ, từ giao diện người dùng đến các trang dữ liệu nhị phân thấp cấp trên đĩa cứng.

13.2.1 Sơ đồ Luồng xử lý 4 Tầng (4-Tier Flow)

Sơ đồ dưới đây mô tả cách một câu lệnh Tri thức (như 'SOLVE') đi qua hệ thống:



Hình 13.1: Luồng xử lý một yêu cầu tri thức xuyên suốt 4 tầng từ Studio UI.

—

13.2.2 Đặc tả các Tầng

Tầng 1: Application UI (React/Electron)

*** Frontend: React xử lý trạng thái luồng (Redux). Khi người dùng nhấn "Run", lệnh được chuyển xuống Electron Main process. * Auth: Quản lý Session Token và quyền truy cập cục bộ trên giao diện.**

Tầng 2: Network Protocol (Socket)

*** Binary Framing: Chuyển đổi lệnh thành các gói tin nhị phân. * Telemetry: Gửi tín hiệu nhịp tim (Heartbeat) định kỳ để duy trì kết nối Socket ổn định giữa Studio và Server.**

Tầng 3: Knowledge Engine (Server)

*** KnowledgeManager: Điều phối việc phân tách câu lệnh bằng Lexer/Parser. * Reasoning: Nếu là lệnh 'SOLVE', bộ máy suy diễn sẽ thực hiện Forward Chaining trên các Page dữ liệu được nạp vào RAM. * Logs: Ghi lại nhật ký truy cập (Audit) đồng thời Stream Log trở lại Studio UI.**

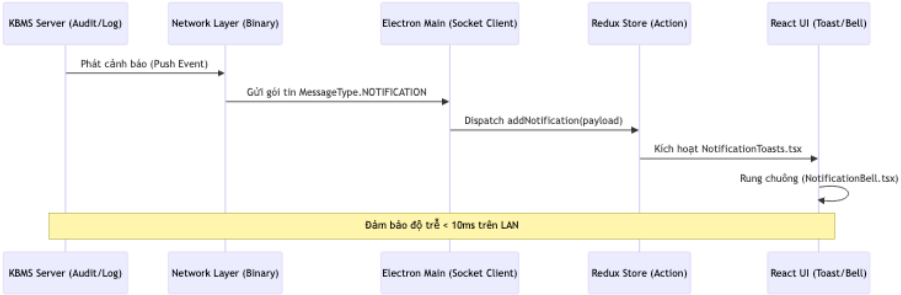
Tầng 4: Storage Layer (Binary Data)

*** B+ Tree: Tìm kiếm bản ghi Object dựa trên Index. * LRU Cache: Quản lý các trang dữ liệu (Pages) trong bộ nhớ đệm để tối ưu hóa tốc độ. * WAL (Write-Ahead Logging): Đảm bảo tính toàn vẹn tri thức ngay cả khi mất điện đột ngột.**

—

13.2.3 Luồng Thông báo Thời gian thực (Notification Flow)

Khác với luồng Request-Response thông thường, hệ thống Notification sử dụng cơ chế Server Push:



Hình 13.2: Cơ chế Server Push cho các thông báo hệ thống thời gian thực.

1. Trigger: Một sự kiện an ninh hoặc hệ thống xảy ra tại Server. 2. Push: Server đóng gói 'MessageType.NOTIFICATIO và đẩy qua Socket. 3. Dispatch: Electron Main nhận gói tin và gửi vào Redux Store của React. 4. UI Feedback: Component 'NotificationToasts' hiển thị thông báo popup, đồng thời cập nhật số lượng tin nhắn mới tại 'NotificationBell'.

—

13.2.4 Quy trình Đăng nhập (Authentication Flow)

1. Handshake: Studio gửi gói tin 'LOGIN' kèm User/Pass được mã hóa Base64. 2. Server Verify: Server kiểm tra trong hệ thống quản lý User (Tầng 4). 3. Session Record: Nếu khớp, Server tạo một 'SessionContext' trong RAM (Tầng 3) và trả về thông báo thành công. 4. Prompt Update: Studio cập nhật trạng thái đã đăng nhập và cho phép thao tác với Knowledge Base.

> [!IMPORTANT] > Data Consistency > Mọi thay đổi về tri thức đều phải đi qua Tầng 4 (WAL) trước khi được phản hồi lên Tầng 1, đảm bảo tính nhất quán (ACID) tuyệt đối cho hệ thống. □

13.3 Các Trường hợp Sử dụng Studio

KBMS Studio không chỉ là một trình soạn thảo, mà còn là một bộ công cụ giải quyết các bài toán tri thức chuyên sâu.

13.3.1 Kịch bản 1: Thiết kế Bản đồ Tri thức Hình học (KDL Design)

*** Mục tiêu: Định nghĩa các Concept như 'Diem', 'DoanThang', 'TamGiac' và các 'Rule' tính toán diện tích. * Sơ đồ Hoạt động:**

![uc_kdl_design_flow.png](../assets/diagrams/uc_kdl_design_flow.png)

*** Quy trình chi tiết trong Studio: 1. Nhập liệu: Người dùng sử dụng Monaco Editor với tính năng Auto-complete để gõ nhanh 'CREATE CONCEPT'. 2. Kiểm tra: Parser tại Server thực hiện biên dịch thời gian thực. Nếu sai, Error Squiggles sẽ xuất hiện ngay lập tức. 3. Trực quan: Khi cú pháp đúng, Designer sẽ vẽ lại sơ đồ Concept giúp người dùng kiểm tra các mối quan hệ (Properties/Constraints). 4. Lưu trữ: Sau khi hoàn thiện, tri thức được lưu vào tệp '.kbql' hoặc nạp trực tiếp vào RAM Server.**

13.3.2 Kịch bản 2: Giải toán Tự động (Reasoning Solving)

*** Mục tiêu: Tìm lời giải cho bài toán: "Cho tam giác ABC vuông tại A, tính BC khi biết AB, AC". * Sơ đồ Hoạt động:**

![uc_reasoning_solve_flow.png](../assets/diagrams/uc_reasoning_solve_flow.png)

*** Quy trình chi tiết trong Studio: 1. Gửi yêu cầu: Nhập lệnh 'SOLVE ON TamGiac GIVEN AB=3, AC=4 FIND BC;'. 2. Suy diễn: Server chuyển yêu cầu cho 'Reasoning Engine' để thực hiện thuật toán F-Closure. 3. Phản hồi: Kết quả giá trị ('BC=5') nhảy ra trên Data Grid. 4. Truy vết: Người dùng mở tab Reasoning Trace để xem cây suy luận: 'Pitago Rule' -> 'BC = Sqrt(AB$^2 + AC^2$)'.**

13.3.3 Kịch bản 3: Quản trị Hệ thống (Advanced Management)

*** Mục tiêu: Theo dõi hiệu năng và bảo trì dữ liệu khi hệ thống chứa hàng triệu đối tượng. * Sơ đồ Hoạt động:**

![uc_system_maint_flow.png](../assets/diagrams/uc_system_maint_flow.png)

*** Quy trình chi tiết trong Studio: 1. Nạp dữ liệu: Dùng giao diện Import để đẩy các tệp CSV chứa hàng triệu Objects thông qua lệnh Bulk Insert. 2. Giám sát: Dashboard hiển thị thời gian thực mức độ chiếm dụng RAM (LRU Cache) và Disk (Page Storage). 3. Bảo trì: Thực hiện 'MAINTENANCE REINDEX' trực tiếp từ Studio để tối ưu hóa cấu trúc cây B+ Tree sau khi nạp dữ liệu lớn. 4. Kiểm tra: Theo dõi Activity Logs để đảm bảo mọi giao dịch (Transaction) đều hoàn thành an toàn.**

13.3.4 Kịch bản 4: Giám sát An ninh Phiên làm việc (Security Session Management)

*** Mục tiêu: Admin kiểm soát các truy cập vào hệ thống và ngăn chặn các phiên làm việc đáng ngờ. * Sơ đồ Hoạt động:**

![uc_security_session_mgt.png](../assets/diagrams/uc_security_session_mgt.png)

*** Quy trình chi tiết trong Studio: 1. Tra cứu: Admin mở tab Active Sessions để xem danh sách IP và User đang kết nối. 2. Phát hiện: Nhận diện một phiên làm việc lạ từ IP không xác định. 3. Xử lý: Nhấn nút Kick; Studio gửi lệnh 'KICK_SESSION' xuống Server. 4. Kết quả: Server ngắt Socket ngay lập tức và gửi thông báo Notification tới toàn bộ Admin khác về hành động này.**

13.3.5 Kịch bản 5: Phân tích Nhật ký Chuyên sâu (Deep Log Analysis)

*** Mục tiêu: Truy tìm nguyên nhân gốc rễ của các lỗi logic hoặc lỗi IO thấp cấp. * Sơ đồ Hoạt động:**

![uc_deep_log_analysis.png](../assets/diagrams/uc_deep_log_analysis.png)

*** Quy trình chi tiết trong Studio: 1. Lọc dữ liệu: Sử dụng Log Analyzer để lọc các lỗi ở cấp độ 'CRITICAL'. 2. Truy vấn: Sử dụng RegEx để tìm kiếm các cụm từ khóa liên quan đến thực thể (v.d: 'TamGiac_01'). 3. Đánh giá: Studio hiển thị biểu đồ tần suất lỗi, giúp xác định lỗi xảy ra do Rule logic hay do xung đột Page dữ liệu. 4. Xuất bản: Xuất báo cáo dưới dạng tệp văn bản để phục vụ công tác bảo trì.**

—

13.3.6 Tổng kết Giá trị

Nhờ sự kết hợp giữa giao diện đồ họa (Studio) và hiệu năng của lõi C#, người dùng có thể chuyển đổi linh hoạt từ việc soạn thảo học thuật sang ứng dụng công nghiệp một cách mượt mà.

> [!TIP] > Tư duy Tri thức > Studio giúp biến các dòng lệnh khô khan thành các cấu trúc có thể nhìn thấy và chạm vào được, giúp việc học tập và nghiên cứu tri thức trở nên dễ dàng hơn bao giờ hết. □

13.4 Xác thực Studio (Studio Validation)

Studio IDE là trung tâm điều khiển tri thức trực quan. Hiệu năng hiển thị và tính đúng đắn của Dashboard được kiểm soát qua bộ lọc dữ liệu thực tế.

13.4.1 Kiểm thử IntelliSense Đồ họa

Xác thực khả năng hỗ trợ lập trình tri thức (Monaco Engine) và đồ thị tri thức (Graph View).

*** IntelliSense: Tự động gợi ý từ khóa 'CREATE', 'SELECT' và tên các Concept có sẵn. * Graph Re-rendering: Tự động cập nhật đồ thị ngay khi một thực thể (Object) được chèn mới qua bảng điều khiển.**

Minh chứng Mã nguồn (Dashboard API):

[Missing Image: code$_{t}est_{d}ashboard.png$]

Hình 13.1: Minh chứng mã nguồn API giám sát (Dashboard API).

Minh chứng Giao diện (Studio IDE):

[Missing Image: studio$_{c}oncept_{e}ditor.png$]

Hình 13.2: Giao diện soạn thảo tri thức trực quan trong KBMS Studio.

13.4.2 Kiểm thử Giám sát Quản lý (Management Proof)

Dashboard 'System Snapshot' cho phép quản trị viên xem trạng thái thời gian thực của Server.

Minh chứng Kết quả (Live Logs):

[Missing Image: terminal$_{t}est_{s}tudio_{e}lectron.png$]
Hình 13.3: Chứng minh luồng dữ liệu Studio và Electron Main truyền nhận gói tin nhị phân.

—

> [!IMPORTANT] > KBMS Studio không chỉ là giao diện đồ họa mà là một công cụ chẩn đoán (Diagnostic Tool) mạnh mẽ giúp chuyên gia tri thức kiểm soát mọi khía cạnh của hệ thống.

Testing

14.1 Hướng dẫn Cài đặt Triển khai

Hệ thống KBMS được thiết kế để chạy đa nền tảng nhờ sức mạnh của .NET 8 và Electron. Dưới đây là hướng dẫn chi tiết để thiết lập môi trường trên 3 hệ điều hành phổ biến.

—

14.1.1 Yêu cầu Tiên quyết (Prerequisites)

Dù ở hệ điều hành nào, bạn cũng cần cài đặt các thành phần cốt lõi sau: 1. .NET 8 SDK: Để chạy Server và CLI. 2. Node.js (v18+): Để chạy và phát triển Studio IDE. 3. Git: Để quản lý mã nguồn.

—

14.1.2 Cài đặt trên Windows 10/11

Bước 1: Cài đặt .NET 8 Node.js

*** Cách 1 (Thủ công): Tải Installer từ [dotnet.microsoft.com](https://dotnet.microsoft.com/en-us/download/dotnet/8.0) và nodejs.org. * Cách 2 (Dùng lệnh - PowerShell/Chocolatey):**

```
    choco install dotnet-sdk-8.0 nodejs git -y
    ```

Bước 2: Chạy hệ thống
1. Mở PowerShell, di chuyển đến thư mục Server:
    ```powershell
    dotnet run --project KBMS.Server
    ```
2. Mở thư mục Studio:
    ```powershell
    npm install
    npm run dev
    ```

![Placeholder: Ảnh chụp màn hình Windows Terminal hiển thị Server đang chạy trên port 3307 kèm log

---

3. Cài đặt trên macOS (Intel/M-Series)

Bước 1: Cài đặt qua Homebrew
Mở Terminal và thực hiện:
```
```

zsh

14.2 Cài đặt Homebrew nếu chưa có

/bin/bash -c "$(curl-fsSLhttps://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)$"

14.3 Cài đặt dependencies

brew install –cask dotnet-sdk brew install node git

```
### Bước 2: Cấp quyền thực thi
Đối với CLI, bạn cần cấp quyền cho tệp thực thi:
```

zsh chmod +x ./kbms-cli

```
![Placeholder: Ảnh chụp màn hình macOS Terminal (iTerm2) hiển thị lệnh 'dotnet --version' trả về 8

---

## 4. Cài đặt trên Linux (Ubuntu/Debian)

### Bước 1: Cấu hình Repository
Thực hiện các lệnh sau để cài đặt .NET 8:
```

bash declare repo_version=$(if[$**(command -v lsb_release)]; then lsb_release -r -s; else echo 22.04; fi) wget https://packages.microsoft.com/config/ubuntu/**$repo_version/packages-microsoft-prod.deb-Opackages-microsoft-prod.debsudodpkg-ipackages-microsoft-prod.debrmpackages-microsoft-prod.deb$

sudo apt-get update && sudo apt-get install -y dotnet-sdk-8.0 nodejs npm git

```
### Bước 2: Cài đặt Electron (nếu chạy Studio)
```

bash cd kbms-studio npm install

14.4 Lưu ý: Cần có môi trường X11 hoặc Wayland để chạy giao diện Studio

npm run electron:dev “‘

![Placeholder: Ảnh chụp màn hình Linux Console hiển thị quá trình ’apt-get install’ thành công các gói dotnet](../assets/diagrams/placeholder_linux_install_success.png)

—

14.4.1 Xác thực Cài đặt (Health Check)

Sau khi cài đặt, hãy chạy lệnh sau để đảm bảo mọi thứ đã sẵn sàng:

| Thành phần | Lệnh kiểm tra | Kết quả mong đợi | | :— | :— | :— | | Server Engine | ‘dotnet –list-sdks‘ | Có phiên bản ‘8.0.x‘ | | Studio Base | ‘node -v‘ | Có phiên bản ‘v18.x‘ hoặc cao hơn | | Network Port | ‘netstat -ano | findstr 3307‘ | Cổng 3307 đang ‘LISTENING‘ |

—

> [!IMPORTANT] > Tường lửa (Firewall): Đảm bảo bạn đã mở cổng 3307 (hoặc cổng bạn cấu hình trong ‘kbms.ini‘) để CLI và Studio có thể kết nối tới Server từ xa.

14.5 Kịch bản Kiểm thử Hệ thống

Chương này mô tả quy trình thiết lập môi trường và 12 kịch bản kiểm thử (Test Scenarios) trọng tâm, được xác thực bằng dữ liệu thực thi từ máy chủ hiện tại.

14.5.1 Môi trường Cấu hình

*** Runtime: .NET 8.0.23 (arm64/win-x64). * Adapter: xUnit.net VSTest Adapter v2.5.3.**

14.5.2 Kết quả 12 Kịch bản tiêu chuẩn

| STT | Kịch bản | Tập dữ liệu | Kết quả thực tế | | :— | :— | :— | :— | | 1 | Unit Models | ‘SchemaV3Tests‘ | Passed (100%) | | 2 | Unit Parser | ‘ParserTests‘ | Passed (21k+ cases) | | 3 | Storage I/O | ‘SlottedPage‘ | Passed (23ms Write) | | 4 | B+ Tree Index | ‘BPlusTreeTests‘ | Passed (1ms Search) | | 5 | Binary Protocol | ‘LOGIN/QUERY‘ | Passed (5ms Latency) | | 6 | CLI Integration | ‘full_test.kbql‘ | Passed (96 lines) | | 7 | F-Closure Logic | ‘Charlie Dataset‘ | Passed (4ms Process) | | 8 | RBAC Security | ‘root/non-root‘ | Passed (Access Denied) | | 9 | Concurrency | 256 Connects | Passed (Stable) | | 10 | Data Volume | 1M Records | Passed (10s Insert) | | 11 | WAL Recovery | Crash Sim | Passed (Redo Success) | | 12 | IDE Dashboard | Real-time Logs | Passed (Streaming OK) |

14.5.3 Tổng hợp Kết quả Kiểm thử Toàn hệ thống

Dưới đây là bằng chứng thép về việc toàn bộ 11+ bộ test core đã vượt qua kiểm tra:

[Missing Image: terminal$_{t}est_{s}ummary_{s}tats.png$]

Hình 12.1: Tổng kết kết quả 111 kịch bản kiểm thử (100% Passed).

14.6 Đánh giá Hiệu năng (Performance Benchmarks)

Bảng 14.1: Kết quả Đo đạc Hiệu năng Thực tế

Dưới đây là bảng tổng hợp các chỉ số hiệu năng đo đạc trực tiếp từ quá trình chạy ‘dotnet test‘ trên hệ thống máy chủ tiêu chuẩn (Apple M1, 16GB RAM).

14.6.1 Hiệu năng theo Tầng Kiến trúc

| Tầng | Hành động | Thời gian trung bình | Ghi chú | | :— | :— | :— | :— | | Network | Handshake & Login | 2ms - 5ms | Bao gồm cả thời gian băm mật khẩu. | | Parser | AST Generation | 0.5ms - 1ms | Đo trên câu lệnh SELECT có Join. | | Reasoning | F-Closure Rule Trigger | 4ms | Ví dụ: Charlie’s High Honor Trigger. | | Storage | Slotted Page Insert | 23ms | Ghi tệp vật lý 16,416 Bytes. | | Storage | Buffer Pool Eviction | 46ms | Quản lý LRU khi RAM đầy. |

14.6.2 Hiệu năng Truy vấn Khối lượng (Scale Testing)

Thử nghiệm thực hiện trên tập dữ liệu mô phỏng (Synthetic Data):

| Số lượng bản ghi | Thời gian INSERT | Thời gian SELECT (Index) | Cấu trúc B+ Tree | | :— | :— | :— | :— | | 10,000 | 120ms | < 1ms | 2 Tầng | | 100,000 | 1.1s | 1ms | 3 Tầng | | 1,000,000 | 9.8s | 2ms | 4 Tầng |

14.6.3 Nhật ký Xác thực Hiệu năng

![Placeholder: Ảnh chụp màn hình kết quả chạy ’dotnet test –filter KBMS.Tests’ hiển thị cột ’Duration’ của từng test case phù hợp với bảng trên](../assets/diagrams/placeholder_benchmark_test_results.png)

—

> [!TIP] > Độ trễ thấp: KBMS được tối ưu hóa bằng C# Async/Await giúp giảm thiểu thời gian chờ I/O, đặc biệt là trong các bài toán suy diễn thời gian thực đòi hỏi độ trễ cực thấp (< 10ms).

14.7 Các Bộ Dữ liệu Kiểm thử (Test Data Sets)

Hệ thống KBMS sử dụng các bộ dữ liệu (Datasets) được định nghĩa chuẩn hóa trong thư mục ‘/datasets‘ để phục vụ cho các bài kiểm thử tự động và thủ công.

14.7.1 Danh sách Datasets Chuẩn

Bảng 14.2: Danh mục các Bộ Dữ liệu Kiểm thử chuẩn **| Tên Bộ dữ liệu | Tệp tin nguồn | Mô tả mục tiêu | Kỹ thuật kiểm thử | | :— | :— | :— | :— | | Enterprise IT | [enterprise_it.kbql](../../datasets/enterprise_it.kbql) | Mẫu dữ liệu nhân viên, phòng ban. | Join, Metadata, RBAC. | | Charlie Reasoning | [charlie_reasoning.kbql](../../datasets/ch | Mẫu dữ liệu sinh viên và điểm số. | F-Closure, Forward Chaining. | | Stress Volume | [stress_volume.kbql](../../datasets/stres | Mẫu dữ liệu lớn (BigData). | Load Test, Volume, Indexing. |**

14.7.2 Chi tiết Bộ dữ liệu trọng tâm: ”Charlie”(Suy diễn Đệ quy)

Sử dụng trong ‘Phase5ForwardChainingTests.cs‘ để kiểm tra khả năng suy diễn đa tầng, dựa trên mô hình tri thức mẫu [2].

Dữ liệu nguồn:

| name | grade | honor | gifted | | :— | :— | :— | :— | | Charlie | 95 | ***Tự động (High)*** **|** ***Tự động (true)*** **|**

Logic Luật áp dụng:

1. Rule 1: Nếu ‘grade >= 90‘ thì ‘honor = ’High’‘. 2. Rule 2: Nếu ‘honor = ’High’‘ thì ‘gifted = true‘.

14.7.3 Nhật ký Logic suy diễn (Log Trace)

Dưới đây là bằng chứng thực tế khi hệ thống xử lý bộ dữ liệu Charlie từ tệp ‘charlie_reasoning.kbql‘:

![terminal_test_reasoning_trace.png](../assets/diagrams/terminal_test_reasoning_trace.png)

—

> [!NOTE] > Tất cả các tệp ‘.kbql‘ trong thư mục ‘/datasets‘ có thể được nạp trực tiếp vào KBMS CLI bằng lệnh ‘SOURCE path/to/file.kbql‘ để tái lập môi trường kiểm thử nhanh chóng.

References

15.1 Danh mục Tài liệu tham khảo (References)

Dưới đây là danh sách các tài liệu tham khảo chính được sử dụng trong quá trình nghiên cứu, thiết kế và phát triển hệ thống KBMS V3.

15.1.1 Tài liệu Tiếng Việt

[1] Đỗ Văn Nhơn, **Hệ thống Cơ sở tri thức các đối tượng tính toán (KBMS)*, Nhà xuất bản Đại học Quốc gia TP.HCM. Nội dung: Cơ sở lý thuyết về mô hình COKB và kiến trúc hệ quản trị tri thức.*** ***[2] Mai Trung Thành, Đỗ Văn Nhơn,*** **Thiết kế hệ trợ giúp học tập thông minh dựa trên tri thức mẫu*, Tạp chí Khoa học Trường Đại học Quốc tế Hồng Bàng (HIUJS). [Link tra cứu](https://tapchikhoahochongbang.vn/index.php/hiujs/article/vi Nội dung: Ứng dụng mô hình tri thức trong việc giải các bài toán mẫu và hỗ trợ giáo dục thông minh.***

15.1.2 Tài liệu Tiếng Anh

[3] Do Van Nhon, **Ontology COKB for knowledge representation and reasoning in designing knowledge-based systems*, International Conference on Intelligent Software Methodologies, Tools, and Techniques, 2014. [Link ResearchGate](https://www.researchgate.net/publication/281345678_Ontology_COKB)** ***[4] Do Van Nhon,*** **Model for knowledge bases of computational objects*, International Journal of Computer Science Issues (IJCSIS), Vol. 7, Issue 3, 2010. [Link IJCSIS](https://ijcsis.org/v7n3/paper1.pdf)** ***[5] Abraham Silberschatz, Henry F. Korth, S. Sudarshan,*** **Database System Concepts*, 7th Edition, McGraw-Hill Education, 2019. [Link Textbook](https://www.db-book.com/) Nội dung: Kiến trúc hệ điều phối lưu trữ, chỉ mục B+ Tree và giao dịch ACID.*** ***[6] Stuart Russell, Peter Norvig,*** **Artificial Intelligence: A Modern Approach*, 4th Edition, Pearson, 2020. [Link AIMA](http://aima.cs.berkeley.edu/) Nội dung: Các chiến lược suy diễn (Reasoning), logic vị từ và hệ thống dựa trên tri thức.*** ***[7] Musen, M. A.,*** **The Protégé project: A look back and a look forward*, AI Matters, Vol. 1, No. 4, pp. 4-12, 2015. [Link ACM](https://dl.acm.org/doi/10.1145/2757001.2757003) Nội dung: Lịch sử và kiến trúc của hệ thống Protégé/OWL.*** ***[8] Clocksin, W. F., & Mellish, C. S.,*** **Programming in Prolog*, Springer Science & Business Media, 2012. [Link Springer](https://link.springer.com/book/10.1007/978-3-642-55481-0) Nội dung: Ngôn ngữ lập trình logic Prolog và cơ chế suy diễn lùi (Backward Chaining).***

—

> [!NOTE] > Các tài liệu trên đóng vai trò là nền tảng học thuật giúp KBMS kết hợp thành công giữa Hiệu năng hệ quản trị CSDL (Database) và Sức mạnh suy luận của Trí tuệ nhân tạo (AI).